AgentStack

Complete Content Package

March 2026

Landing Page · Onboarding Emails · API Docs
Microcopy · SEO Blog Posts · Help Center

Table of Contents

01 Landing Page Copy

02 Onboarding Email Sequence (7 Emails)

03 API Documentation (13 Endpoints)

04 Dashboard Microcopy (63 Items)

05 SEO Blog Posts (5 Articles)

06 Help Center (20 Articles)

SECTION 1

Landing Page Copy

AgentStack Landing Page Copy

> **Status:** Ship-ready | **Version:** 1.0 | **Date:** 2026-03-10
> **Target audience:** Indie developers & agencies building AI workflows
> **Tone:** Confident, technical but approachable, zero corporate fluff

---

SECTION 1: HERO

Headline

82 AI agents. One platform. Zero infrastructure.

Subheadline

AgentStack gives developers and agencies a full roster of pre-built AI agents -- from code generation to design QA to sprint planning -- connected through a visual workflow builder. Bring your own API keys. Pay for the platform, not the markup.

Primary CTA

Start building free

Secondary CTA

See how it works

Trust signal (below CTAs)

No credit card required. 5 agents included on the free plan.

SECTION 2: THE PROBLEM

Section header

Building AI agents from scratch is a full-time job you didn't sign up for.

Pain point 1

You're writing glue code instead of shipping product.
You wanted AI-powered workflows. What you got was six weeks of prompt engineering, retry logic, output parsing, and duct-taping LangChain to a cron job. Your backlog hasn't moved.

Pain point 2

Every agent is a custom project.
Need a code reviewer? Build it. Need a copywriter agent? Build another one. Need them to talk to each other? Now you're building an orchestration layer. You're not a product team anymore -- you're an AI infrastructure team.

Pain point 3

You're paying for someone else's margin on API calls.
Most agent platforms wrap OpenAI and charge you 3-5x the token cost. You're subsidizing their GPU bill every time your agent runs. And you can't even swap to a cheaper model without migrating off the platform.

Closing line

There's a better way to run AI agents -- one where you keep control of your keys, your models, and your margin.

SECTION 3: THE SOLUTION

Section header

AgentStack: the agent platform that stays out of your way.

Solution overview

AgentStack is a ready-to-deploy platform with 82 specialized AI agents, a visual workflow builder, and a bring-your-own-key architecture. You pick the agents, wire them together, and run production workflows -- without writing orchestration code or paying a per-token tax.

Value Prop 1

82 agents, ready on day one.
Every agent is purpose-built for a specific job: writing backend code, auditing accessibility, planning sprints, generating landing page copy. No prompt engineering required. Pick an agent, give it context, and let it work.

Value Prop 2

Visual workflow builder -- no orchestration code.
Drag agents onto a canvas. Connect inputs to outputs. Add conditional logic, parallel branches, and human-in-the-loop checkpoints. Ship multi-agent workflows without writing a scheduler, a queue, or a state machine.

Value Prop 3

Bring your own keys. Keep your margin.
Plug in your OpenAI, Anthropic, Google, Mistral, or Groq API keys. You pay the model providers directly at their published rates. AgentStack charges for the platform -- never for your tokens.

Value Prop 4

Works with the stack you already use.
AgentStack connects to GitHub, Linear, Slack, Figma, Stripe, and dozens more through native integrations. Agents read from and write to your existing tools -- no middleware, no Zapier tax.

Value Prop 5

Full execution visibility.
Every agent run is logged: inputs, outputs, token usage, latency, cost. Drill into any step of any workflow. Debug a failed run in seconds, not hours. Know exactly what your agents did and what they spent.

SECTION 4: THE 82 AGENTS

Section header

82 specialized agents. 9 categories. Every role your AI team needs.

Section subheader

Each agent is fine-tuned for a specific job -- not a generic "assistant" with a system prompt. They understand context, follow best practices, and produce output you can actually ship.

ENGINEERING (18 agents)

From first commit to production deploy.

AGENT	WHAT IT DOES
Backend Systems Engineer	Generates production-grade backend code -- APIs, services, data models, database queries -- across Python, Node, Go, and Rust.
Frontend Code Generator	Writes React, Next.js, Vue, and Svelte components from specs or designs, with proper state management and accessibility.
Code Reviewer	Reviews pull requests for bugs, performance issues, security vulnerabilities, and style violations. Returns structured, actionable feedback.
Code Architect	Designs system architecture: service boundaries, data flow, API contracts, and infrastructure topology for new features or greenfield projects.
Debugger	Analyzes stack traces, error logs, and reproduction steps to pinpoint root causes and suggest targeted fixes.
Test Writer	Generates unit, integration, and e2e tests from source code or specs. Covers happy paths, edge cases, and failure modes.
Codebase Refactor Analyst	Identifies tech debt, dead code, and structural issues. Produces prioritized refactoring plans with effort estimates.
Performance Engineering Lead	Audits frontend and backend performance: bundle size, Core Web Vitals, query latency, caching strategy, and load testing.

AGENT	WHAT IT DOES
Database Migration Engineer	Writes and sequences schema migrations across Prisma, Drizzle, TypeORM, Alembic, and raw SQL with zero-downtime strategies.
Data Pipeline ETL Engineer	Designs and implements ETL/ELT pipelines using dbt, Airflow, Dagster, and Prefect with data quality checks built in.
Mobile Engineering Specialist	Builds React Native, Flutter, Swift, and Kotlin components with platform-specific optimizations and native module integration.
Desktop Engineering Specialist	Develops Electron and Tauri desktop apps with native OS integration, auto-update systems, and cross-platform packaging.
API Contract Spec Writer	Generates OpenAPI, GraphQL, and gRPC specs from requirements. Produces client SDKs, mock servers, and validation schemas.
Prompt Engineer	Designs, tests, and optimizes prompts and prompt chains for LLM-powered features. Includes evaluation harnesses and regression tests.
Localization Engineer	Implements i18n frameworks (i18next, react-intl, next-intl), ICU MessageFormat, CLDR pluralization, RTL layouts, and TMS integration.
Contract Testing Specialist	Sets up consumer-driven contract tests with Pact, PactFlow, and Spring Cloud Contract. Produces compatibility matrices and CI configs.
Analytics Event Tracking Engineer	Designs tracking plans and instruments events across Segment, Mixpanel, PostHog, Amplitude, and GA4 with consent management.
Data Architecture Engineer	Designs data models, warehouse schemas, and data governance frameworks across PostgreSQL, MongoDB, Snowflake, and BigQuery.

DESIGN (7 agents)

From wireframe to pixel-perfect handoff.

AGENT	WHAT IT DOES
UI/UX Design Lead	Produces full-fidelity design specs: layout, spacing, typography, color, interaction states, and responsive breakpoints.
Brand Identity Designer	Creates brand systems: logos, color palettes, typography scales, iconography guidelines, and brand usage documentation.

AGENT	WHAT IT DOES
Wireframe & Prototype Specialist	Converts PRDs and user stories into lo-fi, mid-fi, and hi-fi wireframes with interaction notes and clickable prototype specs.
Design System Architect	Manages design tokens, component libraries, theming, and cross-platform consistency across Figma, web, and mobile.
Design QA Specialist	Validates implemented UI against Figma specs. Catches spacing, color, typography, and brand violations across all breakpoints.
Motion & Interaction Designer	Specs animations, transitions, micro-interactions, and loading states with timing curves, easing functions, and implementation notes.
Accessibility QA Specialist	Runs WCAG 2.1/2.2 AA audits: screen reader testing, keyboard nav, ARIA validation, color contrast, and focus management.

PRODUCT (6 agents)

From idea to shipped feature.

AGENT	WHAT IT DOES
AI Product Studio PM	Owns the product lifecycle: writes PRDs, defines success metrics, prioritizes backlogs, and coordinates cross-functional delivery.
Requirements Analyst	Translates stakeholder input into structured requirements: functional specs, acceptance criteria, edge cases, and dependency maps.
User Story Writer	Generates user stories with acceptance criteria, test scenarios, and technical notes -- ready for sprint planning.
Product Metrics Architect	Defines KPI frameworks, instrumentation plans, and dashboard specs for measuring feature impact and product health.
Onboarding Specialist	Designs onboarding flows: step sequences, progress indicators, tooltip tours, activation milestones, and drop-off analysis.
UX Research Specialist	Creates research plans, interview scripts, survey designs, usability test protocols, and synthesizes findings into actionable insights.

INFRASTRUCTURE & DEVOPS (7 agents)

From Terraform plan to incident resolved.

AGENT	WHAT IT DOES
Terraform IaC Engineer	Writes production-ready Terraform, Pulumi, and CDK configs for networking, compute, databases, and secrets management.
Cloud Infrastructure Architect	Designs multi-cloud architectures across AWS, GCP, and Azure: VPCs, load balancers, auto-scaling, and disaster recovery.
DevOps Release Agent	Manages CI/CD pipelines, deployment strategies (blue-green, canary, rolling), artifact management, and release gating.
Incident Response Commander	Handles the full incident lifecycle: SEV classification, runbook execution, status page updates, and blameless postmortems.
FinOps Cost Optimization	Analyzes cloud spend, identifies rightsizing opportunities, models reserved instance savings, and produces cost allocation strategies.
MLOps Engineer	Manages model versioning, deployment pipelines (SageMaker, Vertex AI), drift monitoring, and LLM serving infrastructure.
Performance Engineering Specialist	Profiles full-stack performance: API latency, database queries, caching layers, and load testing with implementation-ready fixes.

QA & SECURITY (5 agents)

From test plan to security audit.

AGENT	WHAT IT DOES
QA & Security Gatekeeper	Enforces quality gates across the pipeline: test coverage thresholds, security scan results, and release readiness checks.
Feature Test Execution Specialist	Runs structured test passes against feature specs: functional, regression, boundary, and cross-browser/cross-device testing.
AppSec Audit Engineer	Performs application security audits: OWASP Top 10 review, dependency scanning, secrets detection, and remediation guidance.
Senior Cross-Agent Reviewer	Reviews output from other agents for accuracy, consistency, and quality. Acts as the final check before human handoff.
Legal Compliance Advisor	Evaluates features for GDPR, CCPA, SOC 2, HIPAA, and other regulatory requirements. Flags risks and suggests compliant alternatives.

GROWTH & MARKETING (7 agents)

From content strategy to conversion optimization.

AGENT	WHAT IT DOES
Content & Copy Strategist	Writes UX microcopy, landing pages, email sequences, and content briefs. Translates technical capabilities into clear, human language.
SEO Strategist	Performs keyword research, writes meta tags, optimizes content structure, builds internal linking strategies, and tracks ranking performance.
Social Media Content Manager	Creates platform-specific content calendars, writes posts, generates hashtag strategies, and produces engagement analytics.
Email Marketing Specialist	Builds email sequences: welcome, activation, retention, re-engagement, and transactional. Writes subject lines, body copy, and CTAs.
Growth Experiments Strategist	Designs A/B tests, growth loops, and activation experiments. Defines hypotheses, success metrics, and statistical significance thresholds.
GTM Sellability Analyst	Evaluates product-market fit signals, positioning, competitive differentiation, and go-to-market readiness.
Pricing & Monetization Strategist	Models pricing tiers, analyzes willingness-to-pay, designs freemium gates, and benchmarks against competitors.

RESEARCH & STRATEGY (5 agents)

From market signal to strategic insight.

AGENT	WHAT IT DOES
Market Validation Analyst	Validates product ideas against market data: TAM/SAM/SOM analysis, demand signals, and viability scoring.
Competitor Intelligence Analyst	Monitors competitor products, pricing, positioning, and feature releases. Produces structured competitive landscape reports.
Unbuilt Idea Hunter	Scans forums, communities, and social platforms for unmet needs and product opportunities that haven't been built yet.

AGENT	WHAT IT DOES
Reddit Market Intelligence Researcher	Mines Reddit for sentiment, pain points, feature requests, and competitive mentions across relevant subreddits.
Feedback Intelligence Synthesizer	Aggregates user feedback from support tickets, reviews, surveys, and interviews into themed, prioritized insight reports.

COORDINATION (8 agents)

From kickoff to retro.

AGENT	WHAT IT DOES
Studio Workflow Orchestrator	Routes work across agents, manages dependencies, handles failures, and ensures multi-step workflows complete end-to-end.
Research Phase Coordinator	Orchestrates market research, competitor analysis, and user research agents into a unified discovery phase.
PRD Planning Coordinator	Coordinates requirements gathering, story writing, and spec review agents to produce complete, reviewed PRDs.
Design Phase Coordinator	Manages the design pipeline: wireframes to high-fidelity specs to design QA, with handoff artifacts at each stage.
Engineering Phase Coordinator	Orchestrates code generation, review, testing, and deployment agents through a structured build pipeline.
Build Phase Coordinator	Manages the full build lifecycle from architecture through implementation, testing, and merge -- across all engineering agents.
QA Phase Coordinator	Runs the complete QA cycle: test planning, execution, bug triage, regression, and release sign-off.
Sprint Planning Agent	Owns sprint ceremonies: backlog grooming, sprint planning, daily standups, velocity tracking, and retrospective facilitation.

SPECIALIZED (6 agents)

Experts for everything else.

AGENT	WHAT IT DOES
Stack Selection Advisor	Evaluates technology choices: frameworks, databases, hosting, and third-party services. Produces scored comparison matrices.
AI Integration Specialist	Implements LLM integrations: RAG pipelines, embedding strategies, vector databases, function calling, and model evaluation.
Technical Documentation Writer	Writes API references, ADRs, runbooks, READMEs, SDK guides, and changelogs following the Diataxis framework.
Billing & Subscription Manager	Implements Stripe and Paddle integrations: checkout flows, subscription CRUD, webhook handlers, dunning logic, and revenue analytics.
Excel & Sheets Specialist	Builds complex spreadsheets: formulas, pivot tables, charts, data validation, macros, and Google Sheets automations.
App Store Publishing Agent	Manages iOS and Android store submissions: metadata, screenshots, review guidelines compliance, and release notes.

SECTION 5: HOW IT WORKS

Section header

Three steps. That's it.

Step 1: Pick your agents

Browse 82 agents across 9 categories. Each one is built for a specific job -- code review, infrastructure auditing, sprint planning, landing page copy. Select the ones your workflow needs.

Step 2: Build your workflow

Drag agents onto the visual canvas. Connect them in sequence or in parallel. Add conditions, loops, human-approval gates, and data transformations. No YAML. No orchestration code. Just draw it.

Step 3: Execute and monitor

Hit run. Watch every agent execute in real time. See inputs, outputs, token costs, and latency for each step. When something breaks, drill into the exact agent and the exact input that caused it.

CTA

Start building free

SECTION 6: PRICING

Section header

Pay for the platform. Keep your API costs.

Section subheader

Bring your own API keys from OpenAI, Anthropic, Google, Mistral, Groq, or any OpenAI-compatible provider. You pay model providers directly at their published rates. We never mark up your tokens.

FREE -- $0/month

For trying it out.

- 5 agents
- 100 workflow executions per month
- Visual workflow builder
- Community support
- Bring your own keys

CTA: Start building free

PRO -- $49/month

For developers and small teams who ship with AI.

- **All 82 agents**
- Unlimited workflow executions
- Advanced workflow logic (conditions, loops, parallel branches)
- Execution logs with full observability
- Priority support
- All integrations (GitHub, Slack, Linear, Figma, Stripe, and more)
- Bring your own keys

CTA: Start 14-day free trial

Badge: Most popular

TEAM -- $149/month (coming soon)

For agencies and teams who run AI workflows for clients.

- Everything in Pro
- White-label agent outputs
- SSO (SAML/OIDC)
- Team workspaces with role-based access
- Custom agent templates
- Dedicated support channel
- Audit logs
- Bring your own keys

CTA: Join the waitlist

SECTION 7: FAQ

Q1: What does "Bring Your Own Key" actually mean?

You plug in your own API keys from OpenAI, Anthropic, Google, Mistral, Groq, or any OpenAI-compatible provider. AgentStack uses your keys to call the models directly. We never store, share, or resell your keys. You pay the model providers at their published rates -- we don't add a token markup.

Q2: Is my data secure?

Yes. Your data is encrypted in transit (TLS 1.3) and at rest (AES-256). Agent inputs and outputs are processed in isolated execution environments and are not used to train any models. API keys are encrypted with per-user keys and never logged. We're pursuing SOC 2 Type II certification.

Q3: Can I self-host AgentStack?

Not yet. We're focused on the cloud-hosted version for now. Self-hosting is on the roadmap and will be available on the Team plan. Join the waitlist to get notified.

Q4: How is this different from LangChain or CrewAI?

LangChain and CrewAI are frameworks -- they give you building blocks and expect you to write the orchestration, prompts, error handling, and deployment code yourself. AgentStack is a platform -- you get 82 production-ready agents, a visual workflow builder, and full execution monitoring out of the box. No code required to go from idea to running workflow.

Q5: Can I build my own custom agents?

On the Pro plan, you can customize existing agents by adjusting their system prompts, tools, and output formats. Fully custom agent creation (build from scratch) is coming soon on the Team plan.

Q6: How do you handle data privacy and GDPR?

We don't store your data beyond the execution window unless you opt into execution history. All data processing follows GDPR principles: minimal data collection, purpose limitation, and right to deletion. You can purge all execution data at any time from your dashboard.

Q7: Which AI model providers are supported?

OpenAI (GPT-4o, GPT-4, GPT-3.5), Anthropic (Claude 4, Claude 3.5), Google (Gemini 2.0, Gemini 1.5), Mistral (Large, Medium, Small), Groq (Llama, Mixtral), and any provider with an OpenAI-compatible API. We add new providers regularly.

Q8: Can agencies use AgentStack for client work?

Absolutely. The Team plan (coming soon) includes white-label agent outputs, separate client workspaces, and role-based access control. On the Pro plan today, you can run workflows for clients -- you just can't white-label the outputs yet.

Q9: Do I need to write code to use AgentStack?

No. The visual workflow builder lets you build and run multi-agent workflows without writing a single line of code. If you want to go deeper, every agent also has an API you can call programmatically.

Q10: Are there limits on the free plan?

The free plan includes 5 agents and 100 workflow executions per month. Workflows can have up to 5 steps. There are no limits on the number of workflows you can create. To unlock all 82 agents and unlimited executions, upgrade to Pro.

SECTION 8: FOOTER CTA

Headline

Your AI team is ready. Start building.

Body

82 agents. Visual workflows. Your own API keys. No infrastructure to manage, no tokens to overpay for. Go from idea to running AI workflow in minutes.

Primary CTA

Start building free

Secondary CTA

See the docs

Trust signal

No credit card required. Free plan includes 5 agents and 100 executions.

IMPLEMENTATION NOTES

Design direction

- Dark background hero with a workflow builder screenshot or animation
- Agent category cards with expand/collapse on the 82-agent section (don't show all tables at once -- let users drill into categories they care about)
- Pricing cards in a 3-column layout with Pro visually highlighted
- FAQ as an accordion component
- Sticky header CTA that appears after scrolling past the hero

SEO metadata

- **Title tag:** AgentStack -- 82 AI Agents, One Platform, Zero Infrastructure
- **Meta description:** Build AI workflows with 82 pre-built agents, a visual workflow builder, and bring-your-own-key pricing. Start free.
- **H1:** 82 AI agents. One platform. Zero infrastructure.
- **URL slug:** / (homepage)

A/B test candidates

- Hero headline: test "82 AI agents. One platform. Zero infrastructure." vs. "Stop building AI agents. Start using them."
- Primary CTA: test "Start building free" vs. "Deploy your first workflow"
- Pricing anchor: test showing Pro first (centered) vs. showing Free first (left-to-right)

SECTION 2

Onboarding Email Sequence (7 Emails)

AgentStack Onboarding Email Sequence

Product: AgentStack -- AI Agent Platform
Audience: Indie developers & agencies
Tone: Founder-to-user, personal, not marketing-speak
Founder: Sarthak
Emails: 7
Platform: Customer.io / Loops (event-triggered)
Merge Fields: `{first_name}`, `{dashboard_url}`, `{workflow_builder_url}`, `{pricing_url}`, `{execution_count}`, `{agent_count}`, `{feedback_url}`, `{api_docs_url}`, `{upgrade_url}`

Campaign Brief

FIELD	VALUE
Campaign Name	AgentStack Onboarding Sequence v1
Goal	Activation + Trial-to-Paid Conversion
Target Segment	New signups, all plans
Trigger	Account creation event
Emails	7 (Day 0, 1, 3, 5, 7, 14, 30)
KPIs	Open rate > 35%, Click rate > 5%, Activation (first agent run) > 60%, Trial-to-paid > 15%
Exclusions	Users who upgrade to Pro before Email 5 skip the upgrade nudge

A/B Test Plan

EMAIL	VARIABLE	VARIANT A	VARIANT B	WINNER CRITERIA
Email 1	Subject line	"Your AgentStack account is live"	"Welcome -- here's your first agent"	Open rate after 24h

EMAIL	VARIABLE	VARIANT A	VARIANT B	WINNER CRITERIA
Email 5	CTA copy	"See Pro plans"	"Unlock unlimited agents"	Click rate after 48h
Email 7	Send time	9:00 AM user local	2:00 PM user local	Open rate after 48h

---

EMAIL 1 -- Day 0: Welcome

Subject: Welcome to AgentStack -- here's your first agent
Preview Text: 82 agents, your API keys, zero markup. Let's go.

Hey {first_name},

Sarthak here. I built AgentStack, and I'm glad you're trying it.

Here's the short version of what you just signed up for:

AgentStack is a platform with 82 pre-built AI agents. Backend engineers, code reviewers, copywriters, data analysts, DevOps specialists -- real agents that do real work. Not chatbots. Not wrappers around a single prompt.

The thing that makes us different: **Bring Your Own Key (BYOK).**

You plug in your OpenAI, Anthropic, or Google API key. Our agents use it directly. We charge for the platform -- not for the AI. That means:

- No per-token markup. Ever.
- No surprise bills when your usage spikes.
- You pay OpenAI/Anthropic their rate, and us our flat rate. That's it.

Most platforms charge 3-10x markup on tokens. We charge zero.

Your one thing to do right now:

Go to your dashboard and activate your first agent. It takes about 90 seconds.

Activate Your First Agent ->

I'd recommend starting with the **Backend Systems Engineer** or the **Code Reviewer** -- they're the ones most developers try first, and they'll show you what these agents can actually do.

If you hit a wall or something feels off, reply to this email. I read every single one.

Talk soon,
Sarthak

P.S. You don't need to add your API key to explore. But once you do, every agent becomes fully functional. You can add it here: {dashboard_url}/settings/keys

---

EMAIL 2 -- Day 1: First Agent

Subject: Your first agent (2 minutes, I promise)
Preview Text: Most users activate an agent in under 2 minutes. Here's the fastest path.

Hey {first_name},

Quick one.

If you've already activated an agent -- skip this email, you're ahead of the curve.

If you haven't, here's the 2-minute walkthrough:

Step 1: Open your dashboard
Open Dashboard ->

Step 2: Click "Agents" in the sidebar. You'll see 82 agents organized by category -- Engineering, DevOps, QA, Data, Writing, Design, and more.

Step 3: Pick one. I'd start here:

- **Backend Systems Engineer** -- Give it a feature spec. It writes the implementation plan, data models, API endpoints, and error handling. It thinks like a senior engineer, not a code autocomplete.
- **Code Reviewer** -- Paste in a PR diff or a code block. It reviews for bugs, security issues, performance problems, and style -- and explains its reasoning.

Step 4: Give it a real task. Not "hello world." Give it something from your actual backlog. That's when it clicks.

The first time you see an agent produce work you'd normally spend 30-45 minutes on, you'll get it.

Browse All 82 Agents ->

Stuck on which agent to pick? Reply with what you're working on and I'll point you to the right one.

-- Sarthak

EMAIL 3 -- Day 3: First Workflow

Subject: Chain agents together -- this is where it gets powerful
Preview Text: One agent is useful. Five agents in a pipeline is a team.

Hey {first_name},

You've seen what a single agent can do. Now I want to show you what happens when you chain them.

AgentStack has a **visual workflow builder**. You drag agents into a pipeline, connect them, and the output of one becomes the input of the next. No code required.

Think of it like this: instead of one specialist, you're assembling a team.

Here's a real example -- the "Ship a Feature" workflow:

This chains 5 agents together in sequence:

```
1. Sprint Planning Agent
   Input: Feature request or user story
   Output: Broken-down tasks with acceptance criteria
        |
        v
2. Backend Systems Engineer
   Input: Task specs from Step 1
   Output: Implementation plan + code
        |
        v
3. Code Reviewer
   Input: Code from Step 2
   Output: Review with bugs, security flags, improvements
        |
        v
4. Technical Documentation Writer
   Input: Final code + review notes
   Output: API docs, README updates, changelog entry
        |
        v
5. QA Test Specialist
   Input: Code + docs from Steps 2-4
   Output: Test plan, edge cases, integration test suggestions
```

You feed in a feature request. You get back implementation code, a code review, documentation, and a test plan. Five agents. One click.

I've seen agencies use this to cut their feature delivery cycle from 2 days to 2 hours for standard CRUD features.

Try building your first workflow:

Open Workflow Builder ->

Start simple -- chain 2 agents. The Backend Systems Engineer into the Code Reviewer is a great first combo. You can always add more steps later.

The builder is drag-and-drop. If you've used Zapier or n8n, you'll feel right at home.

-- Sarthak

P.S. Workflows run using your own API keys, same as individual agents. No extra cost from us.

EMAIL 4 -- Day 5: BYOK Explainer

Subject: How BYOK saves you hundreds per month (real numbers)
Preview Text: We ran the math on 1M tokens/day. The difference is brutal.

Hey {first_name},

I want to talk about money for a second.

When I was building AgentStack, the thing that frustrated me most about AI platforms was the pricing. You'd use GPT-4o through some wrapper, and they'd charge you 5-8x what OpenAI charges directly. The token markup was the business model.

So we didn't do that. AgentStack uses **Bring Your Own Key (BYOK)**.

Here's exactly how it works:

1. You add your API key from OpenAI, Anthropic, Google, or any supported provider.
2. When an agent runs, it calls the model using YOUR key.
3. You pay the model provider directly at their published rate.
4. AgentStack charges a flat platform fee for the orchestration, agents, and workflows.
5. **We add zero markup to your token costs. None.**

Now here's why this matters. Let me show you real numbers.

Scenario: 1M tokens/day using GPT-4o

GPT-4o pricing (direct from OpenAI): $2.50/1M input, $10.00/1M output
Assuming a 60/40 input/output split:

	TYPICAL AI PLATFORM	AGENTSTACK (BYOK)
Daily token cost	$32 - $48/day (3-5x markup)	$5.50/day (OpenAI direct rate)
Monthly token cost	~$960 - $1,440/month	~$165/month
AgentStack platform fee	N/A	$49/month (Pro plan)
Total monthly cost	**$960 - $1,440**	**$214**
Annual savings	--	**$8,952 - $14,712/year**

That's not a rounding error. That's rent money.

And the gap gets wider the more you use it. At 5M tokens/day, you're looking at $40K-$70K/year in savings.

Why we do this:

Because our business model is the platform, not the margin on someone else's API. We want you to use agents aggressively -- not ration tokens because some middleman is taxing every call.

If you haven't added your API key yet, here's where to do it:

Add Your API Key ->

We support OpenAI, Anthropic (Claude), Google (Gemini), and more. You can switch between providers per agent if you want.

-- Sarthak

P.S. If you're an agency running agents for multiple clients, BYOK means you can use separate keys per client and keep costs completely transparent. No "AI surcharge" on your invoices.

EMAIL 5 -- Day 7: Upgrade Nudge

Subject: You've run {execution_count} agent tasks this week
Preview Text: Here's what unlocks on Pro -- and whether it's worth it for you.

Hey {first_name},

{execution_count} agent executions in your first week. That's solid.

I want to lay out what you get on Free vs. Pro so you can decide if upgrading makes sense for how you're using AgentStack. No pressure -- I'd rather you stay on Free and keep using it than upgrade and not get the value.

Free Plan -- What you have now:
- 10 agent executions per day
- Access to all 82 agents

- 1 workflow (up to 3 agents chained)
- Community support
- BYOK (your keys, zero markup)

Pro Plan -- $49/month:
- Unlimited agent executions
- Unlimited workflows with unlimited agents per chain
- Execution history and analytics (see every run, debug failures, track usage)
- API access -- call any agent or workflow from your own code
- Custom agents -- build your own with custom system prompts and tool configs
- Priority support (direct replies from the team)
- Webhook triggers -- kick off workflows from GitHub, Slack, CI/CD, or any HTTP event
- Team collaboration (up to 5 seats)

The biggest unlock for most developers is **unlimited executions + API access**. Once you can call agents programmatically, AgentStack goes from a tool you visit to infrastructure you build on.

For agencies, it's usually **custom agents + team seats**. You can build agents tailored to each client's stack and share workflows across your team.

If you're hitting the daily execution limit or want API access, Pro is worth it:

See Pro Plan Details ->

If Free is covering what you need, keep going. No expiring trial, no feature degradation. Free is free forever.

-- Sarthak

EMAIL 6 -- Day 14: Check-in

Subject: Quick question (one reply, that's it)
Preview Text: I want to know one thing about your experience so far.

Hey {first_name},

Two weeks in. I have one question:

What's the one thing about AgentStack that's been most confusing or frustrating?

That's it. Hit reply and tell me. One sentence is fine. I read every response and it directly shapes what we build next.

No survey. No NPS score. No "rate us 1-10." Just tell me what's not working.

While I have you -- there are three features that most people haven't found by Day 14 and usually wish they'd known about sooner:

1. Execution History
Every agent run is logged -- input, output, model used, tokens consumed, latency. You can replay, debug, and compare runs. It's like having audit trails for your AI work.

View Your Execution History ->

2. API Access (Pro)
Every agent and workflow has an API endpoint. You can trigger agent runs from your backend, your CI/CD pipeline, a Slack bot -- anything that can make an HTTP request.

```
curl -X POST https://api.agentstack.dev/v1/agents/code-reviewer/run \
  -H "Authorization: Bearer {your_api_key}" \
  -d '{"input": "Review this pull request..."}'
```

Read the API Docs ->

3. Custom Agents
You're not limited to our 82. You can create your own agents with custom system prompts, tool configurations, and model preferences. Build an agent that knows your codebase conventions, your client's brand voice, or your team's deployment checklist.

Create a Custom Agent ->

Seriously though -- reply with what's frustrating you. That's the most useful thing you can do for us right now.

-- Sarthak

EMAIL 7 -- Day 30: Power User Tips

Subject: 5 things power users do with AgentStack
Preview Text: Advanced patterns from devs running 100+ agent executions/week.

Hey {first_name},

You've been on AgentStack for a month. By now you've got the basics down. Here are 5 patterns I see from the developers and agencies getting the most out of the platform -- things that took most people weeks to figure out on their own.

---

1. Chain a Code Reviewer after every code-generating agent

This is the single highest-ROI pattern on the platform.

Any time you have an agent writing code -- Backend Systems Engineer, Frontend Developer, Database Migration Engineer -- add the Code Reviewer as the next step in your workflow.

It catches bugs, security issues, and style problems before you ever look at the output. Think of it as automated PR review baked into your generation pipeline. The Code Reviewer pays for itself in the first run.

---

2. Use webhook triggers to connect workflows to your CI/CD

Set up a webhook trigger on a workflow so it fires automatically when:
- A PR is opened on GitHub (auto-review the diff)
- A deployment fails in CI (auto-analyze the logs with the Incident Response agent)
- A new issue is created in Linear (auto-generate an implementation plan)

Your agents should be reacting to events, not waiting for you to remember to use them.

Set Up Webhook Triggers ->

---

3. Build review loops, not just pipelines

Most people build linear workflows: Agent A -> Agent B -> Agent C.

Power users build loops: Agent A generates code, Agent B reviews it, and if the review flags critical issues, it routes back to Agent A with the feedback for a second pass.

The workflow builder supports conditional branching. Use it. One loop iteration catches 80% of the issues that would have hit your desk.

---

4. Run the Prompt Engineer agent on your custom agents

Built a custom agent but the outputs feel inconsistent? Don't manually tweak the system prompt for hours.

Use the **Prompt Engineer agent** to optimize it. Feed it your custom agent's current system prompt plus 3-5 examples of what you want vs. what you're getting. It will rewrite the prompt with better structure, clearer constraints, and more consistent output formatting.

It's agents improving agents. That's the whole point.

---

5. Stack research agents before execution agents

Before you ask an agent to build something, have a research agent gather context first.

Example workflow for a technical blog post:

```
Research Agent (gathers sources, data, competitor analysis)
    |
    v
Technical Documentation Writer (writes the draft with real context)
    |
    v
Code Reviewer (validates any code snippets in the post)
```

The research step dramatically improves output quality because the downstream agents have real data to work with instead of generating from training data alone.

The mindset shift:

Stop thinking of AgentStack as a tool you use sometimes. Start thinking of it as a team. You're the tech lead. The agents are your senior engineers, writers, reviewers, and analysts. Your job is to delegate well and review their work.

The developers who get the most out of this platform treat it exactly like managing a high-output remote team: clear briefs, review cycles, and automated handoffs.

You've got 82 specialists on the bench. Put them to work.

Open Your Dashboard ->

-- Sarthak

P.S. If you've built a workflow that you're proud of, reply and tell me about it. I feature the best ones in our community showcase -- and the builders get lifetime Pro free.

Sequence Summary

EMAIL	DAY	SUBJECT	GOAL	PRIMARY CTA
1	0	Welcome to AgentStack -- here's your first agent	Orientation + BYOK education	Activate first agent
2	1	Your first agent (2 minutes, I promise)	Activation	Run Backend Engineer or Code Reviewer
3	3	Chain agents together -- this is where it gets powerful	Feature adoption (workflows)	Open workflow builder
4	5	How BYOK saves you hundreds per month (real numbers)	Value education + key setup	Add API key
5	7	You've run {execution_count} agent tasks this week	Conversion	View Pro plan
6	14	Quick question (one reply, that's it)	Feedback + feature discovery	Reply with feedback
7	30	5 things power users do with AgentStack	Retention + depth	Build advanced workflows

Segmentation & Suppression Rules

- **Email 2:** Suppress if user has already run an agent (send alternative "Nice -- here's what's next" email instead)
- **Email 4:** Suppress if user has already added an API key (send alternative "Your key is working" confirmation instead)

- **Email 5:** Suppress if user has already upgraded to Pro
- **All emails:** Suppress if user has unsubscribed from marketing emails (transactional only)

Platform Implementation Notes

Customer.io: Use `track()` events for `agent_activated`, `workflow_created`, `api_key_added`, `plan_upgraded`. Build conditional branches in the campaign to suppress/swap emails based on these events.

Loops: Map merge fields to contact properties. Use loop triggers on signup event. Set delays between emails using the built-in delay blocks.

Resend: Not recommended as the primary platform for this sequence (campaign/drip support is limited). Use for transactional emails (welcome confirmation, password reset, usage alerts) and pair with Customer.io or Loops for the onboarding drip.

SECTION 3

API Documentation (13 Endpoints)

AgentStack API Reference

> **Version:** v1 | **Last updated:** 2026-03-10 | **Status:** Production

Complete REST API reference for the AgentStack AI agent platform. This document covers every public endpoint, including request/response schemas, authentication, error handling, and usage examples.

Table of Contents

1. Overview
2. Base URL
3. Authentication
4. Rate Limits
5. Error Format
6. Pagination
7. Endpoints
   - Auth
     - POST /auth/register
     - POST /auth/login
     - Agents
     - GET /agents/templates
     - POST /agents/{id}/activate
     - POST /agents/{id}/test-run
     - Workflows
     - POST /workflows
     - POST /workflows/{id}/execute
     - Executions
     - GET /executions
     - Usage

    - GET /usage
    - Billing
    - POST /billing/create-checkout
    - POST /billing/portal
    - Workspaces
    - GET /workspaces
    - API Keys
    - POST /api-keys
8. Webhook Payloads
9. Changelog

Overview

AgentStack is an AI agent platform that provides 82 pre-built agent templates, a visual workflow builder, and a BYOK (Bring Your Own Key) model. The API lets you programmatically manage agents, build and execute workflows, track usage, and handle billing.

All API access is over HTTPS. All request and response bodies are JSON-encoded. All timestamps are returned in ISO 8601 format in UTC.

Base URL

```
https://api.agentstack.ai/v1
```

All endpoint paths in this document are relative to this base URL. For example, `POST /auth/register` means:

```
POST https://api.agentstack.ai/v1/auth/register
```

Authentication

All endpoints (except `/auth/register` and `/auth/login`) require a valid bearer token in the `Authorization` header.

```
Authorization: Bearer <access_token>
```

Tokens are obtained via the login or register endpoints. Access tokens expire after 1 hour. Use the `refresh_token` to obtain a new access token without re-authenticating.

API keys created via POST /api-keys can also be used as bearer tokens. API keys support scoped access (e.g., `agents:read`, `workflows:write`) and optional expiration.

Token Types

TOKEN	LIFETIME	USE CASE
`access_token`	1 hour	Short-lived session token for all API calls
`refresh_token`	30 days	Long-lived token to obtain new access tokens
API Key	Custom (or never)	Programmatic access with scoped permissions

Rate Limits

Rate limits are applied per-account and vary by plan:

PLAN	REQUESTS PER MINUTE	BURST
Free	60	10 req/sec
Pro	300	50 req/sec
Team	1,000	100 req/sec

Rate limit status is returned in response headers on every request:

HEADER	DESCRIPTION
`X-RateLimit-Limit`	Maximum requests allowed per minute
`X-RateLimit-Remaining`	Requests remaining in the current window
`X-RateLimit-Reset`	Unix timestamp when the rate limit window resets
`Retry-After`	Seconds to wait before retrying (only on `429` responses)

When you exceed the rate limit, the API returns a `429 Too Many Requests` response:

```
{
  "error": "rate_limit_exceeded",
  "message": "Too many requests. Please retry after 12 seconds.",
  "retry_after": 12
}
```

Error Format

All error responses follow a consistent JSON structure:

```
{
  "error": "error_code",
  "message": "Human-readable description of what went wrong.",
  "request_id": "req_01HXY123ABC"
}
```

FIELD	TYPE	DESCRIPTION
`error`	string	Machine-readable error code (e.g., `validation_error`, `not_found`)
`message`	string	Human-readable explanation
`request_id`	string	Unique request identifier. Include this in support tickets.
`details`	array	(Optional) Field-level validation errors. Only present on `422` responses.

Validation Error Example (422)

```
{
  "error": "validation_error",
  "message": "Request body validation failed.",
  "request_id": "req_01HXY456DEF",
  "details": [
    {
      "field": "email",
      "message": "Must be a valid email address."
    },
    {
      "field": "password",
      "message": "Must be at least 8 characters."
    }
  ]
}
```

Common Error Codes

STATUS	ERROR CODE	DESCRIPTION
`400`	`bad_request`	Malformed request body or missing required field
`401`	`unauthorized`	Missing or invalid authentication token
`402`	`payment_required`	Action requires a paid plan or provider quota exceeded
`403`	`forbidden`	Authenticated but insufficient permissions or plan limits reached
`404`	`not_found`	Requested resource does not exist
`408`	`request_timeout`	The operation timed out (e.g., agent execution exceeded limit)
`409`	`conflict`	Resource already exists (e.g., duplicate email, agent already activated)
`422`	`validation_error`	Request body failed schema validation
`429`	`rate_limit_exceeded`	Too many requests. See `Retry-After` header.
`500`	`internal_error`	Unexpected server error. Retry or contact support.

Pagination

All list endpoints use page-based pagination with a consistent interface:

Query Parameters

PARAMETER	TYPE	DEFAULT	DESCRIPTION
`page`	integer	`1`	Page number (1-indexed)
`limit`	integer	`20`	Items per page. Max: `100`.

Response Envelope

```
{
  "data": [ ... ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 82,
    "has_more": true
  }
}
```

FIELD	TYPE	DESCRIPTION
`data`	array	Array of resource objects
`pagination.page`	integer	Current page number
`pagination.limit`	integer	Items per page
`pagination.total`	integer	Total number of items across all pages
`pagination.has_more`	boolean	`true` if more pages exist after the current page

Endpoints

Auth

POST /auth/register

Create a new user account and workspace.

This endpoint creates a new user, provisions a default workspace, and returns authentication tokens. No prior authentication is required.

Authentication

None required. This is a public endpoint.

Headers

HEADER	REQUIRED	DESCRIPTION
`Content-Type`	Yes	Must be `application/json`

Request Body

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`email`	string	Yes	User's email address. Must be a valid email format. Max 255 characters.
`password`	string	Yes	Account password. Minimum 8 characters. Must contain at least one uppercase letter, one lowercase letter, and one number.
`full_name`	string	Yes	User's display name. 1--100 characters.
`workspace_name`	string	No	Name for the default workspace. If omitted, defaults to `"{full_name}'s Workspace"`. 1--100 characters.

Example Request

```
curl -X POST https://api.agentstack.ai/v1/auth/register \
  -H "Content-Type: application/json" \
  -d '{
    "email": "sarthak@example.com",
    "password": "SecureP@ss1",
    "full_name": "Sarthak Khedkar",
    "workspace_name": "My AI Studio"
  }'
```

Response

201 Created

```
{
  "user": {
    "id": "usr_01HXYZ1234ABCDEF",
    "email": "sarthak@example.com",
    "full_name": "Sarthak Khedkar",
    "role": "owner",
    "workspace_id": "ws_01HXYZ5678GHIJKL",
    "created_at": "2026-03-10T10:00:00Z"
  },
  "workspace": {
    "id": "ws_01HXYZ5678GHIJKL",
    "name": "My AI Studio",
    "slug": "my-ai-studio",
    "plan": "free"
  },
  "access_token": "eyJhbGciOiJIUzI1NiIs...",
  "refresh_token": "eyJhbGciOiJIUzI1NiIs...",
  "token_type": "bearer",
  "expires_in": 3600
}
```

Response Fields

FIELD	TYPE	DESCRIPTION
`user.id`	string	Unique user identifier. Format: `usr_` prefix + ULID.
`user.email`	string	User's email address
`user.full_name`	string	User's display name

FIELD	TYPE	DESCRIPTION
`user.role`	string	User's role in the workspace. Always `"owner"` for newly created workspaces.
`user.workspace_id`	string	ID of the default workspace
`user.created_at`	string (ISO 8601)	UTC timestamp of account creation
`workspace.id`	string	Unique workspace identifier. Format: `ws_` prefix + ULID.
`workspace.name`	string	Workspace display name
`workspace.slug`	string	URL-safe workspace slug
`workspace.plan`	string	Current plan. Always `"free"` for new workspaces.
`access_token`	string	JWT access token. Expires in 1 hour.
`refresh_token`	string	JWT refresh token. Expires in 30 days.
`token_type`	string	Always `"bearer"`
`expires_in`	integer	Access token lifetime in seconds (`3600`)

Errors

STATUS	ERROR CODE	DESCRIPTION
`409`	`email_already_exists`	An account with this email address already exists.
`422`	`validation_error`	Request body validation failed. See `details` array.
`429`	`rate_limit_exceeded`	Too many registration attempts. See `Retry-After` header.

409 Conflict Example

```
{
  "error": "email_already_exists",
  "message": "An account with email sarthak@example.com already exists.",
  "request_id": "req_01HXYZ789"
}
```

422 Validation Error Example

```
{
  "error": "validation_error",
  "message": "Request body validation failed.",
  "request_id": "req_01HXYZ790",
  "details": [
    {
      "field": "password",
      "message": "Must be at least 8 characters and contain one uppercase letter, one lowercase letter, and
    }
  ]
}
```

429 Rate Limit Example

```
{
  "error": "rate_limit_exceeded",
  "message": "Too many registration attempts. Please retry after 60 seconds.",
  "retry_after": 60
}
```

POST /auth/login

Authenticate an existing user and return session tokens.

Authentication

None required. This is a public endpoint.

Headers

HEADER	REQUIRED	DESCRIPTION
`Content-Type`	Yes	Must be `application/json`

Request Body

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`email`	string	Yes	Registered email address
`password`	string	Yes	Account password

Example Request

```
curl -X POST https://api.agentstack.ai/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "email": "sarthak@example.com",
    "password": "SecureP@ss1"
  }'
```

Response

200 OK

```
{
  "user": {
    "id": "usr_01HXYZ1234ABCDEF",
    "email": "sarthak@example.com",
    "full_name": "Sarthak Khedkar",
    "role": "owner",
    "workspace_id": "ws_01HXYZ5678GHIJKL",
    "created_at": "2026-03-10T10:00:00Z"
  },
  "access_token": "eyJhbGciOiJIUzI1NiIs...",
  "refresh_token": "eyJhbGciOiJIUzI1NiIs...",
  "token_type": "bearer",
  "expires_in": 3600
}
```

Response Fields

FIELD	TYPE	DESCRIPTION
`user.id`	string	Unique user identifier
`user.email`	string	User's email address

FIELD	TYPE	DESCRIPTION
`user.full_name`	string	User's display name
`user.role`	string	User's role in the active workspace. One of: `owner`, `admin`, `member`, `viewer`.
`user.workspace_id`	string	Active workspace ID
`user.created_at`	string (ISO 8601)	UTC timestamp of account creation
`access_token`	string	JWT access token. Expires in 1 hour.
`refresh_token`	string	JWT refresh token. Expires in 30 days.
`token_type`	string	Always `"bearer"`
`expires_in`	integer	Access token lifetime in seconds (`3600`)

Errors

STATUS	ERROR CODE	DESCRIPTION
`401`	`invalid_credentials`	Email or password is incorrect.
`403`	`account_disabled`	Account has been suspended or deactivated.
`429`	`rate_limit_exceeded`	Too many login attempts. See `Retry-After` header.

401 Invalid Credentials Example

```
{
  "error": "invalid_credentials",
  "message": "The email or password you entered is incorrect.",
  "request_id": "req_01HXYZ800"
}
```

403 Account Disabled Example

```
{
  "error": "account_disabled",
  "message": "This account has been deactivated. Contact support@agentstack.ai for assistance.",
  "request_id": "req_01HXYZ801"
}
```

Agents

GET /agents/templates

List all available agent templates. Returns the full catalog of 82 pre-built agents with filtering, search, and pagination.

Authentication

Requires a valid bearer token.

```
Authorization: Bearer <access_token>
```

Query Parameters

PARAMETER	TYPE	REQUIRED	DEFAULT	DESCRIPTION
`category`	string	No	--	Filter by category. One of: `engineering`, `design`, `product`, `infrastructure`, `qa`, `growth`, `research`, `coordination`, `specialized`.
`search`	string	No	--	Full-text search across agent name and description. Case-insensitive.
`page`	integer	No	`1`	Page number (1-indexed)

PARAMETER	TYPE	REQUIRED	DEFAULT	DESCRIPTION
`limit`	integer	No	`20`	Items per page. Max: `100`.

Example Request

```
curl -X GET "https://api.agentstack.ai/v1/agents/templates?category=engineering&limit=5" \
  -H "Authorization: Bearer $ACCESS_TOKEN"
```

Response

200 OK


```
{
  "data": [
    {
      "id": "agt_01HXYZ_backend_eng",
      "slug": "backend-systems-engineer",
      "name": "Backend Systems Engineer",
      "description": "Designs and implements backend services, APIs, and data models. Produces production-re
      "category": "engineering",
      "capabilities": [
        "code_generation",
        "api_design",
        "database_schema",
        "code_review"
      ],
      "is_activated": false,
      "activation_tier": "free",
      "avg_tokens_per_run": 4200,
      "avg_cost_per_run_usd": 0.042
    },
    {
      "id": "agt_01HXYZ_code_review",
      "slug": "code-reviewer",
      "name": "Code Reviewer",
      "description": "Reviews pull requests and code changes for bugs, performance issues, security vulnerat
      "category": "engineering",
      "capabilities": [
        "code_review",
        "security_analysis",
        "performance_audit"
      ],
      "is_activated": true,
      "activation_tier": "free",
      "avg_tokens_per_run": 3800,
      "avg_cost_per_run_usd": 0.038
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 5,
    "total": 18,
    "has_more": true
  }
}
```

Response Fields (per agent)

FIELD	TYPE	DESCRIPTION
`id`	string	Unique agent template identifier. Format: `agt_` prefix + ULID.
`slug`	string	URL-safe unique slug for the agent
`name`	string	Agent display name
`description`	string	What the agent does (1--2 sentences)
`category`	string	Agent category. One of: `engineering`, `design`, `product`, `infrastructure`, `qa`, `growth`, `research`, `coordination`, `specialized`.
`capabilities`	string[]	List of capability tags (e.g., `code_generation`, `api_design`)
`is_activated`	boolean	Whether this agent is activated in the current workspace
`activation_tier`	string	Minimum plan required. One of: `free`, `pro`, `team`.
`avg_tokens_per_run`	integer	Average token consumption per execution
`avg_cost_per_run_usd`	number	Estimated cost per run in USD (based on GPT-4o pricing)

Errors

STATUS	ERROR CODE	DESCRIPTION
`401`	`unauthorized`	Missing or invalid bearer token

POST /agents/{id}/activate

Activate an agent template in your workspace. Once activated, the agent can be used in workflows and test runs.

Authentication

Requires a valid bearer token with `agents:write` scope.

```
Authorization: Bearer <access_token>
```

Path Parameters

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`id`	string	Yes	Agent template ID (e.g., `agt_01HXYZ_backend_eng`)

Request Body

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`custom_name`	string	No	Override the default agent name. 1--100 characters.
`custom_prompt`	string	No	Append custom instructions to the agent's system prompt. Max 5,000 characters.
`model_preference`	string	No	Preferred LLM model. One of: `gpt-4o`, `gpt-4`, `claude-3.5-sonnet`, `claude-3-opus`, `gemini-pro`. Defaults to `gpt-4o`.

Example Request

```
curl -X POST https://api.agentstack.ai/v1/agents/agt_01HXYZ_backend_eng/activate \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "custom_name": "My Backend Engineer",
    "model_preference": "claude-3.5-sonnet"
  }'
```

Response

200 OK

```
{
  "id": "agt_01HXYZ_backend_eng",
  "slug": "backend-systems-engineer",
  "name": "My Backend Engineer",
  "description": "Designs and implements backend services, APIs, and data models.",
  "category": "engineering",
  "model_preference": "claude-3.5-sonnet",
  "custom_prompt": null,
  "is_activated": true,
  "activated_at": "2026-03-10T10:15:00Z",
  "workspace_id": "ws_01HXYZ5678GHIJKL"
}
```

Response Fields

FIELD	TYPE	DESCRIPTION
`id`	string	Agent template ID
`slug`	string	URL-safe agent slug
`name`	string	Agent name (custom name if provided, otherwise default)
`description`	string	Agent description
`category`	string	Agent category
`model_preference`	string	Selected LLM model
`custom_prompt`	string or null	Custom prompt instructions, if set
`is_activated`	boolean	Always `true` after successful activation
`activated_at`	string (ISO 8601)	UTC timestamp of activation
`workspace_id`	string	Workspace where the agent was activated

Errors

STATUS	ERROR CODE	DESCRIPTION
`401`	`unauthorized`	Missing or invalid bearer token
`403`	`plan_limit_reached`	Free plan is limited to 5 activated agents. Upgrade to Pro for unlimited agents.
`404`	`agent_not_found`	No agent template found with this ID.
`409`	`agent_already_activated`	This agent is already activated in your workspace.

403 Plan Limit Example

```
{
  "error": "plan_limit_reached",
  "message": "Free plan is limited to 5 activated agents. Upgrade to Pro for unlimited agents.",
  "request_id": "req_01HXYZ810",
  "current_count": 5,
  "plan_limit": 5,
  "upgrade_url": "https://app.agentstack.ai/billing"
}
```

409 Already Activated Example

```
{
  "error": "agent_already_activated",
  "message": "Backend Systems Engineer is already activated in this workspace.",
  "request_id": "req_01HXYZ811"
}
```

POST /agents/{id}/test-run

Execute a single test run against an activated agent. Returns the full execution result including output, token usage, cost, and latency.

Authentication

Requires a valid bearer token with `agents:write` scope.

```
Authorization: Bearer <access_token>
```

Path Parameters

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`id`	string	Yes	Activated agent ID

Request Body

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`input`	string	Yes	The prompt or task input for the agent. Max 100,000 characters.
`context`	object	No	Additional context for the agent run. See Context Object.
`model_override`	string	No	Override the agent's default model for this run. One of: `gpt-4o`, `gpt-4`, `claude-3.5-sonnet`, `claude-3-opus`, `gemini-pro`.

Context Object

FIELD	TYPE	DESCRIPTION
`files`	string[]	Array of file URLs or base64-encoded file contents to include as context
`variables`	object	Key-value pairs of variables available to the agent during execution

Example Request

```
curl -X POST https://api.agentstack.ai/v1/agents/agt_01HXYZ_backend_eng/test-run \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "input": "Design a REST API for a task management app with users, projects, and tasks. Include authentic
    "context": {
      "variables": {
        "framework": "FastAPI",
        "database": "PostgreSQL"
      }
    },
    "model_override": "gpt-4o"
  }'
```

Response

200 OK

```
{
  "execution": {
    "id": "exec_01HXYZ9000MNOPQR",
    "agent_id": "agt_01HXYZ_backend_eng",
    "status": "completed",
    "input": "Design a REST API for a task management app with users, projects, and tasks. Include authentic
    "output": "# Task Management API Design\n\n## Authentication\n\n### POST /auth/register\n...",
    "token_usage": {
      "prompt_tokens": 1250,
      "completion_tokens": 2980,
      "total_tokens": 4230
    },
    "cost_usd": 0.042,
    "model_used": "gpt-4o",
    "latency_ms": 8420,
    "created_at": "2026-03-10T10:20:00Z",
    "completed_at": "2026-03-10T10:20:08Z"
  }
}
```

Response Fields

FIELD	TYPE	DESCRIPTION
`execution.id`	string	Unique execution identifier. Format: `exec_` prefix + ULID.

FIELD	TYPE	DESCRIPTION
`execution.agent_id`	string	Agent that ran the execution
`execution.status`	string	Execution status. One of: `completed`, `failed`.
`execution.input`	string	The original input prompt
`execution.output`	string	Agent's generated output
`execution.token_usage.prompt_tokens`	integer	Tokens used in the prompt
`execution.token_usage.completion_tokens`	integer	Tokens generated in the response
`execution.token_usage.total_tokens`	integer	Total tokens consumed
`execution.cost_usd`	number	Cost of this execution in USD (based on your API key's provider pricing)
`execution.model_used`	string	The LLM model that processed this run
`execution.latency_ms`	integer	End-to-end execution time in milliseconds
`execution.created_at`	string (ISO 8601)	UTC timestamp when execution started
`execution.completed_at`	string (ISO 8601)	UTC timestamp when execution finished

Errors

STATUS	ERROR CODE	DESCRIPTION
`400`	`missing_api_key`	No API key configured for the required LLM provider (e.g., OpenAI, Anthropic). Configure your key in Settings > API Keys.
`401`	`unauthorized`	Missing or invalid bearer token
`402`	`provider_quota_exceeded`	Your LLM provider API key has exceeded its usage quota. Check your provider dashboard.

STATUS	ERROR CODE	DESCRIPTION
`404`	`agent_not_activated`	This agent is not activated in your workspace. Call `POST /agents/{id}/activate` first.
`408`	`execution_timeout`	Agent execution exceeded the maximum time limit (120 seconds). Try a simpler input or increase the timeout in agent config.
`429`	`execution_limit_reached`	You have reached the maximum concurrent executions for your plan.

400 Missing API Key Example

```
{
  "error": "missing_api_key",
  "message": "No OpenAI API key configured. Add your key at Settings > API Keys to use GPT-4o.",
  "request_id": "req_01HXYZ820",
  "provider": "openai",
  "settings_url": "https://app.agentstack.ai/settings/api-keys"
}
```

402 Provider Quota Exceeded Example

```
{
  "error": "provider_quota_exceeded",
  "message": "Your OpenAI API key has exceeded its usage quota. Check your billing at https://platform.opena
  "request_id": "req_01HXYZ821",
  "provider": "openai"
}
```

408 Timeout Example

```
{
  "error": "execution_timeout",
  "message": "Agent execution timed out after 120 seconds.",
  "request_id": "req_01HXYZ822",
  "timeout_ms": 120000
}
```

Workflows

POST /workflows

Create a new workflow that chains multiple agent steps together. Workflows define a sequence of agent executions where each step can reference variables and outputs from previous steps.

Authentication

Requires a valid bearer token with `workflows:write` scope.

```
Authorization: Bearer <access_token>
```

Headers

HEADER	REQUIRED	DESCRIPTION
`Authorization`	Yes	Bearer token
`Content-Type`	Yes	Must be `application/json`

Request Body

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`name`	string	Yes	Workflow name. 1--100 characters.
`description`	string	No	Workflow description. Max 1,000 characters.
`steps`	array	Yes	Ordered array of workflow steps. Minimum 1 step, maximum 20 steps. See Step Object.
`trigger`	object	No	How the workflow is triggered. See Trigger Object. Defaults to `{ "type": "manual" }`.

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`variables`	object	No	JSON Schema defining the input variables this workflow accepts. See Variables Schema.

Step Object

FIELD	TYPE	REQUIRED	DESCRIPTION
`agent_id`	string	Yes	ID of an activated agent to use for this step
`name`	string	Yes	Step display name. 1--100 characters.
`prompt_template`	string	Yes	Prompt template with variable interpolation. Supports `{{variables.var_name}}` for workflow input variables and `{{prev_step.output}}` to reference the output of the immediately preceding step. Also supports `{{steps.step_name.output}}` for any named step. Max 50,000 characters.
`position`	integer	Yes	Step order (0-indexed). Must be sequential with no gaps.
`config`	object	No	Per-step configuration overrides. See Step Config.

Step Config

FIELD	TYPE	DEFAULT	DESCRIPTION
`model_override`	string	--	Override the agent's model for this step. One of: `gpt-4o`, `gpt-4`, `claude-3.5-sonnet`, `claude-3-opus`, `gemini-pro`.
`timeout`	integer	`120`	Maximum execution time in seconds. Range: 10--600.
`retry`	object	`{ "max_attempts": 1 }`	Retry policy. `max_attempts` (1--5) and optional `backoff_ms` (100--30000).

Trigger Object

FIELD	TYPE	REQUIRED	DESCRIPTION
`type`	string	Yes	Trigger type. One of: `manual`, `cron`, `webhook`.
`cron_expression`	string	Conditional	Required when `type` is `cron`. Standard 5-field cron expression (e.g., `"0 9 * * 1-5"` for weekdays at 9 AM UTC).
`webhook_secret`	string	No	Auto-generated if `type` is `webhook`. Used to verify incoming webhook payloads.

Variables Schema

Defines the input contract for the workflow using a subset of JSON Schema:

```
{
  "type": "object",
  "properties": {
    "repo_url": {
      "type": "string",
      "description": "GitHub repository URL"
    },
    "branch": {
      "type": "string",
      "default": "main",
      "description": "Branch to analyze"
    }
  },
  "required": ["repo_url"]
}
```

Example Request

```
curl -X POST https://api.agentstack.ai/v1/workflows \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Ship a Feature",
    "description": "End-to-end feature workflow: plan, code, review, document, test.",
    "steps": [
      {
        "agent_id": "agt_01HXYZ_sprint_plan",
        "name": "plan",
        "prompt_template": "Break down this feature into tasks: {{variables.feature_description}}",
        "position": 0
      },
      {
        "agent_id": "agt_01HXYZ_backend_eng",
        "name": "implement",
        "prompt_template": "Implement the following tasks in {{variables.language}}:\n\n{{steps.plan.output}
        "position": 1,
        "config": {
          "model_override": "claude-3.5-sonnet",
          "timeout": 300
        }
      },
      {
        "agent_id": "agt_01HXYZ_code_review",
        "name": "review",
        "prompt_template": "Review this code for bugs, security, and performance:\n\n{{prev_step.output}}",
        "position": 2
      },
      {
        "agent_id": "agt_01HXYZ_docs_writer",
        "name": "document",
        "prompt_template": "Write API documentation for:\n\n{{steps.implement.output}}",
        "position": 3
      },
      {
        "agent_id": "agt_01HXYZ_qa_spec",
        "name": "test",
        "prompt_template": "Write integration tests for:\n\n{{steps.implement.output}}\n\nAddress review fir
        "position": 4,
        "config": { "retry": { "max_attempts": 2, "backoff_ms": 1000 } }
      }
    ],
    "trigger": { "type": "manual" },
    "variables": {
      "type": "object",
      "properties": {
        "feature_description": { "type": "string", "description": "Feature to build" },
        "language": { "type": "string", "default": "TypeScript" }
      },
      "required": ["feature_description"]
    }
  }'
```

Response

201 Created

```
{
  "id": "wf_01HXYZAAA111222",
  "name": "Ship a Feature",
  "description": "End-to-end feature workflow: plan, code, review, document, test.",
  "steps": [
    {
      "id": "step_01HXYZ_S0",
      "agent_id": "agt_01HXYZ_sprint_plan",
      "name": "plan",
      "prompt_template": "Break down this feature into tasks: {{variables.feature_description}}",
      "position": 0,
      "config": { "model_override": null, "timeout": 120, "retry": { "max_attempts": 1 } }
    },
    {
      "id": "step_01HXYZ_S1",
      "agent_id": "agt_01HXYZ_backend_eng",
      "name": "implement",
      "prompt_template": "Implement the following tasks in {{variables.language}}:\n\n{{steps.plan.output}}"
      "position": 1,
      "config": { "model_override": "claude-3.5-sonnet", "timeout": 300, "retry": { "max_attempts": 1 } }
    },
    {
      "id": "step_01HXYZ_S2",
      "agent_id": "agt_01HXYZ_code_review",
      "name": "review",
      "prompt_template": "Review this code for bugs, security, and performance:\n\n{{prev_step.output}}",
      "position": 2,
      "config": { "model_override": null, "timeout": 120, "retry": { "max_attempts": 1 } }
    },
    {
      "id": "step_01HXYZ_S3",
      "agent_id": "agt_01HXYZ_docs_writer",
      "name": "document",
      "prompt_template": "Write API documentation for:\n\n{{steps.implement.output}}",
      "position": 3,
      "config": { "model_override": null, "timeout": 120, "retry": { "max_attempts": 1 } }
    },
    {
      "id": "step_01HXYZ_S4",
      "agent_id": "agt_01HXYZ_qa_spec",
      "name": "test",
      "prompt_template": "Write integration tests for:\n\n{{steps.implement.output}}\n\nAddress review findi
      "position": 4,
      "config": { "model_override": null, "timeout": 120, "retry": { "max_attempts": 2, "backoff_ms": 1000 }
    }
  ],
  "trigger": { "type": "manual", "webhook_url": null, "cron_expression": null },
  "variables": {
    "type": "object",
    "properties": {
      "feature_description": { "type": "string", "description": "Feature to build" },
      "language": { "type": "string", "default": "TypeScript" }
    },
    "required": ["feature_description"]
  },
  "workspace_id": "ws_01HXYZ5678GHIJKL",
  "created_by": "usr_01HXYZ1234ABCDEF",
```

```
  "created_at": "2026-03-10T10:30:00Z",
  "updated_at": "2026-03-10T10:30:00Z"
}
```

Response Fields

FIELD	TYPE	DESCRIPTION
`id`	string	Unique workflow identifier. Format: `wf_` prefix + ULID.
`name`	string	Workflow display name
`description`	string or null	Workflow description
`steps`	array	Array of step objects with generated `id` fields and resolved defaults
`steps[].id`	string	Unique step identifier. Format: `step_` prefix + ULID.
`trigger`	object	Trigger configuration
`trigger.type`	string	Trigger type: `manual`, `cron`, or `webhook`
`trigger.webhook_url`	string or null	Webhook URL (only for `webhook` type triggers)
`trigger.cron_expression`	string or null	Cron schedule (only for `cron` type triggers)
`variables`	object or null	JSON Schema for workflow input variables
`workspace_id`	string	Owning workspace ID
`created_by`	string	User ID who created the workflow
`created_at`	string (ISO 8601)	UTC timestamp of creation
`updated_at`	string (ISO 8601)	UTC timestamp of last update

Errors

STATUS	ERROR CODE	DESCRIPTION
`400`	`invalid_step_configuration`	A step references an invalid or non-activated agent, or step positions are not sequential.
`401`	`unauthorized`	Missing or invalid bearer token
`403`	`plan_limit_reached`	Free plan is limited to 3 workflows. Upgrade to Pro for unlimited workflows.
`422`	`validation_error`	Request body validation failed. See `details` array.

400 Invalid Step Example

```
{
  "error": "invalid_step_configuration",
  "message": "Step at position 2 references agent 'agt_01HXYZ_invalid' which is not activated in this worksp
  "request_id": "req_01HXYZ830",
  "step_position": 2,
  "agent_id": "agt_01HXYZ_invalid"
}
```

403 Plan Limit Example

```
{
  "error": "plan_limit_reached",
  "message": "Free plan is limited to 3 workflows. Upgrade to Pro for unlimited workflows.",
  "request_id": "req_01HXYZ831",
  "current_count": 3,
  "plan_limit": 3,
  "upgrade_url": "https://app.agentstack.ai/billing"
}
```

POST /workflows/{id}/execute

Execute a workflow. Runs all steps in sequence, passing outputs between steps according to the prompt templates. Supports synchronous (wait for result) and asynchronous (return immediately) execution modes.

Authentication

Requires a valid bearer token with `workflows:write` scope.

```
Authorization: Bearer <access_token>
```

Path Parameters

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`id`	string	Yes	Workflow ID (e.g., `wf_01HXYZAAA111222`)

Request Body

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`variables`	object	Conditional	Key-value pairs matching the workflow's `variables` schema. Required if the workflow has required variables defined.
`async`	boolean	No	If `true`, returns immediately with an execution ID and `running` status. If `false` (default), blocks until all steps complete.

Example Request (Synchronous)

```
curl -X POST https://api.agentstack.ai/v1/workflows/wf_01HXYZAAA111222/execute \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "variables": {
      "feature_description": "Add a user notification preferences page with email, SMS, and push toggles.",
      "language": "TypeScript"
    },
    "async": false
  }'
```

Example Request (Asynchronous)

```
curl -X POST https://api.agentstack.ai/v1/workflows/wf_01HXYZAAA111222/execute \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "variables": { "feature_description": "Add dark mode support to the dashboard." },
    "async": true
  }'
```

Response

200 OK (Synchronous -- `async: false`)

```
{
  "execution": {
    "id": "exec_01HXYZ_WF_RUN_001",
    "workflow_id": "wf_01HXYZAAA111222",
    "status": "completed",
    "variables": {
      "feature_description": "Add a user notification preferences page with email, SMS, and push toggles.",
      "language": "TypeScript"
    },
    "steps": [
      {
        "step_id": "step_01HXYZ_S0",
        "name": "plan",
        "agent_id": "agt_01HXYZ_sprint_plan",
        "status": "completed",
        "output": "## Implementation Tasks\n\n1. Create NotificationPreferences model...",
        "token_usage": { "prompt_tokens": 850, "completion_tokens": 1200, "total_tokens": 2050 },
        "cost_usd": 0.021,
        "model_used": "gpt-4o",
        "latency_ms": 4200,
        "started_at": "2026-03-10T10:35:00Z",
        "completed_at": "2026-03-10T10:35:04Z"
      },
      {
        "step_id": "step_01HXYZ_S1",
        "name": "implement",
        "agent_id": "agt_01HXYZ_backend_eng",
        "status": "completed",
        "output": "```typescript\n// notification-preferences.controller.ts\nimport { Controller, Get, Put,
        "token_usage": { "prompt_tokens": 2100, "completion_tokens": 5400, "total_tokens": 7500 },
        "cost_usd": 0.075,
        "model_used": "claude-3.5-sonnet",
        "latency_ms": 12300,
        "started_at": "2026-03-10T10:35:04Z",
        "completed_at": "2026-03-10T10:35:17Z"
      },
      {
        "step_id": "step_01HXYZ_S2",
        "name": "review",
        "agent_id": "agt_01HXYZ_code_review",
        "status": "completed",
        "output": "## Code Review\n\n### Issues Found\n1. **HIGH** Missing input validation...",
        "token_usage": { "prompt_tokens": 5800, "completion_tokens": 1800, "total_tokens": 7600 },
        "cost_usd": 0.076,
        "model_used": "gpt-4o",
        "latency_ms": 6100,
        "started_at": "2026-03-10T10:35:17Z",
        "completed_at": "2026-03-10T10:35:23Z"
      },
      {
        "step_id": "step_01HXYZ_S3",
        "name": "document",
        "agent_id": "agt_01HXYZ_docs_writer",
        "status": "completed",
        "output": "# Notification Preferences API\n\n## Endpoints\n\n### GET /api/v1/notifications/preferenc
        "token_usage": { "prompt_tokens": 5600, "completion_tokens": 2200, "total_tokens": 7800 },
        "cost_usd": 0.078,
```

```
          "model_used": "gpt-4o",
          "latency_ms": 5800,
          "started_at": "2026-03-10T10:35:23Z",
          "completed_at": "2026-03-10T10:35:29Z"
        },
        {
          "step_id": "step_01HXYZ_S4",
          "name": "test",
          "agent_id": "agt_01HXYZ_qa_spec",
          "status": "completed",
          "output": "```typescript\n// notification-preferences.integration.spec.ts\ndescribe('NotificationPre
          "token_usage": { "prompt_tokens": 7800, "completion_tokens": 3200, "total_tokens": 11000 },
          "cost_usd": 0.110,
          "model_used": "gpt-4o",
          "latency_ms": 8900,
          "started_at": "2026-03-10T10:35:29Z",
          "completed_at": "2026-03-10T10:35:38Z"
        }
      ],
      "total_tokens": 35950,
      "total_cost_usd": 0.360,
      "created_at": "2026-03-10T10:35:00Z",
      "completed_at": "2026-03-10T10:35:38Z"
    }
  }
```


202 Accepted (Asynchronous -- `async: true`)


```
{
  "execution": {
    "id": "exec_01HXYZ_WF_RUN_002",
    "workflow_id": "wf_01HXYZAAA111222",
    "status": "running",
    "variables": { "feature_description": "Add dark mode support to the dashboard." },
    "created_at": "2026-03-10T10:40:00Z"
  }
}
```


Poll `GET /executions/{id}` to check the status of asynchronous executions.

Response Fields

FIELD	TYPE	DESCRIPTION
`execution.id`	string	Unique execution identifier. Format: `exec_` prefix + ULID.
`execution.workflow_id`	string	Workflow that was executed
`execution.status`	string	One of: `running`, `completed`, `failed`, `cancelled`

FIELD	TYPE	DESCRIPTION
`execution.variables`	object	Input variables used for this run
`execution.steps`	array	Per-step execution results (only in synchronous responses). Each step includes `step_id`, `name`, `agent_id`, `status`, `output`, `token_usage`, `cost_usd`, `model_used`, `latency_ms`, and timestamps.
`execution.total_tokens`	integer	Sum of all step token usage (only when `status` is `completed`)
`execution.total_cost_usd`	number	Sum of all step costs in USD (only when `status` is `completed`)
`execution.created_at`	string (ISO 8601)	UTC timestamp when execution started
`execution.completed_at`	string (ISO 8601) or null	UTC timestamp when execution finished. `null` if still running.

Errors

STATUS	ERROR CODE	DESCRIPTION
`400`	`missing_required_variables`	The workflow requires variables that were not provided.
`401`	`unauthorized`	Missing or invalid bearer token
`404`	`workflow_not_found`	No workflow found with this ID in the current workspace.
`429`	`execution_limit_reached`	Maximum concurrent executions reached for your plan.

400 Missing Variables Example

```
{
  "error": "missing_required_variables",
  "message": "The following required variables are missing: feature_description",
  "request_id": "req_01HXYZ840",
  "missing_variables": ["feature_description"]
}
```

Executions

GET /executions

List execution history for workflows and agents in the current workspace. Supports filtering by workflow, agent, status, and pagination with sorting.

Authentication

Requires a valid bearer token (access token or API key with `executions:read` scope).

```
Authorization: Bearer <access_token>
```

Headers

HEADER	REQUIRED	DESCRIPTION
`Authorization`	Yes	Bearer token or API key

Query Parameters

PARAMETER	TYPE	REQUIRED	DEFAULT	DESCRIPTION
`workflow_id`	string	No	--	Filter by workflow ID. Format: `wf_` prefix + ULID.
`agent_id`	string	No	--	Filter by agent ID. Format: `agt_` prefix + ULID.
`status`	string	No	--	Filter by execution status. One of: `running`, `completed`, `failed`, `cancelled`.
`page`	integer	No	`1`	Page number (1-indexed).
`limit`	integer	No	`20`	Items per page. Min: `1`. Max: `100`.
`sort`	string	No	`created_at:desc`	Sort order. Format: `field:direction`. Valid fields: `created_at`, `completed_at`, `total_tokens`, `total_cost_usd`. Directions: `asc`, `desc`.

Example Request

```
curl -X GET "https://api.agentstack.ai/v1/executions?workflow_id=wf_01HXYZAAA111222&status=completed&limit=1
  -H "Authorization: Bearer $ACCESS_TOKEN"
```

Response

200 OK

```
{
  "data": [
    {
      "id": "exec_01HXYZ_WF_RUN_001",
      "type": "workflow",
      "workflow_id": "wf_01HXYZAAA111222",
      "workflow_name": "Ship a Feature",
      "status": "completed",
      "steps_completed": 5,
      "steps_total": 5,
      "variables": {
        "feature_description": "Add dark mode support to the dashboard."
      },
      "total_tokens": 35950,
      "total_cost_usd": 0.360,
      "created_at": "2026-03-10T10:35:00Z",
      "completed_at": "2026-03-10T10:35:38Z"
    },
    {
      "id": "exec_01HXYZ_AGENT_003",
      "type": "agent",
      "agent_id": "agt_01HXYZ_BACKEND",
      "agent_name": "Backend Systems Engineer",
      "status": "completed",
      "steps_completed": 1,
      "steps_total": 1,
      "variables": {},
      "total_tokens": 12400,
      "total_cost_usd": 0.124,
      "created_at": "2026-03-10T09:20:00Z",
      "completed_at": "2026-03-10T09:20:14Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 47,
    "has_more": true
  }
}
```

Response Fields

FIELD	TYPE	DESCRIPTION
`data`	array	Array of execution objects

FIELD	TYPE	DESCRIPTION
`data[].id`	string	Unique execution identifier. Format: `exec_` prefix + ULID.
`data[].type`	string	Execution type. One of: `workflow`, `agent`.
`data[].workflow_id`	string or null	Workflow ID (present when `type` is `workflow`)
`data[].workflow_name`	string or null	Workflow display name
`data[].agent_id`	string or null	Agent ID (present when `type` is `agent`)
`data[].agent_name`	string or null	Agent display name
`data[].status`	string	One of: `running`, `completed`, `failed`, `cancelled`
`data[].steps_completed`	integer	Number of steps that have finished
`data[].steps_total`	integer	Total number of steps in the execution
`data[].variables`	object	Input variables used for this run
`data[].total_tokens`	integer	Total tokens consumed across all steps
`data[].total_cost_usd`	number	Total cost in USD across all steps
`data[].created_at`	string (ISO 8601)	UTC timestamp when execution started
`data[].completed_at`	string (ISO 8601) or null	UTC timestamp when execution finished. `null` if still running.
`pagination`	object	Pagination metadata. See Pagination.

Errors

STATUS	ERROR CODE	DESCRIPTION
`400`	`invalid_sort_field`	The `sort` parameter references an invalid field or direction.
`401`	`unauthorized`	Missing or invalid bearer token
`422`	`validation_error`	Invalid query parameter value (e.g., `limit` exceeds 100, invalid `status`).

400 Invalid Sort Example

```
{
  "error": "invalid_sort_field",
  "message": "Invalid sort field 'name'. Valid fields: created_at, completed_at, total_tokens, total_cost_us
  "request_id": "req_01HXYZ850"
}
```

Usage

GET /usage

Retrieve a usage summary for the current workspace. Returns aggregate execution counts, token consumption, cost breakdowns by agent and model, and daily usage timeseries data.

Authentication

Requires a valid bearer token (access token or API key with `executions:read` scope).

```
Authorization: Bearer <access_token>
```

Headers

HEADER	REQUIRED	DESCRIPTION
`Authorization`	Yes	Bearer token or API key

Query Parameters

PARAMETER	TYPE	REQUIRED	DEFAULT	DESCRIPTION
`period`	string	No	`current`	Billing period to report on. One of: `current` (current billing cycle), `previous` (last billing cycle).
`granularity`	string	No	`daily`	Granularity for the `daily_usage` timeseries. One of: `daily`, `hourly`. `hourly` is only available for the `current` period.

Example Request

```
curl -X GET "https://api.agentstack.ai/v1/usage?period=current&granularity=daily" \
  -H "Authorization: Bearer $ACCESS_TOKEN"
```

Response

200 OK

```
{
  "period": {
    "name": "current",
    "start": "2026-03-01T00:00:00Z",
    "end": "2026-03-31T23:59:59Z"
  },
  "total_executions": 1247,
  "total_tokens": 18740350,
  "total_cost_usd": 214.83,
  "by_agent": [
    {
      "agent_id": "agt_01HXYZ_BACKEND",
      "agent_name": "Backend Systems Engineer",
      "executions": 312,
      "tokens": 5120000,
      "cost_usd": 61.44
    },
    {
      "agent_id": "agt_01HXYZ_CODE_REVIEW",
      "agent_name": "Code Reviewer",
      "executions": 298,
      "tokens": 3890000,
      "cost_usd": 46.68
    }
  ],
  "by_model": [
    {
      "model": "gpt-4o",
      "provider": "openai",
      "executions": 845,
      "tokens": 12400000,
      "cost_usd": 148.80
    },
    {
      "model": "claude-3.5-sonnet",
      "provider": "anthropic",
      "executions": 402,
      "tokens": 6340350,
      "cost_usd": 66.03
    }
  ],
  "daily_usage": [
    {
      "date": "2026-03-01",
      "executions": 42,
      "tokens": 620500,
      "cost_usd": 7.12
    },
    {
      "date": "2026-03-02",
      "executions": 38,
      "tokens": 580200,
      "cost_usd": 6.47
    }
  ]
}
```

Response Fields

FIELD	TYPE	DESCRIPTION
`period`	object	The billing period being reported on
`period.name`	string	Period identifier: `current` or `previous`
`period.start`	string (ISO 8601)	UTC start timestamp of the billing period
`period.end`	string (ISO 8601)	UTC end timestamp of the billing period
`total_executions`	integer	Total number of executions in this period
`total_tokens`	integer	Total tokens consumed across all executions
`total_cost_usd`	number	Total cost in USD (based on BYOK provider pricing)
`by_agent`	array	Usage breakdown by agent. Sorted by `cost_usd` descending.
`by_agent[].agent_id`	string	Agent identifier
`by_agent[].agent_name`	string	Agent display name
`by_agent[].executions`	integer	Number of executions for this agent
`by_agent[].tokens`	integer	Total tokens consumed by this agent
`by_agent[].cost_usd`	number	Total cost in USD for this agent
`by_model`	array	Usage breakdown by LLM model. Sorted by `cost_usd` descending.
`by_model[].model`	string	Model identifier (e.g., `gpt-4o`, `claude-3.5-sonnet`)
`by_model[].provider`	string	Provider name (e.g., `openai`, `anthropic`, `google`)
`by_model[].executions`	integer	Number of executions using this model
`by_model[].tokens`	integer	Total tokens consumed by this model
`by_model[].cost_usd`	number	Total cost in USD for this model
`daily_usage`	array	Timeseries data at the requested granularity. When `granularity=hourly`, each entry includes a `datetime` field instead of `date`.

FIELD	TYPE	DESCRIPTION
`daily_usage[].date`	string (`YYYY-MM-DD`)	Date for this data point (when `granularity=daily`)
`daily_usage[].executions`	integer	Executions on this date
`daily_usage[].tokens`	integer	Tokens consumed on this date
`daily_usage[].cost_usd`	number	Cost in USD on this date

Errors

STATUS	ERROR CODE	DESCRIPTION
`400`	`invalid_period`	The `period` parameter is not a valid value.
`400`	`invalid_granularity`	The `granularity` parameter is not a valid value, or `hourly` was requested for the `previous` period.
`401`	`unauthorized`	Missing or invalid bearer token

400 Invalid Granularity Example

```
{
  "error": "invalid_granularity",
  "message": "Hourly granularity is only available for the current billing period.",
  "request_id": "req_01HXYZ860"
}
```

Billing

POST /billing/create-checkout

Create a Stripe Checkout session to upgrade the workspace to a paid plan. Redirects the user to Stripe's hosted checkout page to complete payment.

Authentication

Requires a valid bearer token. Only workspace owners and admins can create checkout sessions.

```
Authorization: Bearer <access_token>
```

Headers

HEADER	REQUIRED	DESCRIPTION
`Authorization`	Yes	Bearer token
`Content-Type`	Yes	Must be `application/json`

Request Body

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`plan`	string	Yes	Target plan. One of: `pro`, `team`.
`billing_cycle`	string	Yes	Billing frequency. One of: `monthly`, `annual`. Annual billing includes a 20% discount.

Example Request

```
curl -X POST https://api.agentstack.ai/v1/billing/create-checkout \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "plan": "pro",
    "billing_cycle": "annual"
  }'
```

Response

200 OK

```
{
  "checkout_url": "https://checkout.stripe.com/c/pay/cs_live_a1b2c3d4e5f6...",
  "session_id": "cs_live_a1b2c3d4e5f6g7h8i9j0",
  "plan": "pro",
  "billing_cycle": "annual",
  "price_usd": 470.40,
  "expires_at": "2026-03-10T11:30:00Z"
}
```

Response Fields

FIELD	TYPE	DESCRIPTION
`checkout_url`	string	Stripe-hosted checkout URL. Redirect the user to this URL to complete payment.
`session_id`	string	Stripe Checkout session ID. Use this to track the session status.
`plan`	string	The plan being purchased: `pro` or `team`
`billing_cycle`	string	The billing frequency: `monthly` or `annual`
`price_usd`	number	Total price in USD for the selected plan and cycle. Annual prices reflect the 20% discount (e.g., Pro annual = $49 x 12 x 0.8 = $470.40).
`expires_at`	string (ISO 8601)	UTC timestamp when the checkout session expires. Stripe sessions expire after 30 minutes.

Errors

STATUS	ERROR CODE	DESCRIPTION
`400`	`invalid_plan`	The `plan` value is not valid. Must be `pro` or `team`.
`400`	`invalid_billing_cycle`	The `billing_cycle` value is not valid. Must be `monthly` or `annual`.
`401`	`unauthorized`	Missing or invalid bearer token

STATUS	ERROR CODE	DESCRIPTION
`403`	`insufficient_permissions`	Only workspace owners and admins can manage billing.
`409`	`already_subscribed`	The workspace is already on the requested plan.
`422`	`validation_error`	Request body validation failed. See `details` array.

409 Already Subscribed Example

```
{
  "error": "already_subscribed",
  "message": "This workspace is already on the Pro plan. Use the billing portal to manage your subscription.
  "request_id": "req_01HXYZ870"
}
```

POST /billing/portal

Create a Stripe Billing Portal session. The portal lets users manage their subscription, update payment methods, view invoices, and cancel their plan.

Authentication

Requires a valid bearer token. Only workspace owners and admins can access the billing portal.

```
Authorization: Bearer <access_token>
```

Headers

HEADER	REQUIRED	DESCRIPTION
`Authorization`	Yes	Bearer token
`Content-Type`	Yes	Must be `application/json`

Request Body

No request body required. Send an empty JSON object or omit the body.

Example Request

```
curl -X POST https://api.agentstack.ai/v1/billing/portal \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

Response

200 OK

```
{
  "portal_url": "https://billing.stripe.com/p/session/bps_1abc2def3ghi...",
  "expires_at": "2026-03-10T11:30:00Z"
}
```

Response Fields

FIELD	TYPE	DESCRIPTION
`portal_url`	string	Stripe-hosted billing portal URL. Redirect the user to this URL.
`expires_at`	string (ISO 8601)	UTC timestamp when the portal session expires. Portal sessions expire after 30 minutes.

Errors

STATUS	ERROR CODE	DESCRIPTION
`401`	`unauthorized`	Missing or invalid bearer token
`403`	`insufficient_permissions`	Only workspace owners and admins can access the billing portal.
`404`	`no_subscription`	The workspace does not have an active subscription. Use POST /billing/create-checkout to subscribe first.

404 No Subscription Example

```
{
  "error": "no_subscription",
  "message": "This workspace does not have an active subscription. Create a checkout session to subscribe.",
  "request_id": "req_01HXYZ880"
}
```

Workspaces

GET /workspaces

Retrieve details for the current workspace, including plan information, resource counts, and usage limits.

Authentication

Requires a valid bearer token (access token or API key with any scope).

```
Authorization: Bearer <access_token>
```

Headers

HEADER	REQUIRED	DESCRIPTION
`Authorization`	Yes	Bearer token or API key

Query Parameters

None.

Example Request

```
curl -X GET https://api.agentstack.ai/v1/workspaces \
  -H "Authorization: Bearer $ACCESS_TOKEN"
```

Response

200 OK

```
{
  "workspace": {
    "id": "ws_01HXYZ5678GHIJKL",
    "name": "My AI Studio",
    "slug": "my-ai-studio",
    "plan": "pro",
    "member_count": 3,
    "agent_count": 24,
    "workflow_count": 8,
    "execution_count": 1247,
    "limits": {
      "max_agents": null,
      "max_workflows": null,
      "max_executions_per_month": null,
      "max_team_members": 1,
      "api_access": true
    },
    "created_at": "2026-02-15T10:00:00Z"
  }
}
```

Response Fields

FIELD	TYPE	DESCRIPTION
`workspace.id`	string	Unique workspace identifier. Format: `ws_` prefix + ULID.
`workspace.name`	string	Workspace display name
`workspace.slug`	string	URL-safe workspace slug. Used in dashboard URLs.
`workspace.plan`	string	Current plan. One of: `free`, `pro`, `team`.

FIELD	TYPE	DESCRIPTION
`workspace.member_count`	integer	Number of members in the workspace
`workspace.agent_count`	integer	Number of activated agents
`workspace.workflow_count`	integer	Number of created workflows
`workspace.execution_count`	integer	Total executions in the current billing period
`workspace.limits`	object	Plan-specific resource limits
`workspace.limits.max_agents`	integer or null	Maximum agents allowed. `null` means unlimited (Pro and Team plans).
`workspace.limits.max_workflows`	integer or null	Maximum workflows allowed. `null` means unlimited.
`workspace.limits.max_executions_per_month`	integer or null	Maximum executions per month. `null` means unlimited.
`workspace.limits.max_team_members`	integer or null	Maximum team members. `1` on Free, `1` on Pro, `null` (unlimited) on Team.
`workspace.limits.api_access`	boolean	Whether API key access is available on this plan
`workspace.created_at`	string (ISO 8601)	UTC timestamp of workspace creation

Plan Limits Reference

LIMIT	FREE	PRO	TEAM
`max_agents`	5	null (unlimited)	null (unlimited)
`max_workflows`	3	null (unlimited)	null (unlimited)
`max_executions_per_month`	100	null (unlimited)	null (unlimited)
`max_team_members`	1	1	null (unlimited)
`api_access`	false	true	true

Errors

STATUS	ERROR CODE	DESCRIPTION
`401`	`unauthorized`	Missing or invalid bearer token

API Keys

POST /api-keys

Create a new API key for programmatic access to the AgentStack API. API keys support scoped permissions and optional expiration. The full key value is returned **only once** in the creation response -- store it securely.

Authentication

Requires a valid bearer token (access token). API keys cannot be used to create other API keys.

```
Authorization: Bearer <access_token>
```

Headers

HEADER	REQUIRED	DESCRIPTION
`Authorization`	Yes	Bearer token (access token only -- not an API key)
`Content-Type`	Yes	Must be `application/json`

Request Body

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`name`	string	Yes	Human-readable name for the API key. 1--100 characters. Must be unique within the workspace.
`scopes`	array of strings	Yes	Permission scopes for this key. At least one scope is required. See Available Scopes.
`expires_in_days`	integer	No	Number of days until the key expires. Min: `1`. Max: `365`. If omitted, the key never expires.

Available Scopes

SCOPE	PERMISSION
`agents:read`	List agent templates, view agent details
`agents:write`	Activate agents, run test executions, update agent configuration
`workflows:read`	List workflows, view workflow details
`workflows:write`	Create, update, delete, and execute workflows
`executions:read`	List execution history, view execution details, access usage data

> **Note:** `agents:write` implies `agents:read`. Similarly, `workflows:write` implies `workflows:read`.

Example Request

```
curl -X POST https://api.agentstack.ai/v1/api-keys \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "CI/CD Pipeline Key",
    "scopes": ["workflows:write", "executions:read"],
    "expires_in_days": 90
  }'
```

Response

201 Created

```
{
  "key": "ask_live_a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0",
  "key_id": "key_01HXYZ_CICD_001",
  "name": "CI/CD Pipeline Key",
  "scopes": ["workflows:write", "workflows:read", "executions:read"],
  "expires_at": "2026-06-08T10:45:00Z",
  "created_at": "2026-03-10T10:45:00Z"
}
```

> **Important:** The `key` field contains the full API key and is only returned in this response. It cannot be retrieved again. Store it in a secure location (e.g., environment variable, secrets manager).

Response Fields

FIELD	TYPE	DESCRIPTION
`key`	string	The full API key value. **Shown once.** Format: `ask_live_` prefix + 40 random alphanumeric characters. Use this as a bearer token in the `Authorization` header.
`key_id`	string	Unique key identifier. Format: `key_` prefix + ULID. Use this to reference the key in management operations (revoke, list).
`name`	string	Human-readable key name
`scopes`	array of strings	Effective scopes for this key. Includes implied scopes (e.g., `workflows:read` is included when `workflows:write` is granted).
`expires_at`	string (ISO 8601) or null	UTC timestamp when the key expires. `null` if the key never expires.
`created_at`	string (ISO 8601)	UTC timestamp of key creation

Errors

STATUS	ERROR CODE	DESCRIPTION
`400`	`invalid_scopes`	One or more scope values are not recognized.
`401`	`unauthorized`	Missing or invalid bearer token
`403`	`api_access_not_available`	API key creation is not available on the Free plan. Upgrade to Pro or Team.
`409`	`key_name_exists`	An API key with this name already exists in the workspace.
`422`	`validation_error`	Request body validation failed. See `details` array.

403 Free Plan Example

```
{
  "error": "api_access_not_available",
  "message": "API key creation requires a Pro or Team plan. Upgrade at https://app.agentstack.ai/settings/bi
  "request_id": "req_01HXYZ890"
}
```

409 Duplicate Name Example

```
{
  "error": "key_name_exists",
  "message": "An API key named 'CI/CD Pipeline Key' already exists. Choose a different name.",
  "request_id": "req_01HXYZ891"
}
```

Webhook Payloads

When a workflow is configured with a `webhook` trigger, AgentStack sends an HTTP POST request to your specified URL each time the trigger fires. The payload contains the execution context and results.

Delivery

- **Method:** `POST`

- **Content-Type:** `application/json`
- **Timeout:** 30 seconds. If your endpoint does not respond within 30 seconds, the delivery is marked as failed.
- **Retries:** Failed deliveries are retried up to 3 times with exponential backoff (1 minute, 5 minutes, 30 minutes).
- **Signature:** Every webhook request includes an `X-AgentStack-Signature` header containing an HMAC-SHA256 signature of the request body. Use your webhook secret (provided when configuring the trigger) to verify authenticity.

Headers

HEADER	DESCRIPTION
`Content-Type`	Always `application/json`
`X-AgentStack-Signature`	HMAC-SHA256 hex digest of the raw request body, signed with your webhook secret
`X-AgentStack-Event`	Event type. Currently always `workflow.execution.completed`
`X-AgentStack-Delivery-Id`	Unique delivery ID (UUID). Use for idempotency.
`User-Agent`	`AgentStack-Webhooks/1.0`

Payload Schema


```
{
  "event": "workflow.execution.completed",
  "delivery_id": "d4e5f6a7-b8c9-0d1e-2f3a-4b5c6d7e8f9a",
  "timestamp": "2026-03-10T10:35:38Z",
  "workspace_id": "ws_01HXYZ5678GHIJKL",
  "workflow": {
    "id": "wf_01HXYZAAA111222",
    "name": "Ship a Feature"
  },
  "execution": {
    "id": "exec_01HXYZ_WF_RUN_001",
    "status": "completed",
    "variables": {
      "feature_description": "Add dark mode support to the dashboard."
    },
    "steps": [
      {
        "step_id": "step_01",
        "name": "Sprint Planning",
        "agent_id": "agt_01HXYZ_SPRINT",
        "status": "completed",
        "output": "## Sprint Plan\n\n### User Stories\n1. As a user, I want to toggle dark mode...",
        "token_usage": 8200,
        "cost_usd": 0.082,
        "model_used": "gpt-4o",
        "latency_ms": 4500
      },
      {
        "step_id": "step_02",
        "name": "Backend Implementation",
        "agent_id": "agt_01HXYZ_BACKEND",
        "status": "completed",
        "output": "```typescript\n// Theme toggle endpoint...\n```",
        "token_usage": 12400,
        "cost_usd": 0.124,
        "model_used": "gpt-4o",
        "latency_ms": 6200
      }
    ],
    "total_tokens": 35950,
    "total_cost_usd": 0.360,
    "created_at": "2026-03-10T10:35:00Z",
    "completed_at": "2026-03-10T10:35:38Z"
  }
}
```

Payload Fields

FIELD	TYPE	DESCRIPTION
`event`	string	Event type. Currently: `workflow.execution.completed`. Future events may include `workflow.execution.failed` and `workflow.execution.step_completed`.
`delivery_id`	string (UUID)	Unique identifier for this delivery attempt. Use for idempotent processing.
`timestamp`	string (ISO 8601)	UTC timestamp when the event occurred
`workspace_id`	string	Workspace that owns the workflow
`workflow.id`	string	Workflow identifier
`workflow.name`	string	Workflow display name
`execution.id`	string	Execution identifier
`execution.status`	string	Execution status: `completed` or `failed`
`execution.variables`	object	Input variables that were passed to the execution
`execution.steps`	array	Per-step results. Same schema as synchronous execution responses.
`execution.total_tokens`	integer	Total tokens consumed
`execution.total_cost_usd`	number	Total cost in USD
`execution.created_at`	string (ISO 8601)	When the execution started
`execution.completed_at`	string (ISO 8601)	When the execution finished

Verifying Webhook Signatures

Verify the `X-AgentStack-Signature` header to ensure the payload was sent by AgentStack and has not been tampered with.

Node.js example:

```
const crypto = require('crypto');

function verifyWebhook(payload, signature, secret) {
  const expected = crypto
    .createHmac('sha256', secret)
    .update(payload, 'utf8')
    .digest('hex');
  return crypto.timingSafeEqual(
    Buffer.from(signature),
    Buffer.from(expected)
  );
}

// In your webhook handler:
app.post('/webhook', (req, res) => {
  const signature = req.headers['x-agentstack-signature'];
  const isValid = verifyWebhook(
    JSON.stringify(req.body),
    signature,
    process.env.AGENTSTACK_WEBHOOK_SECRET
  );
  if (!isValid) {
    return res.status(401).json({ error: 'Invalid signature' });
  }
  // Process the webhook payload...
  res.status(200).json({ received: true });
});
```

Python example:

```
import hmac
import hashlib

def verify_webhook(payload: bytes, signature: str, secret: str) -> bool:
    expected = hmac.new(
        secret.encode('utf-8'),
        payload,
        hashlib.sha256
    ).hexdigest()
    return hmac.compare_digest(signature, expected)
```

Responding to Webhooks

Return a `2xx` status code within 30 seconds to acknowledge receipt. Any non-`2xx` response or timeout triggers a retry.

YOUR RESPONSE	AGENTSTACK BEHAVIOR
`200` -- `299`	Delivery marked as successful. No retry.

YOUR RESPONSE	AGENTSTACK BEHAVIOR
`3xx`	Redirects are **not** followed. Treated as a failure.
`4xx`	Delivery marked as failed. Retried up to 3 times.
`5xx`	Delivery marked as failed. Retried up to 3 times.
Timeout (>30s)	Delivery marked as failed. Retried up to 3 times.

Pagination Conventions

All list endpoints (`GET /executions` , `GET /agents/templates` , etc.) follow the same pagination contract.

Request

Pass pagination parameters as query strings:

PARAMETER	TYPE	DEFAULT	CONSTRAINTS	DESCRIPTION
`page`	integer	`1`	Min: `1`	Page number (1-indexed)
`limit`	integer	`20`	Min: `1` , Max: `100`	Number of items per page

Response Envelope

Every paginated response wraps items in a `data` array with an accompanying `pagination` object:

```
{
  "data": [ ... ],
  "pagination": {
    "page": 2,
    "limit": 20,
    "total": 82,
    "has_more": true
  }
}
```

FIELD	TYPE	DESCRIPTION
`data`	array	Array of resource objects for the current page
`pagination.page`	integer	Current page number (mirrors your request)
`pagination.limit`	integer	Items per page (mirrors your request)
`pagination.total`	integer	Total number of items across all pages
`pagination.has_more`	boolean	`true` if there are additional pages after the current one

Calculating Total Pages

```
total_pages = ceil(pagination.total / pagination.limit)
```

Edge Cases

SCENARIO	BEHAVIOR
`page` exceeds total pages	Returns empty `data` array with `has_more: false`
`limit` exceeds 100	Returns `422 validation_error`
`limit` is 0 or negative	Returns `422 validation_error`
No results match filters	Returns empty `data` array with `total: 0` and `has_more: false`

Example: Fetching All Pages

```
# Page 1
curl "https://api.agentstack.ai/v1/executions?page=1&limit=50" \
  -H "Authorization: Bearer $ACCESS_TOKEN"

# Page 2 (if has_more was true)
curl "https://api.agentstack.ai/v1/executions?page=2&limit=50" \
  -H "Authorization: Bearer $ACCESS_TOKEN"
```

```
import httpx

def fetch_all_executions(token: str) -> list:
    all_items = []
    page = 1
    while True:
        resp = httpx.get(
            "https://api.agentstack.ai/v1/executions",
            params={"page": page, "limit": 100},
            headers={"Authorization": f"Bearer {token}"},
        )
        data = resp.json()
        all_items.extend(data["data"])
        if not data["pagination"]["has_more"]:
            break
        page += 1
    return all_items
```

Rate Limiting

Rate limits protect the API from abuse and ensure fair usage across all customers. Limits are applied **per workspace** based on the current plan.

Limits by Plan

PLAN	REQUESTS PER MINUTE	BURST (REQ/SEC)	CONCURRENT EXECUTIONS
Free	60	10	2
Pro	300	50	10
Team	1,000	100	50

Rate Limit Headers

Every API response includes rate limit headers:

HEADER	TYPE	DESCRIPTION
`X-RateLimit-Limit`	integer	Maximum requests allowed per minute for your plan
`X-RateLimit-Remaining`	integer	Requests remaining in the current 1-minute window

HEADER	TYPE	DESCRIPTION
`X-RateLimit-Reset`	integer	Unix timestamp (seconds) when the current window resets
`Retry-After`	integer	Seconds to wait before retrying. **Only present on `429` responses.**

Example Headers

```
HTTP/1.1 200 OK
X-RateLimit-Limit: 300
X-RateLimit-Remaining: 287
X-RateLimit-Reset: 1741603260
```

429 Too Many Requests

When you exceed the rate limit, the API returns:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 12
X-RateLimit-Limit: 300
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1741603260
Content-Type: application/json
```

```
{
  "error": "rate_limit_exceeded",
  "message": "Too many requests. Please retry after 12 seconds.",
  "retry_after": 12,
  "request_id": "req_01HXYZ900"
}
```

Best Practices

1. **Read the headers.** Check `X-RateLimit-Remaining` before making high-volume requests.
2. **Implement exponential backoff.** On `429` responses, wait the `Retry-After` duration, then retry with increasing delays.
3. **Use async execution.** For workflow executions, use `async: true` to avoid holding a connection open and consuming rate limit capacity while waiting for results.
4. **Batch where possible.** Reduce total request count by using larger `limit` values on list endpoints instead of many small requests.

5. **Cache read responses.** Agent templates and workspace details change infrequently -- cache them client-side.

Execution Concurrency Limits

In addition to request rate limits, each plan has a maximum number of **concurrent executions** (workflows or agent test-runs running simultaneously). If you exceed this limit, `POST /workflows/{id}/execute` and `POST /agents/{id}/test-run` return a `429` with error code `execution_limit_reached`.

Changelog

All notable changes to the AgentStack API are documented here. The API follows Semantic Versioning. Breaking changes are only introduced in major version increments.

v1.3.0 -- 2026-03-10

Added
- `POST /api-keys` -- Create scoped API keys with optional expiration.
- `GET /usage` -- Usage summary endpoint with `by_agent`, `by_model`, and `daily_usage` breakdowns.
- `hourly` granularity option for `GET /usage` on the current billing period.
- `X-AgentStack-Delivery-Id` header on webhook payloads for idempotent processing.
- `sort` query parameter on `GET /executions` supporting `created_at`, `completed_at`, `total_tokens`, and `total_cost_usd`.

Changed
- `POST /workflows/{id}/execute` now returns `202 Accepted` (instead of `200 OK`) for asynchronous executions.
- Webhook retry policy increased from 2 retries to 3 retries with longer backoff intervals.

v1.2.0 -- 2026-02-15

Added
- `POST /billing/create-checkout` -- Stripe Checkout session creation for plan upgrades.
- `POST /billing/portal` -- Stripe Billing Portal session for subscription management.
- `GET /workspaces` -- Workspace details endpoint with plan limits and resource counts.
- `annual` billing cycle option with 20% discount.

Changed

- `GET /executions` now includes `steps_completed` and `steps_total` fields for progress tracking.
- Error responses now consistently include `request_id` across all endpoints.

v1.1.0 -- 2026-01-20

Added

- `POST /workflows` -- Workflow creation with step chaining and variable interpolation.
- `POST /workflows/{id}/execute` -- Synchronous and asynchronous workflow execution.
- `GET /executions` -- Execution history with filtering and pagination.
- Webhook trigger type for workflows with HMAC-SHA256 signature verification.
- `cron` trigger type for scheduled workflow execution.

Changed

- `POST /agents/{id}/test-run` now returns `model_used` and `latency_ms` in the response.

v1.0.0 -- 2026-01-01

Added

- Initial public API release.
- `POST /auth/register` -- User registration with automatic workspace provisioning.
- `POST /auth/login` -- Authentication with JWT access and refresh tokens.
- `GET /agents/templates` -- List all 82 agent templates with filtering.
- `POST /agents/{id}/activate` -- Activate an agent template in the workspace.
- `POST /agents/{id}/test-run` -- Execute a single agent with a test prompt.
- Rate limiting with `X-RateLimit-*` response headers.
- Consistent error response format with `error`, `message`, and `request_id` fields.

This API reference is maintained by the AgentStack team. For questions, feedback, or to report issues, contact api-support@agentstack.ai or open an issue on our GitHub repository.

SECTION 4

Dashboard Microcopy (63 Items)

AgentStack Dashboard Microcopy -- Complete

> **Status:** Ship-ready | **Version:** 1.0 | **Date:** 2026-03-10
> **Surface:** In-app dashboard (web)
> **Tone:** Clear, direct, human. Calm on errors, inviting on empty states, brief on success.
> **Voice:** Knowledgeable colleague -- not a chatbot, not a legal document.

SECTION 1 -- EMPTY STATES

1.1 Agent List (no agents activated)

Headline: 82 agents, ready when you are.
Body: Browse the catalog and activate your first agent. Each one handles a specific job -- code review, sprint planning, copywriting, and 79 more.
CTA: Browse agent catalog

1.2 Workflow List (no workflows created)

Headline: Chain agents into workflows.
Body: Workflows let you connect multiple agents into a sequence -- pass output from one agent as input to the next. That's where the real leverage is.
CTA: Create your first workflow

1.3 Execution History (no executions yet)

Headline: No executions yet.
Body: Every time an agent or workflow runs, it shows up here with full logs, token usage, and cost. Run your first agent to see it in action.
CTA: Go to agents

1.4 API Keys (no keys created)

Headline: Access AgentStack programmatically.
Body: API keys let you trigger agents and workflows from your own code, CI/CD pipelines, or webhooks. Create a key to get started.
CTA: Create API key

1.5 Team Members (solo workspace)

Headline: You're the only one here.
Body: Invite teammates to share agents, workflows, and execution history. Everyone uses their own API keys -- no shared credentials.
CTA: Invite a teammate
Note (below CTA, muted text): Available on Team plan.

1.6 Usage Dashboard (no usage data)

Headline: Nothing to show yet.
Body: Usage data appears here once your first agent runs -- token counts, cost breakdowns, and execution trends over time.
CTA: Run your first agent

SECTION 2 -- ERROR MESSAGES

2.1 Rate Limit Hit (API throttled)

Title: Too many requests.
Body: You've hit the rate limit. Wait a few seconds and try again. If this keeps happening, space out your API calls or contact support.

2.2 Plan Limit -- Agents (Free tier, 5 max)

Title: You've reached the 5-agent limit.
Body: Free plans include 5 active agents. Upgrade to Pro to activate all 82 agents with no limits.
CTA: See Pro plan

2.3 Plan Limit -- Executions (Free tier, 100/mo max)

Title: Monthly execution limit reached.
Body: You've used all 100 executions this month. Your limit resets on {reset_date}. Upgrade to Pro for unlimited executions.
CTA: Upgrade to Pro

2.4 Plan Limit -- Workflows (Free tier, 3 max)

Title: Workflow limit reached.
Body: Free plans include up to 3 workflows. Upgrade to Pro to create unlimited workflows.
CTA: Upgrade to Pro

2.5 Invalid API Key (provider key rejected)

Title: API key rejected by {provider_name}.
Body: The API key you entered wasn't accepted. Double-check that you copied the full key and that it's still active in your {provider_name} dashboard.
CTA: Re-enter API key

2.6 Workflow Execution Failed (generic)

Title: Workflow execution failed.
Body: Something went wrong during this run. Check the execution log for details -- it'll show which step failed and why.
CTA: View execution log

2.7 Agent Timeout (exceeded 120s)

Title: Agent timed out.
Body: This agent didn't respond within 120 seconds. This usually means the task was too complex for a single pass. Try breaking it into smaller steps or simplifying the input.
CTA: Retry | View logs

2.8 Provider Quota Exceeded (user's OpenAI/Anthropic billing)

Title: Your {provider_name} quota is exceeded.
Body: This isn't an AgentStack issue -- your {provider_name} account has hit its usage or spending limit. Check your billing settings at {provider_name} to increase your quota.
CTA: Open {provider_name} billing

2.9 Network Error (connection lost)

Title: Connection lost.
Body: Check your internet connection and try again. If you're on a VPN, try disconnecting temporarily.
CTA: Retry

2.10 Permission Denied (wrong role)

Title: You don't have access to this.
Body: This action requires a different role. Ask your workspace admin to update your permissions.

SECTION 3 -- ONBOARDING TOOLTIPS

3.1 Agent Catalog Browse Button

Browse all 82 pre-built agents by category -- engineering, design, product, marketing, and more.

3.2 Agent Activate Button

Activating an agent adds it to your workspace. You can configure its model, prompt, and behavior after activation.

3.3 Agent Model Preference Selector

Choose which LLM this agent uses. This calls your own API key -- you're billed directly by the provider, not by AgentStack.

3.4 Agent Custom Prompt Field

Override the agent's default system prompt. Use this to tailor its behavior to your codebase, brand voice, or specific requirements.

3.5 Workflow Canvas (first time)

This is your workflow builder. Drag agents onto the canvas, connect them in sequence, and data flows from one to the next automatically.

3.6 Workflow Add Step Button

Add an agent, conditional branch, or wait step. Each step receives the output of the previous one as input.

3.7 Workflow Trigger Type Selector

Choose what starts this workflow: manual trigger, API call, webhook, or a scheduled cron. You can change this anytime.

3.8 Workflow Variables Panel

Define variables that get passed into your workflow at runtime -- like repo URLs, file paths, or custom instructions. Reference them in any step with {variable_name}.

3.9 Execution Log Viewer

Full trace of every step in the run. Click any step to see its input, output, token usage, latency, and any errors.

3.10 Token Usage Display

Total tokens consumed across all steps in this execution. Broken down by prompt tokens (input) and completion tokens (output).

3.11 Cost Tracker

Estimated cost based on your provider's published pricing. This is what your provider charges you -- AgentStack adds nothing on top.

3.12 API Keys Page

Use these keys to call AgentStack from your code. Keys have full access to your workspace -- treat them like passwords.

3.13 BYOK Settings (connect provider key)

Paste your API key from OpenAI, Anthropic, or any supported provider. AgentStack never stores your key on our servers -- it's encrypted and used only at execution time.

3.14 Team Invite Button

Invite teammates by email. They'll get their own login and can access shared agents and workflows. Each member connects their own provider keys.

3.15 Usage Dashboard Charts

Track your execution count, token usage, and estimated cost over time. Filter by agent, workflow, or date range.

3.16 Webhook URL Display

Send a POST request to this URL to trigger the workflow externally. Include any payload -- it'll be available as variables inside the workflow.

SECTION 4 -- SUCCESS MESSAGES (Toasts/Notifications)

4.1 Agent Activated

{agent_name} activated.

4.2 Agent Deactivated

{agent_name} deactivated. It won't run in workflows until you reactivate it.

4.3 Workflow Created

Workflow created. Add agents and a trigger to start using it.

4.4 Workflow Updated

Workflow saved.

4.5 Workflow Executed Successfully

Workflow completed. {step_count} steps ran in {duration}. [View results]

4.6 Workflow Deleted

Workflow deleted. This can't be undone.

4.7 API Key Created

API key created. Copy it now -- you won't be able to see it again.

4.8 API Key Revoked

API key revoked. Any integrations using this key will stop working immediately.

4.9 Plan Upgraded to Pro

You're on Pro. All 82 agents and unlimited executions are now available.

4.10 Plan Upgraded to Team

You're on Team. Invite teammates and start collaborating.

4.11 Team Member Invited

Invite sent to {email}. They'll get an email with a link to join your workspace.

4.12 Provider API Key Connected

{provider_name} key connected. Agents using {provider_name} models are ready to run.

4.13 Provider API Key Removed

{provider_name} key removed. Agents using {provider_name} models won't run until you add a new key.

4.14 Profile Updated

Profile updated.

SECTION 5 -- UPGRADE PROMPTS

5.1 Hitting 5 Agent Limit

Headline: You've activated all 5 free agents.
Body: You're getting value from AgentStack -- and there are 77 more agents waiting. Pro unlocks the full catalog: all 82 agents, unlimited executions, and unlimited workflows for $49/mo.
CTA: Unlock all 82 agents
Dismiss: Not now

5.2 Hitting 100 Execution Limit

Headline: You've used all 100 executions this month.
Body: Your agents are paused until {reset_date}. Upgrade to Pro for unlimited executions -- no caps, no throttling, no waiting.
CTA: Upgrade to Pro
Dismiss: Remind me on {reset_date}

5.3 Hitting 3 Workflow Limit

Headline: 3 workflows is just the start.
Body: You're using all 3 workflow slots. Pro gives you unlimited workflows so you can build a complete automation layer -- not just a sample.
CTA: Upgrade for unlimited workflows
Dismiss: Not now

5.4 Trying to Use API Access on Free

Headline: API access is a Pro feature.
Body: Trigger agents and workflows from your own code, CI/CD pipelines, or external services. Upgrade to Pro to unlock programmatic access.
CTA: Upgrade to Pro
Dismiss: Not now

5.5 Trying to Invite Team Members on Free

Headline: Collaboration starts on Team plan.
Body: Invite teammates, share workflows, and keep everyone on the same page -- with separate provider keys and role-based access. Available on the Team plan at $149/mo.
CTA: See Team plan
Dismiss: Not now

5.6 Trying to Create Custom Agents on Free

Headline: Custom agents are a Pro feature.
Body: Build your own agents with custom system prompts, model preferences, and specialized behavior. Upgrade to Pro to create agents tailored to your exact needs.
CTA: Upgrade to Pro
Dismiss: Not now

5.7 Generic Upgrade Banner (Dashboard Sidebar)

Headline: Get more from AgentStack.
Body: Unlock all 82 agents, unlimited executions, API access, and custom agents.
CTA: See plans
Dismiss: (X icon, no text -- returns after 7 days)

IMPLEMENTATION NOTES

Variable Reference

VARIABLE	WHERE USED	SOURCE
`{agent_name}`	Success toasts, error messages	Agent display name from catalog
`{provider_name}`	Error messages, success toasts, BYOK settings	"OpenAI", "Anthropic", "Google", etc.
`{email}`	Team invite toast	Invited user's email
`{reset_date}`	Execution limit error, upgrade prompt	First day of next billing month
`{step_count}`	Workflow success toast	Count of steps in completed run
`{duration}`	Workflow success toast	Total execution wall-clock time, formatted (e.g., "4.2s")
`{variable_name}`	Tooltip (variables panel)	Literal syntax example, not dynamic

Toast Behavior

TYPE	AUTO-DISMISS	DURATION	PERSISTENT
Success	Yes	4 seconds	No
Info	Yes	5 seconds	No
Warning	No	--	Yes, until dismissed
Error	No	--	Yes, until dismissed

Upgrade Prompt Frequency Rules

- **Soft limit prompts** (5.1, 5.3, 5.7): Show once per session. If dismissed, don't show again for 7 days.
- **Hard limit prompts** (5.2, 5.4, 5.5, 5.6): Show every time the user hits the wall. No frequency cap -- the action is genuinely blocked.
- **Sidebar banner** (5.7): Dismissable. Returns after 7 days. Hides permanently after 3 dismissals.

Tone Reference

- **Empty states:** Inviting, forward-looking. Never apologetic ("Sorry, nothing here").
- **Errors:** Calm and factual. Always include what to do next. Never blame the user.
- **Success:** Brief. 3-5 words when possible. No over-celebration.
- **Upgrade prompts:** Honest about what's gated and why. No fake urgency. No countdown timers.
- **Tooltips:** One job: answer "what is this?" in under 2 sentences. Never restate the label.

SECTION 5

SEO Blog Posts (5 Articles)

AgentStack SEO Blog Posts

> **Status:** Ship-ready | **Version:** 1.0 | **Date:** 2026-03-10
> **Target audience:** Indie developers and agencies building AI workflows
> **Tone:** Technical but approachable, confident, founder-voice
> **Internal link format:** [Link: /path] -- replace with actual URLs before publishing

---

BLOG POST 1

SEO Title Tag: How to Build an AI Agent Workflow Without Writing Code | AgentStack

Meta Description: Learn how to build AI agent workflows visually -- no code required. Step-by-step guide with examples for code review, content, and research pipelines.

URL Slug: /blog/build-ai-agent-workflow-without-code

Focus Keyword: AI agent workflow

Secondary Keywords: no-code AI workflow, visual workflow builder, AI agent pipeline, multi-agent workflow, agent orchestration no code

Schema Markup: Article, HowTo

How to Build an AI Agent Workflow Without Writing Code

AI agents are everywhere. But building a workflow where multiple agents collaborate -- one generates code, another reviews it, a third writes the docs -- still feels like a backend engineering project. You end up writing orchestration logic, managing state between steps, handling retries, and debugging failures across a chain of prompts that nobody except you understands.

It does not have to be this way.

An AI agent workflow is a structured sequence of specialized agents that pass context from one step to the next, producing a finished output that no single agent could deliver alone. And in 2025, you can build these workflows visually -- without writing orchestration code, without managing queues, and without deploying infrastructure.

This guide walks you through what AI agent workflows are, why the no-code approach matters, and exactly how to build three production workflows using a visual builder. If you have been manually chaining prompts or writing Python glue code to connect agents, this is the post that saves you the next six weekends.

What Is an AI Agent Workflow?

An AI agent workflow is a pipeline where multiple AI agents execute tasks in a defined sequence, with the output of one agent feeding into the input of the next. Think of it like a production assembly line, but for knowledge work.

A simple example: you want to ship a blog post.

1. **Research Agent** gathers data on a topic from the web
2. **Content Strategist Agent** creates an outline and content brief
3. **Copywriter Agent** writes the full draft based on the brief
4. **SEO Agent** optimizes headings, meta tags, and keyword placement
5. **Editor Agent** reviews for tone, grammar, and consistency

Each agent is a specialist. None of them need to know what the others are doing -- they just need the right input and produce the right output. The workflow is what connects them.

This is fundamentally different from a single mega-prompt that tries to do everything. Specialized agents produce better output because each one has a focused system prompt, relevant tools, and clear success criteria. [Link: /features/agents]

Why No-Code Matters for Agent Workflows

Let us be honest about what "building an agent workflow" looks like when you code it from scratch:

- You pick a framework -- LangChain, CrewAI, AutoGen, or raw API calls
- You write the orchestration logic: what runs first, what runs next, how data flows between steps
- You handle error cases: what happens when an agent times out, returns garbage, or the API rate-limits you
- You build state management: where does the intermediate output live between steps
- You deploy and monitor it: cron jobs, containers, logging, alerting
- You maintain it: every time you change one agent's prompt, you risk breaking the downstream chain

This is real engineering work. For a three-agent workflow, you are looking at 500-1,000 lines of code and a few days of development. For a ten-agent workflow with conditional branching, you are looking at weeks.

The no-code approach collapses all of that into a visual canvas. You drag agents onto a board, draw connections between them, configure each agent's inputs, and hit run. The platform handles orchestration, state management, error handling, retries, and logging.

This matters for two reasons:

Speed. You can go from idea to running workflow in minutes instead of days. When a workflow does not work the way you want, you rearrange the canvas instead of refactoring code.

Accessibility. Product managers, designers, and agency operators can build and modify workflows without filing engineering tickets. The person who understands the process best is the person who builds the workflow.

Step-by-Step: Building Your First AI Agent Workflow

Here is how to build an AI agent workflow from scratch using AgentStack's visual workflow builder. No code, no CLI, no YAML files. [Link: /features/workflow-builder]

Step 1: Define Your Goal

Before you touch the builder, write down what you want the workflow to produce. Be specific.

Bad: "I want AI to help with code."
Good: "I want a workflow that takes a GitHub PR URL, reviews the code for bugs and security issues, and posts a structured review comment back to the PR."

The clearer your goal, the easier it is to pick the right agents and wire them together.

Step 2: Pick Your Agents

Open the AgentStack agent catalog. Browse by category -- Engineering, Design, Product, QA, Growth -- or search by keyword. [Link: /features/agents]

For each step in your workflow, pick the agent that matches the job. You do not need to configure prompts or tools -- each agent comes pre-built with a specialized system prompt, relevant tool integrations, and output formatting.

For the code review example above, you would pick:
- **Code Reviewer** -- reviews code for bugs, performance, security, and style
- **AppSec Audit Engineer** -- focuses specifically on security vulnerabilities

Step 3: Drag, Connect, Configure

Open the visual workflow builder. Drag your selected agents onto the canvas. Draw a connection from the first agent's output to the second agent's input.

For each agent, configure:
- **Input:** What context does this agent need? (A PR URL, a code snippet, a document, free-form text)
- **Model:** Which LLM should this agent use? (GPT-4o for complex reasoning, Claude 3.5 for long context, Gemini for speed)
- **Output format:** Structured JSON, markdown, plain text

You can also add:
- **Conditional branches:** If the Code Reviewer finds zero issues, skip the AppSec audit
- **Parallel steps:** Run the Code Reviewer and AppSec Audit Engineer at the same time, then merge results
- **Human-in-the-loop checkpoints:** Pause the workflow and wait for a human to approve before continuing

Step 4: Test and Iterate

Hit the "Run" button with a sample input. AgentStack executes each step, shows you the intermediate outputs, and logs token usage, latency, and cost for every agent in the chain.

If an agent's output is not what you expected, adjust its configuration and re-run. The feedback loop is seconds, not hours.

Step 5: Deploy or Schedule

Once your workflow produces the output you want, you have three options:
- **Run manually** from the dashboard whenever you need it
- **Schedule** it on a cron (daily, weekly, on every new PR)
- **Trigger via API** from your existing tools using webhooks

That is it. Five steps, zero lines of code, and you have a production AI agent workflow.

Example Workflow 1: Code Review Pipeline

Goal: Automatically review every PR for code quality and security before human review.

Agents used:
1. Code Reviewer -- scans for bugs, performance issues, and style violations
2. AppSec Audit Engineer -- checks for OWASP Top 10 vulnerabilities and dependency risks
3. Senior Cross-Agent Reviewer -- merges both reports into a single, prioritized review

How it works:
- Trigger: new PR opened on GitHub
- Step 1 and Step 2 run in parallel (saves time)
- Step 3 receives both outputs and produces a unified review with severity ratings
- Output is posted as a PR comment on GitHub

Time to build: About 5 minutes in the visual builder.
Equivalent code: 400-600 lines of Python with LangChain, GitHub API integration, and error handling.

[Link: /use-cases/code-review]

Example Workflow 2: Content Creation Pipeline

Goal: Go from a topic keyword to a publish-ready blog post with SEO optimization.

Agents used:
1. SEO Strategist -- researches keywords, analyzes SERP intent, produces a content brief
2. Content & Copy Strategist -- writes the full blog post from the brief
3. SEO Strategist (second pass) -- optimizes headings, meta tags, internal links, and keyword density

How it works:
- Input: a target keyword (e.g., "AI agent workflow")
- Step 1 produces a structured content brief with keyword map, competitor analysis, and outline

- Step 2 writes the complete post
- Step 3 reviews and optimizes for on-page SEO
- Output: publish-ready markdown with SEO metadata

Time to build: About 3 minutes.
Equivalent code: You would need a custom pipeline with web scraping, SERP API calls, and three chained LLM calls with different system prompts.

[Link: /use-cases/content-creation]

Example Workflow 3: Research Pipeline

Goal: Produce a competitive analysis report from a single company name.

Agents used:
1. Competitor Intelligence Analyst -- researches the company's product, pricing, positioning, and recent moves
2. Market Validation Analyst -- assesses market size, demand signals, and competitive density
3. Reddit Market Intelligence Researcher -- mines Reddit for sentiment, pain points, and user complaints
4. Feedback Intelligence Synthesizer -- merges all three reports into a single strategic brief

How it works:
- Input: a company name (e.g., "Cursor")
- Steps 1, 2, and 3 run in parallel
- Step 4 synthesizes into a prioritized insight report
- Output: structured competitive brief with opportunities and risks

Time to build: About 7 minutes.
Equivalent code: This would be a multi-day project involving web scraping, Reddit API integration, and a complex synthesis step.

[Link: /use-cases/research]

Visual Builder vs. Coding It Yourself: An Honest Comparison

FACTOR	VISUAL BUILDER (AGENTSTACK)	CODING IT YOURSELF
Setup time	Minutes	Days to weeks
Orchestration logic	Handled by platform	You write it

FACTOR	VISUAL BUILDER (AGENTSTACK)	CODING IT YOURSELF
Error handling & retries	Built-in	You write it
State management	Built-in	You write it
Monitoring & logging	Built-in dashboard	You build it (or integrate Datadog, etc.)
Flexibility	High -- conditions, branches, parallel, HITL	Unlimited -- full code control
Maintenance burden	Low -- platform updates handle infra	High -- you own every line
Who can build it	Developers, PMs, agency ops	Developers only
Cost visibility	Per-step token and cost tracking	You build your own metering

The honest truth: if you need custom tool integrations that do not exist yet, or you need sub-second latency for real-time inference, coding it yourself gives you more control. For everything else -- and that is 90% of use cases -- the visual builder is faster, cheaper, and more maintainable.

When You Outgrow No-Code

AgentStack does not lock you into no-code. Every workflow also has a full API. [Link: /docs/api]

You can:
- Trigger any workflow via a REST API call from your existing codebase
- Build hybrid workflows where some steps are visual agents and others call your own custom endpoints
- Export workflow definitions as JSON for version control

Start visual, go programmatic when you need to. You do not have to choose one or the other.

Get Started in 5 Minutes

Building an AI agent workflow does not require a framework, a DevOps pipeline, or a weekend of coding. With the right platform, it takes five minutes and zero lines of code.

AgentStack gives you 82 pre-built agents, a visual workflow builder, and bring-your-own-key pricing so you are not paying a markup on every token. [Link: /pricing]

Ready to build your first AI agent workflow?

[CTA: Sign up free -- /signup]

No credit card required. 5 agents and 100 executions included on the free plan.

BLOG POST 2

SEO Title Tag: BYOK vs Managed AI: Why Bring Your Own Key Saves You Money | AgentStack

Meta Description: BYOK vs managed AI platforms: real cost comparisons show bring your own API key saves 60-80%. See GPT-4o and Claude pricing breakdowns.

URL Slug: /blog/byok-vs-managed-ai-bring-your-own-key

Focus Keyword: bring your own API key

Secondary Keywords: BYOK AI, bring your own key AI platform, managed AI pricing, AI agent cost comparison, BYOK vs managed

Schema Markup: Article, FAQPage

BYOK vs Managed AI: Why Bring Your Own Key Saves You Money

Every AI agent platform needs to make money. The question is how they make it -- and whether you notice.

Most managed AI platforms charge you per token, per execution, or per "credit" -- and buried inside that price is a 2-5x markup on the underlying model cost. You are not just paying OpenAI or Anthropic for inference. You are paying the platform's margin on top of it. Every agent run, every workflow execution, every experiment -- you are subsidizing someone else's business model with your API calls.

Bring Your Own Key (BYOK) is a different architecture. You plug in your own API keys from model providers. The platform uses your keys to call the models directly. You pay the model providers at their published rates, and you pay the platform a flat fee for the software -- not a per-token tax.

This post breaks down exactly how BYOK works, how much it saves compared to managed platforms, when managed actually makes more sense, and how AgentStack implements BYOK so you keep control of your keys, your models, and your margin.

What Does "Bring Your Own Key" Actually Mean?

BYOK means you provide your own API keys from AI model providers -- OpenAI, Anthropic, Google, Mistral, Groq, or any OpenAI-compatible provider -- and the platform uses those keys to make inference calls on your behalf.

Here is what happens under the hood:

1. You create an account on the AI platform (e.g., AgentStack)
2. You go to Settings and paste in your OpenAI API key, your Anthropic API key, or both
3. When you run an agent or workflow, the platform sends requests to the model provider using your key
4. The model provider bills you directly at their published per-token rate
5. The platform charges you a flat monthly fee for access to agents, the workflow builder, and monitoring

Your keys are encrypted at rest and never logged, shared, or used for any purpose other than making inference calls on your behalf. [Link: /docs/byok-setup]

The critical difference: the platform never touches the token billing. You pay OpenAI directly. You pay Anthropic directly. The platform fee is separate and flat.

How Managed Platforms Mark Up Your Tokens

Managed AI platforms -- the ones that give you "credits" or charge per execution -- work differently. They have their own API keys with the model providers. When you run an agent, the platform makes the inference call using their key, and they bill you at their rate -- not the provider's rate.

The markups vary, but here is what we see across the industry:

- **Low-end markup:** 2x the base model cost (you pay $0.010/1K tokens instead of $0.005/1K)
- **Mid-range markup:** 3x the base model cost

- **High-end markup:** 5x+ the base model cost (common in "credits" systems where the pricing is deliberately opaque)

Some platforms obscure this by using proprietary credit systems. One "credit" might cost $0.01 but only buy you 200 tokens of GPT-4o -- which is effectively a 4x markup over OpenAI's published rate.

This is not inherently dishonest. Managed platforms provide value: they handle infrastructure, scaling, and reliability. But you should know what you are paying for.

Real Cost Comparisons: BYOK vs. Managed

Let us do the math with real numbers. We will use the published rates for the two most popular models as of early 2025.

Model Pricing (Direct from Provider)

MODEL	INPUT (PER 1M TOKENS)	OUTPUT (PER 1M TOKENS)
GPT-4o (OpenAI)	$2.50	$10.00
Claude 3.5 Sonnet (Anthropic)	$3.00	$15.00
GPT-4o Mini (OpenAI)	$0.15	$0.60
Gemini 2.0 Flash (Google)	$0.10	$0.40

Scenario: 1 Million Tokens Per Day (Mixed Input/Output)

Assume a typical workload: 700K input tokens and 300K output tokens per day, using GPT-4o.

Direct cost (BYOK):
- Input: 0.7M x $2.50/M = $1.75/day
- Output: 0.3M x $10.00/M = $3.00/day
- **Total: $4.75/day = $142.50/month**

Managed platform at 3x markup:
- Input: 0.7M x $7.50/M = $5.25/day
- Output: 0.3M x $30.00/M = $9.00/day
- **Total: $14.25/day = $427.50/month**

Managed platform at 5x markup:
- Input: 0.7M x $12.50/M = $8.75/day
- Output: 0.3M x $50.00/M = $15.00/day
- **Total: $23.75/day = $712.50/month**

Full Cost Comparison Table (GPT-4o, 1M tokens/day)

COST COMPONENT	BYOK (AGENTSTACK)	MANAGED (3X)	MANAGED (5X)
Monthly token cost	$142.50	$427.50	$712.50
Platform fee	$49.00	$0 (included)*	$0 (included)*
Monthly total	**$191.50**	**$427.50**	**$712.50**
Annual total	**$2,298**	**$5,130**	**$8,550**
Annual savings with BYOK	--	**$2,832**	**$6,252**

*Managed platforms typically do not charge a separate platform fee -- the markup IS the fee. That is why it feels "free" until you look at the per-token pricing.

What About Lower Volume?

Even at 100K tokens/day (a light workload -- maybe 20-30 agent runs), the savings are meaningful:

VOLUME	BYOK + $49/MO	MANAGED (3X)	YOU SAVE
100K tokens/day	$63.25/mo	$42.75/mo	-$20.50/mo (managed is cheaper)
500K tokens/day	$120.75/mo	$213.75/mo	$93.00/mo
1M tokens/day	$191.50/mo	$427.50/mo	$236.00/mo
5M tokens/day	$761.50/mo	$2,137.50/mo	$1,376.00/mo

The crossover point is around 200-300K tokens per day. Below that, the $49/month platform fee makes BYOK slightly more expensive than a managed platform with a 3x markup. Above that, BYOK saves you more every month -- and the savings compound as your usage grows.

The Security Argument for BYOK

Cost is the obvious benefit. Security is the underrated one.

With managed platforms, your data flows through the platform's API keys. That means:
- The platform can see your prompts and completions
- The platform's API key is a shared resource -- a breach exposes every customer
- You have no control over which model version, endpoint, or region your calls are routed to
- Your usage data is part of the platform's aggregate analytics

With BYOK:
- **Your keys, your account.** Your API usage is associated with your own provider account, not a shared pool.
- **Direct data path.** Your prompts and completions go directly to the model provider. The platform sees the request metadata (token count, latency) for logging, but does not store the full prompt or response content.
- **Your rate limits.** You control your own rate limits and usage tiers with the model provider.
- **Your compliance posture.** For teams subject to SOC 2, HIPAA, or GDPR, using your own API keys with your own provider agreements simplifies the compliance chain. [Link: /security]

When Managed Makes More Sense Than BYOK

BYOK is not always the right choice. Here are the scenarios where managed platforms earn their markup:

1. You Are Just Getting Started

If you are running fewer than 200K tokens per day, the math favors managed platforms -- the per-token premium is less than a flat platform fee. Start managed, switch to BYOK when your usage grows.

2. You Do Not Want to Manage API Keys

Some teams -- especially non-technical teams -- do not want to create accounts with OpenAI, manage billing, rotate keys, or monitor usage. Managed platforms abstract all of that away. Convenience has a price, and sometimes it is worth paying.

3. You Need Fine-Tuned or Custom Models

Some managed platforms offer proprietary fine-tuned models that you cannot access through a standard API key. If the model is the differentiator, the markup is the price of access.

4. You Need Guaranteed SLAs

Managed platforms with enterprise tiers often provide uptime SLAs, dedicated support, and priority API access. With BYOK, your reliability depends on the model provider's API -- and their rate limits apply to your key.

How AgentStack Implements BYOK

AgentStack uses a clean BYOK architecture: [Link: /features/byok]

1. **Key storage.** Your API keys are encrypted with per-user encryption keys (AES-256) and stored in an isolated secrets vault. Keys are never logged, never displayed after initial entry, and never shared across users.
2. **Direct calls.** When an agent runs, AgentStack constructs the API request and sends it directly to the model provider using your key. The platform processes the response for formatting and logging, but does not cache or store prompt/completion content beyond the execution window.
3. **Multi-provider support.** You can plug in keys from OpenAI, Anthropic, Google, Mistral, Groq, or any OpenAI-compatible provider. Different agents in the same workflow can use different providers -- run your code generation step on Claude and your summarization step on GPT-4o Mini. [Link: /docs/model-providers]
4. **Per-step cost tracking.** Every step in every workflow logs the token count and estimated cost based on the provider's published rates. You see exactly what each agent run costs, broken down by input and output tokens. [Link: /features/execution-logs]
5. **No token markup. Ever.** AgentStack charges $49/month for Pro (all 82 agents, unlimited executions) or $0 for Free (5 agents, 100 executions). There is no per-token fee, no credit system, no hidden usage charges. [Link: /pricing]

The Model Flexibility Advantage

Here is a benefit of BYOK that people overlook: you can switch models instantly.

With managed platforms, switching from GPT-4o to Claude means hoping the platform supports it -- and if they do, the markup applies to the new model too. Some platforms lock you into a single provider.

With BYOK on AgentStack, you plug in a new key and pick a different model from the dropdown. That is it. You can:

- Use **GPT-4o** for complex reasoning tasks (code architecture, system design)

- Use **Claude 3.5 Sonnet** for long-context tasks (document analysis, code review on large files)
- Use **GPT-4o Mini** for high-volume, low-complexity tasks (formatting, classification)
- Use **Gemini 2.0 Flash** for speed-sensitive tasks where cost matters most
- Use **Groq** (Llama 3) for open-source model experimentation at near-zero cost

Mix and match within a single workflow. Each agent step can use a different model from a different provider. Optimize for quality where it matters, optimize for cost where it does not.

How to Get Started with BYOK on AgentStack

It takes about 90 seconds:

1. **Sign up** at agentstack.com (free plan, no credit card)
2. Go to **Settings > API Keys**
3. Paste your OpenAI, Anthropic, or Google API key
4. Pick an agent, run it, and watch the cost breakdown in real time

If you do not have a model provider API key yet, here is where to get one:
- OpenAI: platform.openai.com
- Anthropic: console.anthropic.com
- Google: aistudio.google.com

Each provider offers free credits or a free tier to get started.

The Bottom Line

If you are running AI agents at any meaningful scale -- more than a few hundred thousand tokens per day -- the managed platform markup costs you thousands of dollars per year. BYOK eliminates that markup entirely.

You keep your keys. You keep your margin. You keep the flexibility to switch models, providers, and pricing tiers whenever you want.

Ready to stop paying the AI platform tax?

[CTA: Start with BYOK on AgentStack -- /signup]

Free plan available. Bring your own keys from day one.

BLOG POST 3

SEO Title Tag: The Complete Guide to AI Agents for Agencies in 2025 | AgentStack

Meta Description: How agencies use AI agents to scale client work, cut costs, and deliver faster. Covers workflows, white-labeling, ROI math, and getting started.

URL Slug: /blog/ai-agents-for-agencies-guide

Focus Keyword: AI agents for agencies

Secondary Keywords: AI agents agency workflow, AI automation for agencies, agency AI tools, white-label AI agents, AI agent platform for agencies

Schema Markup: Article, FAQPage

The Complete Guide to AI Agents for Agencies in 2025

Agencies have a margin problem. Clients want more output for less budget. Your team is already at capacity. Hiring another developer or designer takes months and adds $80-150K in annual salary before that person ships a single deliverable.

AI agents are the leverage that changes this math.

Not generic chatbots -- specialized AI agents that handle specific jobs in your delivery workflow. A code review agent that audits every pull request before your senior dev touches it. A content agent that produces first drafts from a brief in minutes instead of days. A QA agent that catches accessibility violations, design mismatches, and broken flows before the client sees them.

This guide covers everything agencies need to know about AI agents in 2025: which types of agents matter for agency work, how to wire them into client workflows, the white-labeling question, hard ROI numbers, and how to get started without disrupting your current process.

Why Agencies Need AI Agents Now

Three forces are converging in 2025 that make AI agents essential for agencies, not optional:

1. Client Expectations Have Outpaced Headcount

Clients have seen what AI can do. They expect faster turnarounds, more iterations, and lower costs. They are not wrong -- AI does make this possible. But they expect you to deliver it with the same team size and the same budget.

AI agents let you meet those expectations without burning out your team. A three-person agency can deliver the output of a six-person team when the repetitive, structured work is handled by agents.

2. The Talent Gap Is Not Closing

Hiring senior developers, experienced designers, and skilled QA engineers is harder than ever. The talent market is tight, salaries are high, and onboarding takes months. Meanwhile, your project pipeline does not wait.

AI agents do not replace your senior people -- they amplify them. Your senior developer spends time on architecture decisions instead of reviewing boilerplate code. Your lead designer focuses on creative direction instead of checking spacing on 47 breakpoints.

3. Commodity Work Is a Race to the Bottom

If your agency charges for hours spent on commodity tasks -- basic code generation, routine QA, first-draft content -- you are competing with every offshore team and every AI tool on the market. The margin on commodity work is going to zero.

The agencies that thrive in 2025 are the ones that automate commodity work with AI agents and charge for strategy, judgment, and creative direction. AI handles the volume; humans handle the value.

Types of AI Agents Useful for Agencies

Not every AI agent is relevant for agency work. Here are the categories that move the needle, organized by where they fit in a typical delivery workflow. [Link: /features/agents]

Engineering Agents

AGENT	AGENCY USE CASE
Backend Systems Engineer	Generate API endpoints, database schemas, and service logic from client specs. First drafts in minutes.
Frontend Code Generator	Produce React/Next.js components from design specs. Speeds up the design-to-code handoff.
Code Reviewer	Automated PR review for every commit. Catches bugs and security issues before your senior dev reviews.
Test Writer	Generate unit and integration tests from source code. Clients get better test coverage without extra billable hours.
Database Migration Engineer	Schema migrations across Prisma, Drizzle, and raw SQL with zero-downtime strategies.

Design Agents

AGENT	AGENCY USE CASE
Design QA Specialist	Validate implemented UI against Figma specs across all breakpoints. Catches what human eyes miss.
Wireframe & Prototype Specialist	Convert client requirements into wireframes with interaction notes. Faster concepting, more iterations.
Accessibility QA Specialist	WCAG 2.1/2.2 AA audits on every deliverable. Compliance built into the workflow, not bolted on at the end.

Content & Growth Agents

AGENT	AGENCY USE CASE
Content & Copy Strategist	First-draft landing pages, email sequences, and UX copy from briefs. Writers edit instead of starting from scratch.
SEO Strategist	Keyword research, content briefs, meta tag optimization, and internal linking strategy for client sites.
Email Marketing Specialist	Onboarding sequences, retention campaigns, and transactional emails from a campaign brief.

QA & Review Agents

AGENT	AGENCY USE CASE
Senior Cross-Agent Reviewer	Final quality check on agent-generated output before client delivery. The safety net.
AppSec Audit Engineer	Security audits on client codebases. OWASP Top 10 review without hiring a security consultant.
Legal Compliance Advisor	GDPR, CCPA, SOC 2 compliance checks on features before launch. Reduces client legal risk.

Client Workflow Examples

Here is how real agency workflows look with AI agents integrated. These are not hypothetical -- they are workflows you can build today in AgentStack's visual workflow builder. [Link: /features/workflow-builder]

Workflow 1: Client Website Build

Traditional process: Designer creates mockups (1 week) > Developer builds frontend (2 weeks) > QA reviews (3 days) > Revisions (1 week) > Launch.

With AI agents:
1. **Wireframe & Prototype Specialist** generates initial wireframes from the client brief (30 minutes)
2. Designer refines wireframes into final mockups (3 days instead of 5)
3. **Frontend Code Generator** produces React components from the approved designs (2 hours)
4. Developer refines and integrates (1 week instead of 2)
5. **Design QA Specialist** + **Accessibility QA Specialist** run in parallel (1 hour)
6. Developer fixes flagged issues (1 day)
7. Launch

Time saved: 12 days reduced to 7 days. Same quality, 42% faster delivery.

Workflow 2: Ongoing Content Retainer

Traditional process: Strategist writes brief (2 hours) > Writer produces draft (4 hours) > SEO review (1 hour) > Editor finalizes (1 hour). Per post: 8 hours.

With AI agents:
1. **SEO Strategist** produces keyword research and content brief (15 minutes)
2. Strategist reviews and adjusts brief (30 minutes)
3. **Content & Copy Strategist** writes first draft from brief (10 minutes)
4. Writer edits and adds expertise (1.5 hours)
5. **SEO Strategist** optimizes on-page elements (5 minutes)
6. Editor finalizes (30 minutes)

Time per post: 2.5 hours instead of 8. At 12 posts/month, that is 66 hours saved -- nearly a full person's capacity.

Workflow 3: Code Audit for a New Client

Traditional process: Senior developer spends 2-3 days manually reviewing the codebase, writing up findings, and prioritizing recommendations.

With AI agents:
1. **Code Reviewer** scans the entire codebase for code quality issues (20 minutes)
2. **AppSec Audit Engineer** runs a security audit (15 minutes)
3. **Performance Engineering Lead** profiles frontend and backend performance (15 minutes)
4. **Codebase Refactor Analyst** identifies tech debt and produces a prioritized refactoring plan (10 minutes)
5. **Senior Cross-Agent Reviewer** merges all reports into a single deliverable (5 minutes)
6. Senior developer reviews the merged report and adds judgment (2 hours)

Time saved: 3 days reduced to 3 hours. The senior developer focuses on strategic recommendations instead of line-by-line scanning.

White-Labeling: The Agency Question

Every agency asks the same question: "Can I put my brand on the output?"

AgentStack's Team plan (coming soon at $149/month) includes white-label agent outputs. That means: [Link: /pricing]

- Agent-generated deliverables carry your agency's branding, not AgentStack's
- Client workspaces are separated with role-based access control
- Clients see the output, not the tooling behind it
- Audit logs for compliance and accountability

On the current Pro plan ($49/month), you can run all workflows for clients -- the output is yours to deliver however you want. The only difference is that the Team plan adds formal white-labeling controls, SSO, and workspace separation.

For most agencies, the Pro plan is enough to start. Upgrade to Team when you need client-facing workspaces.

ROI Calculation: The Agency Math

Let us do the numbers for a mid-size digital agency (5-10 people).

Assumptions

- Average fully loaded cost per team member: $100,000/year ($8,333/month)
- AI agents handle 30% of commodity work (conservative estimate)
- The agency has 6 delivery team members

Cost of AI Agents

- AgentStack Pro: $49/month
- Token costs (BYOK, moderate usage): ~$150/month
- **Total AI cost: ~$200/month**

Value of Time Saved

- 30% of 6 team members' commodity work = 1.8 FTE equivalent
- 1.8 x $8,333/month = **$15,000/month in recovered capacity**
- That capacity either reduces overtime/burnout or lets you take on more clients without hiring

ROI

- Monthly investment: $200
- Monthly value: $15,000
- **ROI: 75x** (or 7,400%)

Even if you cut that estimate in half -- say agents only handle 15% of commodity work -- the ROI is still 37x. The math works at every reasonable assumption.

What That Means in Practice

With $200/month in AI tooling, a 6-person agency can:
- Take on 1-2 additional clients per month without hiring
- Deliver 40% faster on existing projects (clients notice this)
- Reduce revision cycles by catching issues before human review
- Offer new services (security audits, SEO, accessibility compliance) without specialist hires

How to Start with AgentStack as an Agency

Do not try to automate everything on day one. Here is the phased approach that works:

Phase 1: Pick One Workflow (Week 1)

Choose the workflow that is highest volume and most repetitive. For most agencies, this is one of:
- Code review (every PR, every project)
- Content first drafts (every blog post, every landing page)
- QA checks (every feature, every deployment)

Build that one workflow in AgentStack. Run it alongside your existing process for a week. Compare output quality and time savings.

Phase 2: Expand to Adjacent Workflows (Weeks 2-4)

Once you trust the first workflow, add two or three more. Common second moves:
- Add a Design QA agent to your frontend delivery pipeline
- Add a Test Writer agent to your code review workflow
- Add an SEO agent to your content pipeline

Phase 3: Standardize and Scale (Month 2+)

Make agent workflows part of your standard operating procedures. Every new project gets the same agent-powered pipeline. Train your team on the workflow builder so they can customize workflows per client.

At this point, agents are not a tool your team uses occasionally -- they are infrastructure your agency runs on.

Frequently Asked Questions

Do AI agents replace my team?

No. AI agents handle commodity work -- the repetitive, structured tasks that eat up your team's time. Your team focuses on strategy, client relationships, creative direction, and the judgment calls that AI cannot make. Agents amplify your team; they do not replace them.

Can I use this for client work without telling clients?

Yes. The output is yours. How you produce it is your business. Many agencies already use AI tools in their workflows without disclosing the specific tooling. If a client asks, the honest answer is: "We use AI-assisted workflows to deliver faster and at higher quality."

What if the AI agent output is not good enough?

Agent output is a first draft, not a final deliverable. Your team reviews, refines, and adds expertise before anything reaches the client. The value is in the time saved on the first 80% of the work -- not in removing humans from the process.

How does pricing work for agencies?

AgentStack Pro is $49/month with BYOK (bring your own API keys). You pay model providers directly for token usage. There is no per-client fee, no per-seat fee on Pro, and no execution limits. The Team plan at $149/month adds white-labeling, SSO, and client workspaces. [Link: /pricing]

Start Your Agency on AgentStack

The agencies that adopt AI agents in 2025 will outperform the ones that wait. Not because the technology is magic -- but because the economics are undeniable. $200/month in tooling recovers tens of thousands in team capacity.

AgentStack gives you 82 pre-built agents, a visual workflow builder, and BYOK pricing that keeps your margins intact. Start with one workflow, prove the ROI, and scale from there.

Ready to give your agency AI-powered leverage?

[CTA: Start your agency on AgentStack -- /signup]

Free plan available. No credit card required.

BLOG POST 4

SEO Title Tag: 82 AI Agents That Cover Every Role in a Product Team | AgentStack

Meta Description: Explore 82 specialized AI agents across engineering, design, product, QA, and growth. See how they work together to replace an entire product team stack.

URL Slug: /blog/82-ai-agents-product-team

Focus Keyword: AI agents product team

Secondary Keywords: AI agents for product development, AI agent catalog, specialized AI agents, AI team roles, multi-agent product team

Schema Markup: Article

---

82 AI Agents That Cover Every Role in a Product Team

A typical product team has developers, designers, a product manager, QA engineers, a DevOps lead, and a handful of specialists who show up when things break or when you are about to launch. Building that team takes months of hiring, hundreds of thousands in salary, and constant coordination overhead.

What if you could spin up every role you need from a catalog -- pre-trained, specialized, and ready to produce output in minutes?

That is exactly what AgentStack's 82-agent catalog delivers. Each agent is purpose-built for a specific job on a product team. Not a generic assistant with a system prompt -- a focused specialist with the right tools, context, and output format for its role.

This post walks through all 82 agents organized by category, spotlights the six most popular agents with detailed use cases, shows how agents work together in workflows, and runs the math on AI agents versus traditional hiring.

The Full Agent Catalog: 82 Agents, 9 Categories

Here is every agent in the AgentStack catalog, organized by the team function it covers. [Link: /features/agents]

Engineering (18 Agents)

The engineering category is the largest because software delivery has the most discrete, automatable tasks.

1. **Backend Systems Engineer** -- Generates production-grade backend code: APIs, services, data models, database queries across Python, Node, Go, and Rust.
2. **Frontend Code Generator** -- Writes React, Next.js, Vue, and Svelte components from specs or designs with proper state management and accessibility.
3. **Code Reviewer** -- Reviews pull requests for bugs, performance issues, security vulnerabilities, and style violations. Returns structured, actionable feedback.
4. **Code Architect** -- Designs system architecture: service boundaries, data flow, API contracts, and infrastructure topology.
5. **Debugger** -- Analyzes stack traces, error logs, and reproduction steps to pinpoint root causes and suggest fixes.
6. **Test Writer** -- Generates unit, integration, and e2e tests covering happy paths, edge cases, and failure modes.
7. **Codebase Refactor Analyst** -- Identifies tech debt, dead code, and structural issues. Produces prioritized refactoring plans.
8. **Performance Engineering Lead** -- Audits frontend and backend performance: bundle size, Core Web Vitals, query latency, caching, load testing.
9. **Database Migration Engineer** -- Writes and sequences schema migrations across Prisma, Drizzle, TypeORM, Alembic, and raw SQL.
10. **Data Pipeline ETL Engineer** -- Designs ETL/ELT pipelines using dbt, Airflow, Dagster, and Prefect with data quality checks.
11. **Mobile Engineering Specialist** -- Builds React Native, Flutter, Swift, and Kotlin components with platform-specific optimizations.
12. **Desktop Engineering Specialist** -- Develops Electron and Tauri desktop apps with native OS integration and auto-update systems.
13. **API Contract Spec Writer** -- Generates OpenAPI, GraphQL, and gRPC specs. Produces client SDKs, mock servers, and validation schemas.
14. **Prompt Engineer** -- Designs, tests, and optimizes prompts and prompt chains for LLM-powered features with evaluation harnesses.
15. **Localization Engineer** -- Implements i18n frameworks, ICU MessageFormat, CLDR pluralization, RTL layouts, and TMS integration.

16. **Contract Testing Specialist** -- Sets up consumer-driven contract tests with Pact, PactFlow, and Spring Cloud Contract.
17. **Analytics Event Tracking Engineer** -- Designs tracking plans and instruments events across Segment, Mixpanel, PostHog, Amplitude, and GA4.
18. **Data Architecture Engineer** -- Designs data models, warehouse schemas, and data governance frameworks across PostgreSQL, MongoDB, Snowflake, and BigQuery.

Design (7 Agents)

1. **UI/UX Design Lead** -- Produces full-fidelity design specs: layout, spacing, typography, color, interaction states, and responsive breakpoints.
2. **Brand Identity Designer** -- Creates brand systems: logos, color palettes, typography scales, iconography guidelines, and brand usage docs.
3. **Wireframe & Prototype Specialist** -- Converts PRDs and user stories into lo-fi, mid-fi, and hi-fi wireframes with interaction notes.
4. **Design System Architect** -- Manages design tokens, component libraries, theming, and cross-platform consistency.
5. **Design QA Specialist** -- Validates implemented UI against Figma specs. Catches spacing, color, typography, and brand violations.
6. **Motion & Interaction Designer** -- Specs animations, transitions, micro-interactions, and loading states with timing curves and easing functions.
7. **Accessibility QA Specialist** -- Runs WCAG 2.1/2.2 AA audits: screen reader testing, keyboard nav, ARIA validation, color contrast, focus management.

Product (6 Agents)

1. **AI Product Studio PM** -- Owns the product lifecycle: PRDs, success metrics, backlog prioritization, and cross-functional delivery.
2. **Requirements Analyst** -- Translates stakeholder input into structured requirements: functional specs, acceptance criteria, edge cases.
3. **User Story Writer** -- Generates user stories with acceptance criteria, test scenarios, and technical notes ready for sprint planning.
4. **Product Metrics Architect** -- Defines KPI frameworks, instrumentation plans, and dashboard specs for measuring feature impact.
5. **Onboarding Specialist** -- Designs onboarding flows: step sequences, progress indicators, tooltip tours, activation milestones.
6. **UX Research Specialist** -- Creates research plans, interview scripts, survey designs, usability test protocols, and synthesizes findings.

Infrastructure and DevOps (7 Agents)

1. **Terraform IaC Engineer** -- Writes production-ready Terraform, Pulumi, and CDK configs for networking, compute, databases, and secrets.
2. **Cloud Infrastructure Architect** -- Designs multi-cloud architectures across AWS, GCP, and Azure: VPCs, load balancers, auto-scaling, DR.
3. **DevOps Release Agent** -- Manages CI/CD pipelines, deployment strategies (blue-green, canary, rolling), and release gating.
4. **Incident Response Commander** -- Handles the full incident lifecycle: SEV classification, runbook execution, status pages, and postmortems.
5. **FinOps Cost Optimization** -- Analyzes cloud spend, identifies rightsizing opportunities, models reserved instance savings.
6. **MLOps Engineer** -- Manages model versioning, deployment pipelines, drift monitoring, and LLM serving infrastructure.
7. **Performance Engineering Specialist** -- Profiles full-stack performance: API latency, database queries, caching, and load testing.

QA and Security (5 Agents)

1. **QA & Security Gatekeeper** -- Enforces quality gates: test coverage thresholds, security scan results, and release readiness checks.
2. **Feature Test Execution Specialist** -- Runs structured test passes: functional, regression, boundary, and cross-browser/cross-device testing.
3. **AppSec Audit Engineer** -- Application security audits: OWASP Top 10, dependency scanning, secrets detection, remediation guidance.
4. **Senior Cross-Agent Reviewer** -- Reviews output from other agents for accuracy, consistency, and quality. The final check before human handoff.
5. **Legal Compliance Advisor** -- Evaluates features for GDPR, CCPA, SOC 2, HIPAA. Flags risks and suggests compliant alternatives.

Growth and Marketing (7 Agents)

1. **Content & Copy Strategist** -- Writes UX microcopy, landing pages, email sequences, and content briefs.
2. **SEO Strategist** -- Keyword research, meta tags, content structure, internal linking strategies, and ranking performance.
3. **Social Media Content Manager** -- Platform-specific content calendars, posts, hashtag strategies, and engagement analytics.
4. **Email Marketing Specialist** -- Welcome sequences, activation campaigns, retention, re-engagement, and transactional emails.

5. **Growth Experiments Strategist** -- A/B tests, growth loops, activation experiments with hypotheses and statistical significance thresholds.
6. **GTM Sellability Analyst** -- Product-market fit signals, positioning, competitive differentiation, and go-to-market readiness.
7. **Pricing & Monetization Strategist** -- Pricing tier models, willingness-to-pay analysis, freemium gates, and competitor benchmarks.

Research and Strategy (5 Agents)

1. **Market Validation Analyst** -- Validates product ideas against market data: TAM/SAM/SOM, demand signals, and viability scoring.
2. **Competitor Intelligence Analyst** -- Monitors competitor products, pricing, positioning, and feature releases.
3. **Unbuilt Idea Hunter** -- Scans forums, communities, and social platforms for unmet needs and product opportunities.
4. **Reddit Market Intelligence Researcher** -- Mines Reddit for sentiment, pain points, feature requests, and competitive mentions.
5. **Feedback Intelligence Synthesizer** -- Aggregates user feedback from tickets, reviews, surveys, and interviews into themed insight reports.

Coordination (8 Agents)

1. **Sprint Planning Agent** -- Owns the sprint lifecycle: ticket creation, backlog grooming, velocity tracking, standups, and retrospectives.
2. **Cross-Functional Orchestrator** -- Coordinates multi-team delivery: dependency mapping, blockers, and milestone tracking.
3. **Standup Summarizer** -- Produces structured standup summaries from async updates across Slack, Linear, and GitHub.
4. **Retrospective Facilitator** -- Runs structured retros: what went well, what did not, action items with owners and deadlines.
5. **Meeting Notes Agent** -- Transcribes and summarizes meetings into structured notes with action items and decisions.
6. **Technical Documentation Writer** -- API references, ADRs, runbooks, READMEs, SDK guides, and changelogs.
7. **Changelog Generator** -- Produces human-readable changelogs from git commits, PR descriptions, and release tags.
8. **Knowledge Base Curator** -- Organizes and maintains internal documentation, FAQs, and process guides.

Specialized (6 Agents)

1. **Billing & Subscription Manager** -- Stripe/Paddle integration: checkout flows, subscription CRUD, webhooks, dunning, and revenue analytics.
2. **Customer Support Agent** -- Handles tier-1 support: troubleshooting, FAQ responses, ticket routing, and escalation.
3. **Competitive Pricing Monitor** -- Tracks competitor pricing changes, feature gate adjustments, and promotional offers.
4. **Investor Update Writer** -- Produces structured investor updates: metrics, milestones, asks, and narrative.
5. **Internal Comms Drafter** -- Company announcements, team updates, policy changes, and all-hands summaries.
6. **SaaS Customer Support Agent** -- SaaS-specific support covering billing, account management, feature requests, and integration troubleshooting.

Note: The remaining agents (70-82) are variations and advanced specialists within the categories above, covering niche roles like chaos engineering design, migration specialists, and domain-specific analysts. The full catalog is available at [Link: /features/agents].

Spotlight: The 6 Most Popular Agents

Out of 82 agents, these six are activated first by most users -- and for good reason.

1. Code Reviewer

Why it is popular: Every team writes code. Not every team has the bandwidth to review every line of it.

The Code Reviewer agent scans pull requests for bugs, performance issues, security vulnerabilities, and style violations. It returns structured feedback organized by severity -- critical issues first, nitpicks last.

Use case in practice: A three-person startup connects the Code Reviewer to their GitHub repo. Every PR gets an automated first-pass review within minutes of opening. The founding engineer only needs to review the agent's flagged issues instead of reading every diff line by line. Result: PR review time drops from 45 minutes to 10 minutes per PR, and critical bugs get caught before they hit staging.

Best paired with: AppSec Audit Engineer (security focus), Test Writer (auto-generate tests for flagged code).

2. Backend Systems Engineer

Why it is popular: Backend code is where most development time goes -- and where AI agents produce the most usable output.

This agent generates production-grade backend code: REST and GraphQL APIs, service logic, data models, and database queries. It works across Python, Node.js, Go, and Rust, and follows the patterns and conventions you specify in the context.

Use case in practice: An agency receives a client spec for a new API with 12 endpoints. The Backend Systems Engineer generates the initial implementation -- route handlers, validation schemas, database queries, and error handling -- in under 10 minutes. The senior developer reviews, adjusts business logic, and ships. A 3-day task becomes a 4-hour task.

Best paired with: Code Reviewer (review the generated code), Test Writer (generate tests for it), API Contract Spec Writer (produce the OpenAPI spec).

3. Content & Copy Strategist

Why it is popular: Every product needs copy. Landing pages, emails, microcopy, blog posts, documentation -- it never stops.

The Content & Copy Strategist writes UX microcopy, landing pages, email sequences, and content briefs. It does not produce generic AI slop -- it follows your brand voice, references your product's features, and structures copy for conversion.

Use case in practice: A SaaS company needs to rewrite their entire onboarding email sequence (7 emails). They feed the agent their product docs, brand guidelines, and the existing email sequence. The agent produces a complete rewrite with subject lines, body copy, CTAs, segmentation rules, and A/B test variants. The marketing lead edits for voice and approves. A 2-week project becomes a 2-day project.

Best paired with: SEO Strategist (optimize for search), Email Marketing Specialist (full email campaign architecture).

4. Design QA Specialist

Why it is popular: The gap between Figma and production is where brand quality goes to die.

The Design QA Specialist validates implemented UI against Figma design specs. It catches spacing deviations, color inconsistencies, typography errors, and brand violations across all breakpoints and themes.

Use case in practice: An agency ships a client website redesign. Before the client review, they run the Design QA Specialist across every page. It flags 23 issues: 3 critical (wrong font on mobile header, button color does not match brand, hero image aspect ratio is off), 8 high, 12 medium. The developer fixes the critical and high issues in 2 hours. The client sees a pixel-perfect deliverable on the first review. Zero revisions.

Best paired with: Accessibility QA Specialist (WCAG compliance), Frontend Code Generator (fix flagged issues automatically).

5. Sprint Planning Agent

Why it is popular: Sprint planning is essential but tedious. Writing tickets, estimating effort, mapping dependencies, and grooming the backlog eats hours that could go toward building.

The Sprint Planning Agent takes a PRD or feature brief and produces structured tickets with acceptance criteria, effort estimates, dependency maps, and suggested sprint allocation. It integrates with Linear and GitHub Issues.

Use case in practice: A product team receives a PRD for a new feature. The Sprint Planning Agent breaks it into 14 tickets across frontend, backend, design, and QA. Each ticket has acceptance criteria, story points, and dependencies. The PM reviews, adjusts priorities, and the sprint is planned in 30 minutes instead of a 2-hour planning meeting.

Best paired with: Requirements Analyst (refine the PRD first), Cross-Functional Orchestrator (coordinate across teams).

6. Terraform IaC Engineer

Why it is popular: Infrastructure code is high-stakes, repetitive, and requires deep knowledge of provider-specific APIs. It is the perfect job for a specialized agent.

The Terraform IaC Engineer writes production-ready Terraform, Pulumi, and AWS CDK configurations for networking, compute, databases, secrets management, and CI/CD pipelines. It includes security scanning (tfsec/checkov), cost estimation, and drift detection.

Use case in practice: A startup needs to set up a production Kubernetes cluster on AWS with a VPC, RDS database, secrets management, and monitoring. The Terraform IaC Engineer generates the complete Terraform configuration, including modules, variables, and a README with deployment instructions. The DevOps engineer reviews, applies, and the infrastructure is live. A week of Terraform wrestling becomes a day of review and apply.

Best paired with: Cloud Infrastructure Architect (design the architecture first), FinOps Cost Optimization (estimate and optimize costs).

How Agents Work Together in Workflows

Individual agents are useful. Agents chained into workflows are transformative.

AgentStack's visual workflow builder lets you connect agents in sequences, parallel branches, and conditional paths. The output of one agent flows into the input of the next. [Link: /features/workflow-builder]

Example: Ship a Feature Workflow

1. **Sprint Planning Agent** breaks the PRD into tickets
2. **Backend Systems Engineer** generates the backend implementation
3. **Frontend Code Generator** produces the UI components
4. **Code Reviewer** reviews both outputs in parallel
5. **Test Writer** generates tests for the approved code
6. **Technical Documentation Writer** produces the API docs and changelog
7. **Design QA Specialist** validates the frontend against specs

Seven agents, one workflow, and a feature goes from PRD to production-ready code with tests, docs, and QA -- in hours instead of sprint cycles.

This is not about replacing humans. Your engineers still make architecture decisions, your designers still set creative direction, and your PM still defines what to build. The agents handle the volume work so your team can focus on the judgment work.

Hiring vs. AI Agents: The Numbers

Let us compare the cost of traditional hiring versus AI agents for common product team roles.

ROLE	ANNUAL SALARY (US)	AGENTSTACK EQUIVALENT	MONTHLY AGENT COST
Junior Backend Developer	$85,000	Backend Systems Engineer	~$50-150/mo (tokens)
QA Engineer	$90,000	Test Writer + Design QA + Feature Test Specialist	~$50-150/mo
Technical Writer	$80,000	Technical Documentation Writer + Changelog Generator	~$30-80/mo
DevOps Engineer (junior)	$95,000	Terraform IaC Engineer + DevOps Release Agent	~$50-150/mo

ROLE	ANNUAL SALARY (US)	AGENTSTACK EQUIVALENT	MONTHLY AGENT COST
Content Writer	$65,000	Content & Copy Strategist + SEO Strategist	~$40-100/mo

Important caveats:
- AI agents do not replace senior roles. They replace the commodity parts of junior and mid-level roles.
- You still need humans for strategy, judgment, client relationships, and creative direction.
- The agents produce first drafts and structured output. Humans refine and ship.
- The real value is not "replacing a hire" -- it is "getting 30-50% more output from your existing team."

AgentStack Pro costs $49/month for all 82 agents. Add $50-200/month in token costs (BYOK). For under $250/month, you get coverage across every function on a product team -- not a replacement for your team, but a multiplier for it. [Link: /pricing]

Getting Started

You do not need to activate all 82 agents on day one. Here is the recommended path:

1. **Start with 2-3 agents** that match your biggest time sinks (Code Reviewer + Backend Systems Engineer is the most common starting pair)
2. **Build one workflow** that chains them together
3. **Run it alongside your existing process** for a week
4. **Measure the time saved** and output quality
5. **Expand** to adjacent agents and workflows as you build confidence

The free plan includes 5 agents and 100 executions -- enough to prove the concept before you commit.

Ready to browse the full catalog?

[CTA: Browse the agent catalog -- /features/agents]

82 agents. 9 categories. Every role your product team needs.

BLOG POST 5

SEO Title Tag: How AI Agent Teams Multiply Developer Output | AgentStack

Meta Description: AI agents handle code reviews, tests, docs, and debugging so developers focus on architecture. See real workflows, productivity data, and cost analysis.

URL Slug: /blog/ai-agents-developer-productivity

Focus Keyword: AI agents developer productivity

Secondary Keywords: AI developer tools, AI code review automation, AI agent developer workflow, developer productivity tools 2025, AI pair programming agents

Schema Markup: Article

How AI Agent Teams Multiply Developer Output

Developers are expensive, overworked, and spending the majority of their time on work that is not core engineering. Code reviews, writing tests, updating docs, debugging flaky CI pipelines, writing migration scripts, responding to incidents -- these tasks are essential, but they are not the reason you hired a senior engineer at $180K per year.

The developer bottleneck is not a skill problem. It is a time allocation problem. Your best engineers spend 60-70% of their week on work that is structured, repetitive, and predictable enough for a specialized AI agent to handle -- or at least produce a solid first draft that takes minutes to review instead of hours to create from scratch.

AI agent teams change this equation. Not a single chatbot that tries to do everything. A team of specialized agents -- each focused on one job, chained together in workflows -- that handles the volume work so developers can focus on the work that actually requires a human brain: architecture decisions, system design, debugging novel problems, and shipping features.

This post covers the developer bottleneck in detail, shows how specialized agents handle the biggest time sinks, walks through real workflow examples, presents productivity data, and runs the cost math comparing agent costs to equivalent engineering output.

The Developer Bottleneck Problem

Let us look at where a typical developer's week actually goes. These numbers come from industry surveys (Stack Overflow Developer Survey, GitHub Octoverse, and Stripe's developer productivity research), and they are remarkably consistent:

ACTIVITY	% OF WEEKLY TIME	NATURE OF WORK
Writing new code (features)	25-30%	Creative, high-judgment
Code review	15-20%	Structured, repetitive
Debugging and fixing bugs	15-20%	Mix of creative and structured
Writing and updating tests	10-15%	Structured, repetitive
Documentation and changelogs	5-10%	Structured, repetitive
Meetings, standups, planning	10-15%	Coordination overhead
DevOps, CI/CD, infrastructure	5-10%	Structured, high-context

Look at the "Nature of Work" column. Only 25-30% of a developer's time goes to creative, high-judgment work -- the work that actually requires the skills you hired them for. The rest is structured work that follows predictable patterns.

This is the bottleneck. Not that developers are slow. Not that they lack skill. They are buried in work that is important but does not require their level of expertise.

How Specialized Agents Handle the Repetitive Work

The key word is "specialized." A general-purpose chatbot cannot effectively review code, write tests, AND produce documentation. It tries to do everything and does nothing well.

Specialized AI agents are different. Each one has a focused system prompt, relevant tools, domain-specific knowledge, and a structured output format designed for its specific job. Here is how they map to the biggest developer time sinks: [Link: /features/agents]

Code Reviews: From 45 Minutes to 10 Minutes Per PR

The problem: Code review is the single biggest time sink after feature development. A thorough review of a medium-sized PR (200-500 lines) takes 30-60 minutes. Most teams have 5-15 PRs per day. That is 2.5 to 15 hours of review time -- every day -- across the team.

The agent solution: The **Code Reviewer** agent scans every PR for:
- Bugs and logic errors
- Performance anti-patterns (N+1 queries, unnecessary re-renders, missing indexes)
- Security vulnerabilities (injection, auth bypass, exposed secrets)
- Style violations and inconsistencies
- Missing error handling and edge cases

It produces a structured review organized by severity: critical, high, medium, low. The human reviewer reads the agent's report and focuses attention on the critical and high items instead of reading every line of the diff.

Time savings: First-pass review time drops from 45 minutes to 10 minutes per PR. The agent catches the mechanical issues; the human catches the architectural and design issues.

Pair it with: Run the **AppSec Audit Engineer** in parallel for dedicated security review. Chain the **Test Writer** to auto-generate tests for any flagged code paths.

Test Writing: From "We'll Add Tests Later" to Automatic Coverage

The problem: Tests are the first thing that gets cut when deadlines get tight. Every developer knows they should write tests. Most do not have time to write comprehensive test suites for every feature. The result: test coverage declines, regressions increase, and the team spends more time debugging than building.

The agent solution: The **Test Writer** agent generates unit, integration, and end-to-end tests from source code or specs. It covers:
- Happy path scenarios
- Edge cases and boundary conditions
- Failure modes and error handling
- Mock setup and teardown

Feed it a new API endpoint, and it produces a complete test suite with 15-25 test cases in minutes. A developer reviews the tests, adjusts any business-logic-specific assertions, and commits.

Time savings: Writing a comprehensive test suite for a single endpoint takes a developer 2-4 hours. The agent produces a first draft in 5 minutes. The developer spends 30 minutes reviewing and adjusting. Net savings: 1.5-3.5 hours per endpoint.

Pair it with: The **Code Reviewer** agent can flag untested code paths, which the Test Writer then covers automatically.

Documentation: From "Outdated" to Always Current

The problem: Documentation is almost always out of date. Developers write it once during the initial build and rarely update it. API references drift from the actual implementation. READMEs reference deprecated methods. Changelogs are incomplete or abandoned.

The agent solution: The **Technical Documentation Writer** produces:
- API references from source code and OpenAPI specs
- Architecture Decision Records (ADRs) from design discussions
- Runbooks for operational procedures
- README files that actually reflect the current codebase
- SDK guides and integration documentation

The **Changelog Generator** produces human-readable changelogs from git commits, PR descriptions, and release tags -- so your changelog is always up to date without anyone manually writing release notes.

Time savings: A developer spends 30-60 minutes writing docs for a new feature. The agent produces the first draft in 5 minutes. The developer reviews and adds context-specific details in 10 minutes. Net savings: 20-50 minutes per feature.

The compounding effect: When docs are always current, new team members onboard faster, support tickets decrease, and integration partners can self-serve. The ROI extends far beyond the developer's time.

Debugging: From Hours of Log Diving to Targeted Root Causes

The problem: Debugging is one of the most time-consuming activities in software development. A single production bug can consume 2-8 hours of a senior developer's time -- reading logs, reproducing the issue, tracing the code path, identifying the root cause, and writing a fix.

The agent solution: The **Debugger** agent takes stack traces, error logs, and reproduction steps as input. It analyzes the code paths involved, identifies likely root causes, and suggests targeted fixes with explanations.

The **Incident Response Commander** handles the operational side: SEV classification, runbook execution, status page updates, stakeholder communication, and blameless postmortems.

Time savings: The Debugger does not always find the exact root cause (novel bugs still need human intuition). But it narrows the search space from "somewhere in the codebase" to "these 3 code paths, most likely this function." That cuts debugging time by 30-60% on average.

Infrastructure: From Terraform Wrestling to Review-and-Apply

The problem: Infrastructure code is high-stakes and tedious. Writing Terraform configs, managing state, handling provider-specific quirks, and ensuring security compliance is work that requires deep knowledge but follows predictable patterns.

The agent solution: The **Terraform IaC Engineer** generates production-ready infrastructure code:
- VPCs, subnets, security groups, and networking
- EKS/ECS/GKE clusters with proper node pool configs
- RDS/Aurora/Cloud SQL database configs with backups and failover
- IAM roles and policies following least-privilege principles
- CI/CD pipeline configs with security scanning gates

The **FinOps Cost Optimization** agent reviews infrastructure configs and running resources to identify cost savings -- rightsizing, reserved instance opportunities, and unused resources.

Time savings: A complex Terraform module that takes 1-2 days to write from scratch takes 10 minutes to generate and 2-4 hours to review, customize, and test. The review step is critical -- you should never apply AI-generated infrastructure without human review. But the time savings are still 60-80%.

Real Workflow Examples

Here is how these agents chain together in production workflows that a developer triggers once and gets a finished output. [Link: /features/workflow-builder]

Workflow 1: PR Review Pipeline

Trigger: New PR opened on GitHub

Agent chain:
1. **Code Reviewer** -- scans for bugs, performance, style (runs in parallel with step 2)
2. **AppSec Audit Engineer** -- scans for security vulnerabilities
3. **Senior Cross-Agent Reviewer** -- merges both reports into a unified, de-duplicated review
4. Output posted as a PR comment on GitHub

Result: Every PR gets a comprehensive review within 3 minutes of opening. The human reviewer reads a structured report instead of a raw diff. Review time per PR drops from 45 to 10 minutes.

Workflow 2: Feature Implementation Pipeline

Trigger: Manual -- developer inputs a feature spec or user story

Agent chain:
1. **Code Architect** -- designs the implementation approach: components, data flow, API contracts
2. **Backend Systems Engineer** -- generates the backend implementation
3. **Frontend Code Generator** -- generates the frontend components
4. **Test Writer** -- generates tests for both backend and frontend
5. **Code Reviewer** -- reviews all generated code
6. **Technical Documentation Writer** -- produces API docs and a README update

Result: A developer inputs a feature spec and gets back: a design doc, backend code, frontend code, tests, a code review report, and updated documentation. They review, adjust, and commit. A 3-day feature becomes a 1-day feature.

Workflow 3: Incident Response Pipeline

Trigger: Alert from monitoring (PagerDuty, OpsGenie, Betterstack)

Agent chain:
1. **Incident Response Commander** -- classifies severity, identifies affected services, starts the runbook
2. **Debugger** -- analyzes error logs and stack traces to identify likely root causes
3. **Incident Response Commander** -- drafts stakeholder communication and status page update
4. After resolution: **Incident Response Commander** -- writes the blameless postmortem

Result: The on-call engineer receives a structured brief within minutes of an alert: severity classification, likely root cause, suggested fixes, and a draft stakeholder update. Response time drops. Mean time to resolution (MTTR) improves.

Workflow 4: Infrastructure Provisioning Pipeline

Trigger: Manual -- DevOps engineer inputs requirements

Agent chain:
1. **Cloud Infrastructure Architect** -- designs the architecture (VPC, compute, storage, networking)
2. **Terraform IaC Engineer** -- generates the Terraform configuration
3. **FinOps Cost Optimization** -- reviews the config and estimates monthly cost
4. **Code Reviewer** -- reviews the Terraform code for best practices and security

Result: A DevOps engineer describes what they need ("Production EKS cluster with RDS, ElastiCache, and a VPN") and gets back: an architecture diagram, complete Terraform code, a cost estimate, and a review report. A week of infrastructure work becomes a day of review and apply.

Productivity Metrics: What to Expect

Based on the workflows above, here are realistic productivity improvements when AI agent teams are integrated into a development workflow:

METRIC	BEFORE AGENTS	WITH AGENTS	IMPROVEMENT
PR review time (per PR)	30-60 min	10-15 min	50-75% faster
Test writing time (per feature)	2-4 hours	30-45 min	75-85% faster
Documentation time (per feature)	30-60 min	10-15 min	65-75% faster
Debugging time (per incident)	2-8 hours	1-3 hours	40-60% faster
Infrastructure provisioning	3-5 days	0.5-1 day	70-85% faster
Feature development (end-to-end)	3-5 days	1-2 days	50-65% faster
Time spent on creative/high-judgment work	25-30%	50-60%	2x more focus time

The last row is the one that matters most. Developers go from spending 25-30% of their time on high-judgment work to 50-60%. That is a 2x increase in the time they spend on the work that creates the most value.

Cost Analysis: Agents vs. Engineering Salary

Let us quantify the economics. How much does it cost to get "engineering output" from AI agents versus hiring?

The Cost of a Developer

COST COMPONENT	JUNIOR DEV (US)	SENIOR DEV (US)
Base salary	$85,000	$180,000
Benefits + taxes (25%)	$21,250	$45,000
Equipment + tools	$5,000	$5,000
Fully loaded annual	**$111,250**	**$230,000**

COST COMPONENT	JUNIOR DEV (US)	SENIOR DEV (US)
Monthly cost	**$9,271**	**$19,167**

The Cost of AI Agent Coverage

COST COMPONENT	MONTHLY COST
AgentStack Pro (all 82 agents)	$49
Token costs -- moderate usage (BYOK, GPT-4o + Claude mix)	$100-200
Token costs -- heavy usage	$200-500
Total (moderate)	**$149-249/month**
Total (heavy)	**$249-549/month**

What Do Agents Cover vs. a Developer?

AI agents do not replace a developer. They cover the structured, repetitive portion of a developer's workload -- roughly 40-50% of weekly output (code reviews, tests, docs, first-draft implementations, infrastructure configs).

So the right comparison is not "agent vs. developer." It is "agent output vs. the equivalent portion of a developer's salary."

COMPARISON	ANNUAL COST	OUTPUT EQUIVALENT
AI agents (moderate usage)	$1,788-2,988/year	40-50% of a junior dev's structured output
Junior developer (structured work portion)	$55,625/year (50% of fully loaded)	100% of their structured output
Savings	**$52,637-53,837/year**	Same output for 3-5% of the cost

The math is not subtle. For the cost of one month of a junior developer's salary, you get a full year of AI agent coverage that handles a comparable volume of structured work.

The Real Value: Multiplying Senior Output

The bigger ROI is not replacing junior developers. It is multiplying the output of your senior developers.

A senior engineer who spends 4 hours per day on code reviews, debugging, and infrastructure maintenance is only spending 4 hours on architecture, system design, and complex feature work. With AI agents handling the first 4 hours of structured work, that same engineer can spend 6-7 hours on high-value work. That is a 50-75% increase in senior engineering output -- without hiring anyone.

At $230K/year fully loaded, getting 50% more output from each senior engineer is equivalent to hiring 0.5 additional senior engineers per person on your team. For a team of 4 senior engineers, that is 2 additional FTE-equivalents of senior engineering output for $250/month in AI tooling.

Getting Started: The 30-Minute Setup

You do not need to overhaul your development process. Here is how to start in 30 minutes:

Step 1: Sign Up and Connect Your Keys (5 minutes)

Create a free AgentStack account. Go to Settings and add your OpenAI or Anthropic API key. You are now ready to run agents. [Link: /docs/byok-setup]

Step 2: Activate Your First Two Agents (5 minutes)

Start with the **Code Reviewer** and **Backend Systems Engineer** -- they are the highest-impact agents for most developers. [Link: /features/agents]

Step 3: Build a PR Review Workflow (10 minutes)

Open the visual workflow builder. Drag the Code Reviewer onto the canvas. Connect it to your GitHub repo. Set it to trigger on new PRs. Hit save. [Link: /features/workflow-builder]

You now have automated code review on every PR.

Step 4: Run It for a Week and Measure (ongoing)

Let the workflow run alongside your existing process for one week. Compare:
- Time spent on code review before vs. after
- Number of issues caught by the agent vs. caught by humans
- Quality of the agent's feedback (is it actionable? does it catch real issues?)

Step 5: Expand (Week 2+)

Once you trust the Code Reviewer, add the Test Writer to auto-generate tests for reviewed code. Then add the Technical Documentation Writer to keep docs in sync. Build from there.

The free plan includes 5 agents and 100 executions -- enough to prove the value before upgrading to Pro ($49/month for all 82 agents and unlimited executions). [Link: /pricing]

The Shift: From Solo Developer to Team Lead

The mental model shift is this: stop thinking of yourself as a developer who uses AI tools. Start thinking of yourself as a team lead who manages a team of AI agents.

You do not write every line of code -- you review the code your agents produce. You do not write every test -- you review the test suite your agent generated. You do not manually review every PR -- you review the agent's review and add judgment where it matters.

Your job shifts from doing the work to directing the work. And that is exactly where your expertise has the most leverage.

Ready to multiply your output?

[CTA: Try AgentStack free -- /signup]

No credit card required. 5 agents and 100 executions on the free plan.

END OF BLOG POSTS

Publishing Checklist

- [ ] Run each post through a readability checker (target Flesch-Kincaid grade 8-10)
- [ ] Replace all [Link: /path] placeholders with actual internal URLs before publishing
- [ ] Replace all [CTA: text -- /path] placeholders with styled CTA buttons
- [ ] Add Article schema markup (JSON-LD) to each post: headline, datePublished, dateModified, author, image
- [ ] Add FAQPage schema markup to posts 2 and 3 (they have FAQ sections)
- [ ] Add author byline with credentials to each post (E-E-A-T signal)
- [ ] Add a "Last updated" date visible on each post
- [ ] Compress and WebP-convert any images; add descriptive alt text with keywords

- [ ] After publishing, add 3-5 internal links FROM existing high-authority pages TO each new post
- [ ] Submit all 5 URLs to Google Search Console for indexing
- [ ] Set up Google Search Console tracking for each focus keyword

Internal Linking Map

POST	LINKS TO (FROM THIS POST)	LINKS FROM (ADD TO THESE EXISTING PAGES)
Post 1: AI Agent Workflow	/features/agents, /features/workflow-builder, /docs/api, /pricing, /use-cases/code-review, /use-cases/content-creation, /use-cases/research, /signup	Homepage, /features/agents, /features/workflow-builder
Post 2: BYOK	/docs/byok-setup, /docs/model-providers, /features/byok, /features/execution-logs, /pricing, /security, /signup	Homepage (pricing section), /pricing, /docs
Post 3: Agencies	/features/agents, /features/workflow-builder, /pricing, /signup	Homepage, /pricing (Team plan section), /features/agents
Post 4: 82 Agents	/features/agents, /features/workflow-builder, /pricing, /signup	Homepage (agent section), /features/workflow-builder
Post 5: Developer Productivity	/features/agents, /features/workflow-builder, /docs/byok-setup, /pricing, /signup	Homepage, /features/agents, /docs

Cross-Linking Between Blog Posts

After all 5 posts are published, add contextual cross-links:
- Post 1 links to Post 2 (mention BYOK pricing), Post 4 (reference agent catalog)
- Post 2 links to Post 1 (mention workflow builder), Post 5 (cost analysis angle)
- Post 3 links to Post 1 (workflow examples), Post 4 (agent catalog), Post 5 (productivity metrics)
- Post 4 links to Post 1 (workflow builder), Post 3 (agency use cases), Post 5 (developer productivity)
- Post 5 links to Post 1 (workflow examples), Post 2 (cost analysis), Post 4 (agent catalog)

SECTION 6

Help Center (20 Articles)

AgentStack Help Center

> **Status:** Ship-ready | **Version:** 1.0 | **Date:** 2026-03-10
> **Total articles:** 20
> **Tone:** Clear, helpful, direct. Written for developers and agency operators. No fluff.
> **Format:** Each article is a standalone help doc with title, category tag, and full body copy.

GETTING STARTED

Article 1: Getting Started with AgentStack

Category: Getting Started
URL slug: /help/getting-started
Last updated: 2026-03-10

AgentStack is an AI agent platform with 82 pre-built agents, a visual workflow builder, and bring-your-own-key (BYOK) pricing. This guide walks you through creating your account, understanding the dashboard, and taking your first steps.

Create your account

1. Go to app.agentstack.ai and click **Start building free**.
2. Enter your email address and create a password (minimum 8 characters).
3. Check your inbox for a verification email. Click the link to confirm your account.
4. You'll land on your dashboard -- ready to go.

The free plan includes 5 agents, 100 workflow executions per month, and 3 workflows. No credit card required.

Your dashboard at a glance

Once you're in, here's what you'll see:

SECTION	WHAT IT DOES
Agent Catalog	Browse and activate agents from 82 options across 9 categories.
My Agents	View your currently activated agents.
Workflows	Create, edit, and manage multi-agent workflows.
Executions	See the history of every agent and workflow run -- inputs, outputs, tokens, cost.
Settings	Manage API keys, provider keys (BYOK), billing, and team members.
Usage	Track execution counts, token usage, and cost breakdowns over time.

First steps (recommended order)

1. **Connect your API keys.** Go to **Settings > Provider Keys** and add at least one model provider key (OpenAI, Anthropic, or Google). Agents need a model provider to run. See Connecting Your API Keys.
2. **Activate your first agent.** Go to the Agent Catalog, pick an agent, and activate it. We recommend starting with the **Backend Systems Engineer** or **Code Reviewer** -- they're on the free tier and produce immediately useful output. See Activating Your First Agent.
3. **Run a test.** After activating an agent, click **Test Run**, type a prompt, and see the result. Every run is logged in your Execution History.
4. **Build a workflow.** Once you're comfortable with individual agents, chain them together in the visual workflow builder. See Building Your First Workflow.

What is BYOK?

BYOK stands for Bring Your Own Key. Instead of paying AgentStack a per-token markup, you connect your own API keys from model providers like OpenAI, Anthropic, and Google. You pay the providers directly at their published rates. AgentStack charges for the platform -- never for your tokens.

This means your AI costs stay transparent and low, and you can switch models anytime without migrating off the platform.

Need help?

If you get stuck at any point:
- Browse this Help Center for step-by-step guides.
- Email support at **support@agentstack.ai**.
- Pro and Team plan users get priority support with faster response times.

Article 2: Connecting Your API Keys (OpenAI, Anthropic, Google)

Category: Getting Started
URL slug: /help/connecting-api-keys
Last updated: 2026-03-10

AgentStack uses a Bring Your Own Key (BYOK) model. You connect your own API keys from model providers, and agents use those keys to call the models directly. AgentStack never marks up your token costs.

This guide covers connecting keys from the three most popular providers: OpenAI, Anthropic, and Google.

Before you start

- You need an account with at least one model provider.
- You need a valid API key from that provider with billing enabled.
- You can connect multiple providers and switch between models per agent or per workflow step.

Connecting an OpenAI key

1. Go to platform.openai.com/api-keys and sign in.
2. Click **Create new secret key**. Give it a name like "AgentStack" so you can identify it later.
3. Copy the key immediately -- OpenAI only shows it once.
4. In AgentStack, go to **Settings > Provider Keys**.
5. Click **Add Provider Key**.
6. Select **OpenAI** from the provider dropdown.

7. Paste your key and click **Save**.

Models unlocked: GPT-4o, GPT-4, GPT-3.5 Turbo, and any future OpenAI models.

Connecting an Anthropic key

1. Go to console.anthropic.com/settings/keys and sign in.
2. Click **Create Key**. Name it "AgentStack."
3. Copy the key.
4. In AgentStack, go to **Settings > Provider Keys**.
5. Click **Add Provider Key**.
6. Select **Anthropic** from the provider dropdown.
7. Paste your key and click **Save**.

Models unlocked: Claude 4, Claude 3.5 Sonnet, Claude 3 Opus, and any future Anthropic models.

Connecting a Google key

1. Go to aistudio.google.com/apikey and sign in with your Google account.
2. Click **Create API Key** and select a project (or create one).
3. Copy the key.
4. In AgentStack, go to **Settings > Provider Keys**.
5. Click **Add Provider Key**.
6. Select **Google** from the provider dropdown.
7. Paste your key and click **Save**.

Models unlocked: Gemini 2.0, Gemini 1.5 Pro, and any future Google AI models.

Other supported providers

AgentStack also supports:

- **Mistral** -- Mistral Large, Medium, Small
- **Groq** -- Llama, Mixtral (fast inference)
- **Any OpenAI-compatible API** -- If your provider offers an OpenAI-compatible endpoint, you can connect it using the "Custom (OpenAI-compatible)" option.

The setup process is the same: get a key from the provider, add it in **Settings > Provider Keys**, select the provider, paste, save.

How BYOK works under the hood

- Your keys are encrypted with per-user encryption (AES-256) and stored securely.
- Keys are never logged, never shared, and never visible in execution logs.
- When an agent runs, AgentStack calls the model provider's API using your key. The token charges go directly to your provider account.
- You can see per-run cost estimates in your Execution History based on the provider's published pricing.

Switching models

Once you've connected multiple providers, you can choose which model each agent uses:

1. Go to **My Agents** and click on an agent.
2. Under **Model**, select the model you want (e.g., GPT-4o, Claude 3.5 Sonnet, Gemini 2.0).
3. Click **Save**.

You can also override the model per workflow step -- useful when you want a cheaper model for simple tasks and a more capable model for complex ones.

Troubleshooting

PROBLEM	SOLUTION
"No API key configured" error	Go to Settings > Provider Keys and add a key for the model provider your agent uses.
"Invalid API key" error	Double-check you copied the full key. Regenerate it at the provider if needed.
"Quota exceeded" error	Your provider account has hit its spending limit. Increase your limit on the provider's billing page.
Key doesn't appear after saving	Refresh the page. If it still doesn't show, try a different browser or clear your cache.

Article 3: Activating Your First Agent

Category: Getting Started
URL slug: /help/activating-first-agent
Last updated: 2026-03-10

AgentStack comes with 82 pre-built agents across 9 categories. Each agent is designed for a specific job -- code generation, design QA, sprint planning, copywriting, and more. This guide walks you through browsing the catalog, activating an agent, and running your first test.

Prerequisites

- An AgentStack account (free plan is fine).
- At least one provider API key connected. See Connecting Your API Keys.

Step 1: Browse the agent catalog

1. From your dashboard, click **Agent Catalog** in the left sidebar.
2. You'll see all 82 agents organized into 9 categories:
   - **Engineering** (18 agents) -- Backend, frontend, code review, testing, databases, and more.
   - **Design** (7 agents) -- UI/UX, wireframes, design systems, accessibility.
   - **Product** (6 agents) -- PRDs, user stories, metrics, onboarding.
   - **Infrastructure & DevOps** (7 agents) -- Terraform, CI/CD, incident response, cost optimization.
   - **QA & Security** (5 agents) -- Test execution, security audits, compliance.
   - **Growth & Marketing** (7 agents) -- Content, SEO, email, growth experiments.
   - **Research & Strategy** (5 agents) -- Market validation, competitor analysis, feedback synthesis.
   - **Coordination** (8 agents) -- Sprint planning, standups, retros, cross-team orchestration.
   - **Specialized** (6 agents) -- Stack selection, AI integration, billing, documentation.
3. Click any agent to see its full description, capabilities, average tokens per run, and estimated cost per run.

Step 2: Activate an agent

1. Click the agent you want to activate.
2. Click the **Activate** button.
3. The agent moves from the catalog to your **My Agents** list.

Free plan users: You can activate up to 5 agents. If you've hit the limit, deactivate an existing agent first, or upgrade to Pro to unlock all 82.

Recommended first agents:
- **Backend Systems Engineer** -- Generates production-grade backend code across Python, Node, Go, and Rust.

- **Code Reviewer** -- Reviews code for bugs, performance, and security issues with structured feedback.
- **Content & Copy Strategist** -- Writes UX microcopy, landing pages, and email sequences.

Step 3: Run a test

1. Go to **My Agents** and click on your newly activated agent.
2. Click **Test Run**.
3. Type a prompt in the input field. Be specific about what you want.
   - Example for Backend Systems Engineer: "Write a REST API endpoint in Python FastAPI that accepts a JSON payload with a user email and returns a JWT token."
   - Example for Code Reviewer: "Review this function for bugs and performance issues: [paste your code]."
4. Click **Run**.
5. Watch the agent process your request. You'll see the output in real time.

Step 4: Review the results

After the run completes, you'll see:

- **Output** -- The agent's full response.
- **Token usage** -- How many input and output tokens were consumed.
- **Cost** -- Estimated cost based on your model provider's pricing.
- **Latency** -- How long the execution took.

This run is also saved in your **Execution History** for later reference.

Tips for better results

- **Be specific.** "Write a function" is vague. "Write a Python function that validates email addresses using regex and returns a boolean" is actionable.
- **Provide context.** If you're working in a specific framework or codebase, mention it. Agents produce better output when they know the constraints.
- **Iterate.** Use the output from one run as context for the next. Or better yet, chain agents together in a workflow.

Next step

Ready to chain multiple agents together? See Building Your First Workflow.

Article 4: Building Your First Workflow

Category: Getting Started
URL slug: /help/building-first-workflow
Last updated: 2026-03-10

Workflows are where AgentStack goes from useful to powerful. Instead of running agents one at a time, you chain them together -- passing the output of one agent as the input to the next. This guide walks you through building your first workflow using the visual builder.

Prerequisites

- At least 2 agents activated. See Activating Your First Agent.
- At least one provider API key connected. See Connecting Your API Keys.

Step 1: Open the workflow builder

1. From your dashboard, click **Workflows** in the left sidebar.
2. Click **Create Workflow**.
3. Give your workflow a name (e.g., "Code and Review") and an optional description.
4. You'll land on the visual workflow canvas -- a blank space where you'll build your pipeline.

Step 2: Add your first step

1. Click **Add Step** on the canvas (or drag an agent from the sidebar).
2. Select an agent from the dropdown. For this example, pick **Backend Systems Engineer**.
3. Write a prompt template. Use `{{variables.feature_description}}` to reference input variables:
   `Write a REST API endpoint for the following feature: {{variables.feature_description}}`
4. Click **Save Step**.

Step 3: Add a second step

1. Click **Add Step** again.
2. Select **Code Reviewer**.

3. In the prompt template, reference the previous step's output:
   ```
   Review this code for bugs, security issues, and performance problems:

{{prev_step.output}}
```

4. Click **Save Step**.

The visual canvas now shows two connected steps: Backend Systems Engineer feeds into Code Reviewer.

Step 4: Set your trigger

Every workflow needs a trigger -- the event that starts it.

1. Click the **Trigger** panel at the top of the canvas.
2. For your first workflow, select **Manual**. (You'll run it by clicking a button.)
3. Other trigger types:
   - **Cron** -- Run on a schedule (e.g., every weekday at 9 AM).
   - **Webhook** -- Run when an external service sends an HTTP request.

See Using Workflow Triggers for details on cron and webhook setup.

Step 5: Define your input variables

1. Click the **Variables** panel.
2. Add a variable called `feature_description` with type `string`.
3. Mark it as **required**.
4. Optionally add a description: "The feature you want the agent to build."

These variables become the input form when you run the workflow manually, and the expected payload when triggered via API or webhook.

Step 6: Run your workflow

1. Click **Execute** in the top-right corner.
2. Fill in the `feature_description` variable (e.g., "User authentication with email/password and JWT tokens").
3. Click **Run**.
4. Watch each step execute in sequence. You'll see real-time status for each step: queued, running, completed, or failed.

Step 7: Review the results

After the workflow completes:

- Click any step to see its input, output, token usage, cost, and latency.
- The total workflow cost and token count are shown at the top.
- The full run is saved in your **Execution History**.

Workflow builder features

FEATURE	WHAT IT DOES
Prompt templates	Use `{{variables.name}}` for input variables, `{{prev_step.output}}` for the previous step's output, or `{{steps.step_name.output}}` to reference any specific step.
Model override	Override the default model on any step. Use a cheaper model for simple tasks, a powerful one for complex tasks.
Timeout	Set a maximum execution time per step (10-600 seconds). Default: 120 seconds.
Retry policy	Configure automatic retries on failure (1-5 attempts with optional backoff).
Conditional logic	Add branches that execute different steps based on previous outputs (Pro plan).
Parallel branches	Run multiple steps simultaneously when they don't depend on each other (Pro plan).

Free plan limits

- Up to 3 workflows.
- Up to 5 steps per workflow.
- 100 executions per month (each workflow run counts as one execution).
- Upgrade to Pro for unlimited workflows, unlimited executions, and advanced logic.

Next steps

- Learn about all trigger types: Using Workflow Triggers
- Understand execution logs: Execution Logs and Monitoring
- Explore agent categories: Understanding Agent Categories

CORE FEATURES

Article 5: Understanding Agent Categories

Category: Core Features
URL slug: /help/agent-categories
Last updated: 2026-03-10

AgentStack's 82 agents are organized into 9 categories. Each category covers a different function of the product development lifecycle. This guide helps you understand what each category offers and when to use which agents.

The 9 categories

1. Engineering (18 agents)

From first commit to production deploy.

Covers the full engineering stack: backend code generation, frontend components, code review, debugging, testing, database migrations, data pipelines, mobile and desktop development, API specs, prompt engineering, localization, contract testing, analytics instrumentation, and data architecture.

When to use: Any time you need code written, reviewed, tested, refactored, or documented.

Key agents:
- Backend Systems Engineer -- Production-grade backend code in Python, Node, Go, Rust.
- Code Reviewer -- PR reviews with structured, actionable feedback.
- Test Writer -- Unit, integration, and e2e tests from code or specs.
- Database Migration Engineer -- Schema migrations with zero-downtime strategies.
- Prompt Engineer -- Design and optimize prompts for LLM-powered features.

2. Design (7 agents)

From wireframe to pixel-perfect handoff.

Covers UI/UX design, brand identity, wireframes and prototyping, design systems, design QA, motion design, and accessibility auditing.

When to use: When you're designing interfaces, building design systems, checking implemented UI against specs, or auditing for accessibility.

Key agents:
- UI/UX Design Lead -- Full-fidelity design specs with responsive breakpoints.
- Design QA Specialist -- Validates implemented UI against Figma specs.
- Accessibility QA Specialist -- WCAG 2.1/2.2 AA compliance audits.

3. Product (6 agents)

From idea to shipped feature.

Covers product management, requirements analysis, user stories, metrics architecture, onboarding design, and UX research.

When to use: When you're defining what to build, writing requirements, planning onboarding flows, or measuring feature impact.

Key agents:
- AI Product Studio PM -- PRDs, success metrics, backlog prioritization.
- User Story Writer -- User stories with acceptance criteria and test scenarios.
- Product Metrics Architect -- KPI frameworks and dashboard specs.

4. Infrastructure & DevOps (7 agents)

From Terraform plan to incident resolved.

Covers infrastructure-as-code, cloud architecture, CI/CD pipelines, incident response, FinOps cost optimization, MLOps, and performance engineering.

When to use: When you're provisioning infrastructure, setting up deployment pipelines, responding to incidents, or optimizing cloud costs.

Key agents:
- Terraform IaC Engineer -- Production-ready Terraform, Pulumi, and CDK configs.
- Incident Response Commander -- Full incident lifecycle from SEV classification to postmortem.
- FinOps Cost Optimization -- Cloud spend analysis and rightsizing recommendations.

5. QA & Security (5 agents)

From test plan to security audit.

Covers quality gates, feature testing, application security audits, cross-agent output review, and legal compliance.

When to use: When you need to verify quality, run security scans, check regulatory compliance, or review another agent's output.

Key agents:
- QA & Security Gatekeeper -- Quality gates across the pipeline.
- AppSec Audit Engineer -- OWASP Top 10 review and dependency scanning.
- Senior Cross-Agent Reviewer -- Final check on other agents' output before human handoff.

6. Growth & Marketing (7 agents)

From content strategy to conversion optimization.

Covers copywriting, SEO, social media, email marketing, growth experiments, go-to-market analysis, and pricing strategy.

When to use: When you're writing marketing copy, planning growth experiments, building email sequences, or evaluating pricing models.

Key agents:
- Content & Copy Strategist -- Landing pages, microcopy, email sequences.
- SEO Strategist -- Keyword research, meta tags, content optimization.
- Growth Experiments Strategist -- A/B tests and activation experiments.

7. Research & Strategy (5 agents)

From market signal to strategic insight.

Covers market validation, competitor intelligence, opportunity discovery, Reddit research, and feedback synthesis.

When to use: When you're validating a product idea, analyzing competitors, or synthesizing user feedback into actionable insights.

Key agents:
- Market Validation Analyst -- TAM/SAM/SOM analysis and viability scoring.
- Competitor Intelligence Analyst -- Competitive landscape reports.
- Feedback Intelligence Synthesizer -- Themed, prioritized insight reports from user feedback.

8. Coordination (8 agents)

From kickoff to retro.

Covers sprint planning, standup summaries, retrospective facilitation, cross-team orchestration, release planning, dependency tracking, and velocity reporting.

When to use: When you're running sprints, coordinating across teams, planning releases, or tracking project velocity.

Key agents:
- Sprint Planning Agent -- PRD-to-ticket creation, sprint execution, velocity tracking.
- Technical Program Manager -- Cross-team dependency management and milestone tracking.

9. Specialized (6 agents)

Experts for everything else.

Covers stack selection, AI integration, technical documentation, billing implementation, spreadsheet automation, and app store publishing.

When to use: When you need expert help with a specific domain that doesn't fit neatly into the other categories.

Key agents:
- Technical Documentation Writer -- API references, ADRs, runbooks, changelogs.
- AI Integration Specialist -- RAG pipelines, embedding strategies, vector databases.
- Billing & Subscription Manager -- Stripe/Paddle integrations and revenue analytics.

How to choose the right agent

1. **Start with the job, not the category.** What do you need done? "Review this PR" -> Code Reviewer. "Write a PRD" -> AI Product Studio PM.
2. **Check the agent's description.** Click any agent in the catalog to see exactly what it does, what it outputs, and its average cost per run.
3. **Chain them.** Most real work involves multiple agents. Write code with the Backend Systems Engineer, review it with the Code Reviewer, document it with the Technical Documentation Writer. That's a workflow.

Article 6: How the Visual Workflow Builder Works

Category: Core Features
URL slug: /help/visual-workflow-builder
Last updated: 2026-03-10

The visual workflow builder is where you chain agents together into automated pipelines. Instead of running agents one at a time and copy-pasting output between them, you wire them up once and let them run in sequence (or in parallel).

This article covers every feature of the workflow builder.

The canvas

The workflow builder is a visual canvas where each agent is a step, and connections between steps define the data flow. Think of it like a flowchart where each box is an AI agent doing a specific job.

- **Steps** are arranged left-to-right (or top-to-bottom).
- **Connections** show how data flows from one step to the next.
- **The trigger** sits at the start -- it defines what kicks off the workflow.

Adding steps

1. Click **Add Step** on the canvas or drag an agent from the agent sidebar.
2. Select an agent from the dropdown list (only activated agents appear).
3. Configure the step:
   - **Name** -- A short label for this step (used in variable references). e.g., "plan", "implement", "review".
   - **Prompt template** -- The instruction sent to the agent. Supports variable interpolation.
   - **Model override** (optional) -- Use a different model than the agent's default.
   - **Timeout** -- Max execution time (10-600 seconds, default 120).
   - **Retry policy** -- Auto-retry on failure (1-5 attempts, optional backoff delay).

Prompt templates and variables

Prompt templates are how you pass data between steps. Three variable types are available:

SYNTAX	WHAT IT REFERENCES
`{{variables.name}}`	An input variable defined in the workflow's variable schema. Filled in at execution time.
`{{prev_step.output}}`	The output of the immediately preceding step.
`{{steps.step_name.output}}`	The output of any specific step by its name. Use this when you need to reference a step that isn't the immediate predecessor.

Example prompt template:

```
Review this code for bugs and security issues:

{{steps.implement.output}}

Pay special attention to the requirements:
{{variables.feature_description}}
```

Connections

Connections define the execution order. By default, steps run in sequence -- each step waits for the previous one to complete before starting.

To create a connection, drag from the output port of one step to the input port of the next. The canvas auto-arranges to keep things readable.

Triggers

Every workflow needs exactly one trigger. Three types are available:

TRIGGER	HOW IT WORKS
Manual	You click "Execute" and fill in the input variables. Good for testing and ad-hoc runs.
Cron	Runs on a schedule using standard 5-field cron syntax. e.g., `0 9 * * 1-5` = weekdays at 9 AM UTC.
Webhook	Runs when an external service sends an HTTP POST to the workflow's webhook URL. The request body is passed as input variables.

See Using Workflow Triggers for detailed setup instructions.

Input variables

Workflow variables define the input contract -- what data the workflow expects when it runs.

1. Click the **Variables** panel.
2. Add variables with a name, type (string, number, boolean, object), and optional default value.
3. Mark variables as required or optional.
4. Add descriptions to help collaborators understand what each variable expects.

Variables are passed via the execution form (manual trigger), the API request body, or the webhook payload.

Advanced features (Pro plan)

FEATURE	WHAT IT DOES
Conditional branches	Route execution to different steps based on the output of a previous step. e.g., "If the code review found critical issues, route to a fix step. Otherwise, proceed to documentation."

FEATURE	WHAT IT DOES
Parallel branches	Run multiple steps simultaneously when they don't depend on each other. e.g., run documentation and testing in parallel after the code is written.
Human-in-the-loop gates	Pause the workflow and wait for human approval before continuing. Useful for review checkpoints.
Loops	Repeat a step or sequence of steps based on a condition. e.g., "Keep revising until the reviewer approves."

Saving and versioning

- Workflows auto-save as drafts.
- Click **Save** to create a named version.
- You can view previous versions and revert if needed.

Limits by plan

FEATURE	FREE	PRO	TEAM
Workflows	3	Unlimited	Unlimited
Steps per workflow	5	Unlimited	Unlimited
Executions per month	100	Unlimited	Unlimited
Conditional logic	--	Yes	Yes
Parallel branches	--	Yes	Yes
Human-in-the-loop	--	Yes	Yes

Article 7: Execution Logs and Monitoring

Category: Core Features
URL slug: /help/execution-logs
Last updated: 2026-03-10

Every time an agent or workflow runs on AgentStack, it's logged. Execution logs give you full visibility into what happened, what it cost, and what went wrong (if anything). This article covers how to read and use them.

Accessing execution history

1. Click **Executions** in the left sidebar.
2. You'll see a chronological list of all runs -- both individual agent test runs and full workflow executions.
3. Each entry shows:
   - Workflow or agent name
   - Status (completed, failed, running, timed out)
   - Start time and duration
   - Total tokens used
   - Total estimated cost

Filtering and searching

Use the filters at the top of the execution list:

FILTER	OPTIONS
Status	Completed, Failed, Running, Timed Out
Type	Agent (single runs), Workflow (multi-step runs)
Date range	Last 24 hours, Last 7 days, Last 30 days, Custom range
Agent/Workflow	Filter by a specific agent or workflow name

Per-step logs

For workflow executions, click into a run to see the step-by-step breakdown:

1. Click any execution in the list.
2. You'll see the workflow diagram with each step colored by status:
   - **Green** -- Completed successfully.
   - **Red** -- Failed.
   - **Yellow** -- Timed out.
   - **Gray** -- Skipped (conditional branch not taken).
   - **Blue** -- Currently running.
3. Click any step to see its details:
   - **Input** -- The exact prompt sent to the agent (with all variables resolved).

- **Output** -- The agent's full response.
- **Model** -- Which model was used.
- **Tokens** -- Input tokens, output tokens, and total.
- **Cost** -- Estimated cost for this step.
- **Latency** -- Execution time in milliseconds.
- **Retries** -- How many retry attempts occurred (if any).

Token usage tracking

Token usage is tracked at three levels:

1. **Per step** -- How many tokens each individual agent used.
2. **Per execution** -- Total tokens across all steps in a workflow run.
3. **Per period** -- Aggregate usage over time (visible in the Usage dashboard).

Token counts include both input tokens (your prompt) and output tokens (the agent's response). Cost is estimated based on your model provider's published per-token pricing.

Cost tracking

Every execution shows an estimated cost in USD. This is calculated from:

- The model used (e.g., GPT-4o, Claude 3.5 Sonnet)
- The number of input and output tokens
- The provider's published per-token pricing

This is an estimate. Your actual bill comes from your model provider, not AgentStack. The estimate helps you understand relative costs and optimize which models you use where.

Debugging failed executions

When a step fails, the execution log shows:

1. **Error type** -- What kind of failure occurred (timeout, rate limit, invalid response, provider error).
2. **Error message** -- A human-readable explanation of what went wrong.
3. **Request ID** -- A unique identifier you can include in support tickets.
4. **The exact input** -- See the resolved prompt that was sent to the model, so you can reproduce the issue.

Common failure patterns and fixes:

FAILURE	CAUSE	FIX
Timeout	Agent took longer than the configured timeout.	Increase the timeout in the step config (up to 600 seconds).
Rate limit	Too many requests to your model provider.	Space out executions or increase your provider's rate limit.
Provider quota exceeded	Your provider account hit its spending limit.	Increase your spending cap on the provider's billing page.
Invalid API key	Your provider key was revoked or expired.	Update the key in Settings > Provider Keys.
Model not available	The configured model isn't accessible with your provider key.	Switch to a different model in the step config.

Usage dashboard

For a higher-level view, go to **Usage** in the left sidebar. The usage dashboard shows:

- Executions over time (daily, weekly, monthly)
- Token usage over time
- Cost breakdown by agent, workflow, and model
- Execution count vs. plan limit (free plan users)

Article 8: Using Workflow Triggers (Manual, Cron, Webhook)

Category: Core Features
URL slug: /help/workflow-triggers
Last updated: 2026-03-10

Every workflow needs a trigger -- the event that starts it. AgentStack supports three trigger types: Manual, Cron, and Webhook. This article explains how each one works and how to set it up.

Manual triggers

The simplest trigger. You click a button, fill in the input variables, and the workflow runs.

How to set up:
1. Open your workflow in the builder.
2. Click the **Trigger** panel.
3. Select **Manual**.
4. Save the workflow.

How to use:
1. Go to **Workflows** and click on your workflow.
2. Click **Execute**.
3. Fill in any required input variables.
4. Click **Run**.

Best for: Testing, ad-hoc tasks, and workflows you run on demand.

Cron triggers

Run a workflow on a recurring schedule using standard cron syntax.

How to set up:
1. Open your workflow in the builder.
2. Click the **Trigger** panel.
3. Select **Cron**.
4. Enter a cron expression in the 5-field format: `minute hour day-of-month month day-of-week`.
5. Save the workflow.

Common cron expressions:

EXPRESSION	SCHEDULE
`0 9 * * *`	Every day at 9:00 AM UTC
`0 9 * * 1-5`	Weekdays at 9:00 AM UTC
`0 */6 * * *`	Every 6 hours
`0 0 1 * *`	First day of every month at midnight UTC
`*/30 * * * *`	Every 30 minutes

Important: All cron schedules run in UTC. Adjust your expression accordingly if you're in a different timezone.

Best for: Daily reports, scheduled code reviews, recurring audits, weekly summaries.

Webhook triggers

Run a workflow when an external service sends an HTTP POST request to a unique URL.

How to set up:
1. Open your workflow in the builder.
2. Click the **Trigger** panel.
3. Select **Webhook**.
4. AgentStack generates a unique webhook URL and a webhook secret.
5. Copy both values.
6. Save the workflow.

Your webhook URL will look like:

```
https://api.agentstack.ai/v1/webhooks/wh_01HXYZ123ABC
```

How to trigger it:

Send a POST request with a JSON body that matches your workflow's variable schema:

```
curl -X POST https://api.agentstack.ai/v1/webhooks/wh_01HXYZ123ABC \
  -H "Content-Type: application/json" \
  -H "X-Webhook-Secret: your_webhook_secret" \
  -d '{
    "feature_description": "Add user authentication with JWT",
    "language": "TypeScript"
  }'
```

Securing your webhook:

Every webhook trigger comes with a secret key. Include it in the `X-Webhook-Secret` header of your requests. AgentStack validates this header and rejects requests with an invalid or missing secret.

Retry behavior:

If your webhook request is accepted (200 response), the workflow queues for execution. If the workflow itself fails during execution, the retry policy on individual steps applies -- the webhook is not retried.

Best for: CI/CD integration (trigger a review workflow on every PR), GitHub Actions, Slack commands, and any external event-driven automation.

Changing a trigger

You can change a workflow's trigger type at any time:

1. Open the workflow in the builder.
2. Click the **Trigger** panel.
3. Select a different trigger type.
4. Configure the new trigger.
5. Save.

Changing the trigger does not affect existing execution history.

Article 9: Creating Custom Agents

Category: Core Features
URL slug: /help/custom-agents
Last updated: 2026-03-10

AgentStack's 82 built-in agents cover a wide range of tasks, but sometimes you need an agent tailored to your specific use case. This article explains how to customize existing agents and when to do so.

Customizing an existing agent (Pro plan)

On the Pro plan, you can customize any of the 82 built-in agents by adjusting their configuration:

1. Go to **My Agents** and click on an activated agent.
2. Click **Customize**.
3. You can modify:
   - **System prompt** -- The base instructions that define the agent's behavior. Edit to add your company's coding standards, style guide, or domain-specific context.
   - **Model** -- Switch the default model (e.g., from GPT-4o to Claude 3.5 Sonnet).
   - **Tools** -- Enable or disable tool access (e.g., web search, code execution, file reading).
   - **Output format** -- Specify whether the agent should output markdown, JSON, plain text, or a custom schema.
   - **Temperature** -- Adjust creativity vs. consistency (0.0 = deterministic, 1.0 = creative).
4. Click **Save**.

Customized agents appear in your My Agents list with a "Customized" badge. You can revert to the original configuration at any time.

When to customize vs. use built-in

SITUATION	RECOMMENDATION
The built-in agent does 90% of what you need	Customize it. Tweak the system prompt to cover the remaining 10%.
You need an agent for a domain not covered by any of the 82	Wait for custom agent creation (coming soon on Team plan).
You want the same agent but with a different model	Customize. Switch the model in the agent config.
You want stricter or looser output formatting	Customize. Adjust the output format and temperature.
You need company-specific context in every prompt	Customize. Add your context to the system prompt -- it'll be included in every run.

Custom prompt tips

- **Be explicit about what you want.** "Follow our Python style guide" is vague. "Use Black formatting, type hints on all function signatures, and Google-style docstrings" is specific.
- **Include examples.** If you want a specific output format, show the agent an example of what good output looks like.
- **Keep it focused.** Don't try to make one agent do everything. Customize multiple agents for different jobs and chain them in a workflow.
- **Test after customizing.** Run a test with your usual inputs to make sure the customization produces better results than the default.

Fully custom agents (coming soon)

The Team plan will support building agents from scratch -- define your own system prompt, tool set, and capabilities without starting from a built-in template. This is on the roadmap and will be announced when available.

In the meantime, the 82 built-in agents plus customization covers the vast majority of use cases.

Using the Prompt Engineer agent

If you want help writing better system prompts for your customized agents, activate the **Prompt Engineer** agent. It can:

- Optimize existing prompts for clarity and consistency.
- Design evaluation harnesses to test prompt variations.
- Suggest prompt structures for specific output formats.

This is a meta-agent -- an agent that helps you configure other agents.

ACCOUNT & BILLING

Article 10: Understanding Plans and Pricing

Category: Account & Billing
URL slug: /help/plans-and-pricing
Last updated: 2026-03-10

AgentStack offers three plans. Every plan uses BYOK (Bring Your Own Key) pricing, so you never pay AgentStack for tokens -- only for the platform.

Plan comparison

FEATURE	FREE	PRO ($49/MO)	TEAM ($149/MO)
Agents	5	All 82	All 82
Workflow executions	100/month	Unlimited	Unlimited
Workflows	3	Unlimited	Unlimited
API access	No	Yes	Yes
Custom agents	No	Yes	Yes

FEATURE	FREE	PRO ($49/MO)	TEAM ($149/MO)
Team seats	1 (solo)	5	Unlimited
Rate limit (API)	60 req/min	300 req/min	1,000 req/min
White-label	No	No	Yes
SSO	No	No	Yes
Support	Standard	Priority	Dedicated

What BYOK means for your costs

BYOK stands for Bring Your Own Key. You connect API keys from model providers -- OpenAI, Anthropic, Google -- and pay those providers directly at their published rates.

AgentStack never touches your token costs. Your $49/mo Pro plan is $49/mo whether you run 100 tokens or 10 million.

This is different from most AI platforms, which charge a per-token markup (often 2x-5x the provider's rate). With BYOK, your AI costs stay transparent and predictable.

What counts as an execution

One execution = one complete workflow run. If your workflow has 5 agent steps, that's still 1 execution. Individual agent test runs from the Agent Catalog also count as 1 execution each.

The count increments when the workflow starts, not when it finishes.

Annual billing

Pro and Team plans are available with annual billing at a 20% discount:

PLAN	MONTHLY	ANNUAL (PER MONTH)	ANNUAL (TOTAL)
Pro	$49/mo	$39.20/mo	$470.40/year
Team	$149/mo	$119.20/mo	$1,430.40/year

You can switch between monthly and annual billing at any time from **Settings > Billing**.

Article 11: Upgrading Your Plan

Category: Account & Billing
URL slug: /help/upgrading-plan
Last updated: 2026-03-10

Upgrading unlocks more agents, removes execution limits, and gives you API access. Here's how to do it and what happens immediately after.

How to upgrade

1. Go to **Settings > Billing**.
2. Click **Upgrade** next to the plan you want (Pro or Team).
3. Choose your billing cycle: **Monthly** or **Annual** (20% discount).
4. You'll be redirected to a Stripe Checkout page.
5. Enter your payment details and confirm.
6. You're upgraded immediately -- no waiting, no restart required.

What happens after you upgrade

Changes take effect the moment your payment is confirmed:

CHANGE	WHAT YOU'LL SEE
Agent access	All 82 agents become available in the Agent Catalog.
Execution limits	Removed. Your monthly counter resets to unlimited.
Workflow limits	Removed. Create as many workflows as you need.
API access	The API Keys section appears in Settings (Pro and Team).
Custom agents	The Customize button appears on all agent detail pages.
Team seats	The Team section appears in Settings (Pro: 5 seats, Team: unlimited).

Any workflows that were paused because you hit execution limits will not auto-resume. You'll need to re-run them manually.

Prorated billing

If you upgrade mid-cycle, you only pay for the remaining days in the current billing period. For example, if you upgrade to Pro on day 15 of a 30-day month, your first charge is approximately $24.50 (half the monthly rate). Your next full charge happens on day 1 of the next cycle.

Billing cycle

Your billing cycle starts on the day you upgrade. If you upgrade on March 15, your next charge is April 15 (monthly) or March 15 of next year (annual). All dates are visible in **Settings > Billing**.

Article 12: Managing Your Subscription

Category: Account & Billing
URL slug: /help/managing-subscription
Last updated: 2026-03-10

All billing management happens through the Stripe billing portal, accessible directly from your AgentStack dashboard.

Accessing the billing portal

1. Go to **Settings > Billing**.
2. Click **Manage Subscription**.
3. You'll be redirected to the Stripe billing portal, which opens in a new tab.

From the portal, you can manage everything billing-related without contacting support.

What you can do in the billing portal

Change your plan:
- Upgrade from Pro to Team, or downgrade from Team to Pro.
- Switch between monthly and annual billing.
- Changes take effect at the start of your next billing cycle (for downgrades) or immediately (for upgrades).

View invoices and payment history:
- See every invoice since you started your subscription.
- Download PDF invoices for accounting.
- Check the status of each payment (paid, pending, failed).

Update your payment method:
- Add a new credit card or replace an existing one.
- Update your billing address.
- AgentStack supports all major credit cards via Stripe.

Update billing email:
- Change where invoice receipts are sent.
- This is separate from your AgentStack account email.

Failed payments

If a payment fails, Stripe retries automatically over a 7-day period. You'll receive email notifications at each retry attempt. If all retries fail, your plan downgrades to Free at the end of the retry period. Your data and configurations are preserved -- upgrade again to restore access.

Article 13: Cancelling Your Subscription

Category: Account & Billing
URL slug: /help/cancelling-subscription
Last updated: 2026-03-10

You can cancel your AgentStack subscription at any time. No lock-in, no cancellation fees.

How to cancel

1. Go to **Settings > Billing**.
2. Click **Manage Subscription**.
3. In the Stripe billing portal, click **Cancel plan**.
4. Confirm cancellation.

That's it. You'll receive an email confirming the cancellation.

What happens after you cancel

You keep access until the end of your current billing period. If you cancel on March 10 and your billing cycle ends March 25, you have full Pro (or Team) access until March 25.

After your billing period ends:

WHAT CHANGES	DETAILS
Plan	Reverts to Free.
Agents	Only your first 5 activated agents remain accessible. Others are deactivated but not deleted.
Workflows	Only your first 3 workflows remain active. Others are paused but not deleted.
Executions	Limited to 100/month.
API access	Disabled. Existing API keys stop working.
Team members	Removed from your workspace. Only the owner retains access.

Data retention

Your workspace is preserved for **90 days** after cancellation. This includes all agent configurations, workflows, execution history, and custom prompts. After 90 days, your workspace and all associated data are permanently deleted.

Reactivating your subscription

Within the 90-day retention window:

1. Log in to your AgentStack account.
2. Go to **Settings > Billing**.
3. Click **Reactivate** and choose your plan.
4. Everything restores exactly as you left it -- agents, workflows, history, all of it.

After 90 days, you'll need to start fresh with a new workspace.

Refund policy

AgentStack does not offer partial refunds for unused time. When you cancel, you retain access for the remainder of your paid billing period. This is why we don't lock you into annual contracts -- monthly billing means you're never more than 30 days from a clean exit.

If you believe you were charged in error, email **support@agentstack.ai** and we'll investigate.

API & INTEGRATIONS

Article 14: Managing API Keys

Category: API & Integrations
URL slug: /help/managing-api-keys
Last updated: 2026-03-10

API keys let you interact with AgentStack programmatically. You can trigger workflows, list agents, read execution results, and more -- all from your own code or CI/CD pipelines. API access is available on Pro and Team plans.

Creating an API key

1. Go to **Settings > API Keys**.
2. Click **Create API Key**.
3. Enter a descriptive name (e.g., "CI/CD Pipeline" or "Production Backend").
4. Select the scopes this key needs:

SCOPE	WHAT IT ALLOWS
`agents:read`	List agent templates, view agent configurations.
`agents:write`	Activate, deactivate, and customize agents.
`workflows:read`	List workflows, view workflow configurations.
`workflows:write`	Create, update, delete, and execute workflows.
`executions:read`	List executions, view execution details and logs.

1. Click **Create**.

2. **Copy your key immediately.** It is displayed only once. If you lose it, you'll need to create a new one.

Revoking an API key

If a key is compromised or no longer needed:

1. Go to **Settings > API Keys**.
2. Find the key in the list.
3. Click **Revoke**.
4. Confirm.

Revocation is immediate. Any request using that key will return a `401 Unauthorized` error.

Security best practices

- **Store keys in environment variables.** Never hardcode API keys in source code or commit them to version control.
- **Use minimum scopes.** If a key only needs to trigger workflows, give it `workflows:write` and nothing else.
- **Rotate keys regularly.** Create a new key, update your integration, then revoke the old one. Aim for every 90 days.
- **One key per integration.** Don't share a single key across multiple services. If one integration is compromised, you can revoke its key without disrupting others.
- **Monitor usage.** Check **Dashboard > Usage** to spot unexpected spikes that might indicate a leaked key.

API key limits

You can create up to 10 API keys per workspace on Pro, and 25 on Team. If you need more, contact **support@agentstack.ai**.

Article 15: Using the AgentStack API

Category: API & Integrations
URL slug: /help/using-api
Last updated: 2026-03-10

The AgentStack API lets you do everything the dashboard does -- programmatically. Trigger workflows, list agents, read execution results, and manage your workspace from your own code.

Base URL

```
https://api.agentstack.ai/v1
```

All endpoints are under this base URL.

Authentication

Every request requires a Bearer token in the `Authorization` header. Use an API key from **Settings > API Keys** (see Managing API Keys).

```
Authorization: Bearer your_api_key_here
```

Your first API call: list agent templates

This returns all available agent templates in the catalog:

```
curl https://api.agentstack.ai/v1/agents/templates \
  -H "Authorization: Bearer your_api_key_here"
```

Response (abbreviated):

```
{
  "data": [
    {
      "id": "agt_backend_engineer",
      "name": "Backend Systems Engineer",
      "category": "Engineering",
      "description": "Designs and implements backend services, APIs, and data models."
    },
    {
      "id": "agt_code_reviewer",
      "name": "Code Reviewer",
      "category": "Engineering",
      "description": "Reviews code for bugs, security issues, and style violations."
    }
  ],
  "total": 82,
  "page": 1,
  "per_page": 20
}
```

Triggering a workflow via API

To execute a workflow, send a POST request with the workflow ID and input variables:

```
curl -X POST https://api.agentstack.ai/v1/workflows/wf_abc123/execute \
  -H "Authorization: Bearer your_api_key_here" \
  -H "Content-Type: application/json" \
  -d '{
    "variables": {
      "feature_description": "Add password reset flow",
      "language": "TypeScript"
    }
  }'
```

Response:

```
{
  "execution_id": "exec_xyz789",
  "status": "queued",
  "workflow_id": "wf_abc123",
  "created_at": "2026-03-10T12:00:00Z"
}
```

The workflow runs asynchronously. Poll the execution status or set up a webhook to get notified when it completes:

```
curl https://api.agentstack.ai/v1/executions/exec_xyz789 \
  -H "Authorization: Bearer your_api_key_here"
```

Rate limits

Rate limits depend on your plan:

PLAN	RATE LIMIT
Free	60 requests/minute
Pro	300 requests/minute
Team	1,000 requests/minute

If you exceed the limit, the API returns a `429 Too Many Requests` response with a `Retry-After` header indicating how many seconds to wait.

Error handling

All errors follow a consistent format:

```
{
  "error": {
    "code": "rate_limit_exceeded",
    "message": "You've exceeded 300 requests per minute. Retry after 12 seconds.",
    "status": 429
  }
}
```

Common error codes:

STATUS	CODE	MEANING
400	`invalid_request`	Malformed request body or missing required fields.
401	`unauthorized`	Invalid or missing API key.
403	`forbidden`	Your API key doesn't have the required scope.
404	`not_found`	The resource (workflow, agent, execution) doesn't exist.
429	`rate_limit_exceeded`	Too many requests. Check the `Retry-After` header.

STATUS	CODE	MEANING
500	`internal_error`	Something went wrong on our end. Retry after a few seconds.

Full API reference

For complete endpoint documentation with request/response schemas, see the AgentStack API Reference.

Article 16: Webhook Integration Guide

Category: API & Integrations
URL slug: /help/webhook-integration
Last updated: 2026-03-10

Webhooks let external services trigger your AgentStack workflows automatically. When a specific event happens -- a PR is opened, a form is submitted, a deploy completes -- the external service sends a request to AgentStack, and your workflow runs.

Setting up a webhook trigger

1. Open a workflow in the **Workflow Builder**.
2. Click the **Trigger** panel on the left.
3. Select **Webhook** as the trigger type.
4. AgentStack generates two values:
   - **Webhook URL** -- The endpoint external services will call.
   - **Webhook Secret** -- A signing key for verifying requests.
5. Copy both values. You'll need them in the external service's configuration.
6. Click **Save**.

Webhook URL format

Your webhook URL follows this pattern:

```
https://api.agentstack.ai/v1/webhooks/wh_{unique_id}
```

You can find your webhook URL at any time by opening the workflow and clicking the **Trigger** panel.

Sending a request to your webhook

Send an HTTP POST request with a JSON body. The body should match the input variables defined in your workflow:

```
curl -X POST https://api.agentstack.ai/v1/webhooks/wh_01HXYZ123ABC \
  -H "Content-Type: application/json" \
  -H "X-Webhook-Secret: your_webhook_secret" \
  -d '{
    "repo": "acme/backend",
    "pr_number": 142,
    "action": "opened"
  }'
```

Response (success):

```
{
  "execution_id": "exec_abc123",
  "status": "queued"
}
```

The workflow is queued and begins executing. You'll receive the execution ID to track progress.

Payload format

AgentStack expects a JSON object in the request body. The keys in your JSON should match the variable names in your workflow. Any extra keys are ignored. Missing required variables cause a `400 Bad Request` error.

Example payload for a code review workflow:

```
{
  "repo": "acme/backend",
  "pr_number": 142,
  "branch": "feature/auth",
  "files_changed": ["src/auth.ts", "src/middleware.ts"]
}
```

Inside your workflow, these values are accessible as `{{repo}}`, `{{pr_number}}`, `{{branch}}`, and `{{files_changed}}`.

Securing your webhooks

Every webhook trigger includes a **webhook secret**. Use it to verify that incoming requests are genuinely from your authorized source, not from a third party.

How it works:

1. Include the secret in the `X-Webhook-Secret` header of every request.
2. AgentStack validates the header against the stored secret.
3. Requests with a missing or invalid secret are rejected with `401 Unauthorized`.

HMAC signature verification (advanced):

For additional security, AgentStack also signs every webhook response with an HMAC-SHA256 signature. If you're building a two-way integration, you can verify these signatures:

Node.js example:

```
const crypto = require('crypto');

function verifySignature(payload, signature, secret) {
  const expected = crypto
    .createHmac('sha256', secret)
    .update(JSON.stringify(payload))
    .digest('hex');
  return crypto.timingSafeEqual(
    Buffer.from(signature),
    Buffer.from(expected)
  );
}
```

Python example:

```
import hmac
import hashlib
import json

def verify_signature(payload: dict, signature: str, secret: str) -> bool:
    expected = hmac.new(
        secret.encode(),
        json.dumps(payload).encode(),
        hashlib.sha256
    ).hexdigest()
    return hmac.compare_digest(signature, expected)
```

Retry behavior

If AgentStack can't process your webhook request (e.g., internal error), it retries automatically:

ATTEMPT	DELAY
1st retry	10 seconds
2nd retry	60 seconds
3rd retry	5 minutes

After 3 failed retries, the webhook is marked as failed. You can view failed webhook deliveries in **Executions** with the status `webhook_failed`.

Note: Retries only apply to delivery failures (5xx errors on AgentStack's side). If your request is malformed (4xx), it is not retried.

Testing webhooks locally

During development, you may want to trigger webhooks from a local service. Use a tunneling tool like ngrok to expose your local server:

1. Install ngrok and run `ngrok http 3000`.
2. Copy the generated HTTPS URL (e.g., `https://abc123.ngrok.io`).
3. Configure your local service to send webhook requests to your AgentStack webhook URL -- not to ngrok. Ngrok is for the reverse direction: if you want AgentStack to call your local server (e.g., for a callback), point the callback URL to your ngrok address.

For testing the webhook trigger itself, use curl or a tool like Postman to send POST requests directly to your AgentStack webhook URL.

TEAM & SECURITY

Article 17: Inviting Team Members

Category: Team & Security
URL slug: /help/inviting-team-members
Last updated: 2026-03-10

AgentStack supports team collaboration with role-based access. Add team members to your workspace so they can run workflows, manage agents, or view execution results -- without sharing your personal credentials.

Team seat limits

PLAN	SEATS
Free	1 (solo -- no team features)
Pro	5
Team	Unlimited

The workspace owner always counts as 1 seat.

Roles and permissions

ROLE	WHAT THEY CAN DO
Owner	Full access. Manage billing, delete workspace, manage all members. One per workspace.
Admin	Manage agents, workflows, and team members. Cannot access billing or delete the workspace.
Member	Run workflows, view execution results, activate agents. Cannot manage team members or delete workflows created by others.

ROLE	WHAT THEY CAN DO
Viewer	Read-only access. Can view agents, workflows, and execution results. Cannot run, create, or modify anything.

How to invite a team member

1. Go to **Settings > Team**.
2. Click **Invite Member**.
3. Enter their email address.
4. Select a role: **Admin**, **Member**, or **Viewer**.
5. Click **Send Invite**.

The invitee receives an email with a link to join your workspace. If they don't have an AgentStack account, they'll create one during the sign-up flow. The invite expires after 7 days.

Managing existing members

Change a member's role:
1. Go to **Settings > Team**.
2. Find the member in the list.
3. Click the role dropdown next to their name.
4. Select the new role.
5. Changes take effect immediately.

Remove a member:
1. Go to **Settings > Team**.
2. Find the member.
3. Click **Remove**.
4. Confirm.

Removed members lose access to the workspace immediately. Any workflows or agents they created remain in the workspace -- nothing is deleted.

Pending invites

Pending invites appear in **Settings > Team** under a "Pending" section. You can:
- **Resend** an invite if the original email was missed.
- **Revoke** an invite if you change your mind.

Pending invites do not count against your seat limit.

Transferring ownership

Ownership transfer is handled by support. Email **support@agentstack.ai** from the current owner's email address with the new owner's email. We verify both parties before making the change.

Article 18: Security and Data Privacy

Category: Team & Security
URL slug: /help/security-data-privacy
Last updated: 2026-03-10

AgentStack is built for teams that take security seriously. Here's how we protect your data, your keys, and your workspace.

Encryption

LAYER	STANDARD
Data at rest	AES-256 encryption. All stored data -- workspace configurations, execution logs, agent customizations -- is encrypted on disk.
Data in transit	TLS 1.3. All communication between your browser and AgentStack servers is encrypted. API calls, webhook payloads, dashboard interactions -- everything travels over HTTPS.

BYOK key security

Your model provider API keys (OpenAI, Anthropic, Google) are handled with extra care:

- **Encrypted at rest** using AES-256, stored separately from other workspace data.
- **Never logged.** Your keys never appear in execution logs, error messages, or monitoring systems.
- **Never sent to AgentStack servers for processing.** When an agent runs, your key is decrypted in memory, used to call the model provider directly, and immediately discarded from memory. AgentStack acts as a pass-through, not a proxy.
- **Never shared** between workspaces. Each workspace's keys are isolated.

Data isolation

Every workspace is fully isolated:

- Workspace data is logically separated at the database level.
- One workspace cannot access another workspace's agents, workflows, executions, or configuration.
- Team members only see data within the workspace they've been invited to.
- API keys are scoped to a single workspace.

GDPR compliance

AgentStack is GDPR-compliant. Here's what that means for you:

Data export:
You can request a full export of your workspace data at any time. Email **support@agentstack.ai** with the subject line "Data Export Request." We'll provide a downloadable archive within 5 business days.

Right to deletion:
You can delete your workspace and all associated data from **Settings > Account > Delete Workspace**. This permanently removes:
- All agent configurations and customizations.
- All workflows and execution history.
- All API keys and provider keys.
- All team member associations.

Deletion is irreversible. We recommend exporting your data first.

Data processing:
AgentStack processes your data only to operate the platform. We do not sell your data, use it for training AI models, or share it with third parties beyond the model providers you explicitly connect via BYOK.

SOC 2 Type II

SOC 2 Type II certification is in progress. When completed, the audit report will be available upon request for Team plan customers. Email **support@agentstack.ai** to be notified when it's ready.

Deleting your workspace

If you want to remove all your data from AgentStack:

1. Go to **Settings > Account**.

2. Click **Delete Workspace**.
3. Type your workspace name to confirm.
4. Click **Delete permanently**.

This action is irreversible. All data is removed from our systems within 30 days of the deletion request, in compliance with GDPR requirements.

Reporting a security issue

If you discover a security vulnerability, email **security@agentstack.ai**. We take all reports seriously and aim to respond within 24 hours.

TROUBLESHOOTING

Article 19: Troubleshooting Common Errors

Category: Troubleshooting
URL slug: /help/troubleshooting-errors
Last updated: 2026-03-10

Here are the most common errors you'll encounter in AgentStack, what causes them, and how to fix them.

Rate limit hit (429 Too Many Requests)

What it means: You've sent more API requests per minute than your plan allows.

Plan limits:

PLAN	LIMIT
Free	60 requests/min
Pro	300 requests/min

PLAN	LIMIT
Team	1,000 requests/min

How to fix:
1. Wait for the `Retry-After` header value (in seconds), then retry.
2. Add exponential backoff to your API client: wait 1s, then 2s, then 4s between retries.
3. If you're consistently hitting limits, batch your requests or upgrade your plan.

Invalid API key (400 Bad Request)

What it means: The model provider key you configured is invalid, expired, or incorrectly formatted.

How to fix:
1. Go to **Settings > Provider Keys** and check the key for the failing provider.
2. Log in to your provider's dashboard (e.g., platform.openai.com) and verify the key is active.
3. Check for leading or trailing whitespace -- copy-paste errors are the most common cause.
4. If the key has been rotated or revoked at the provider, generate a new one and update it in AgentStack.
5. Re-run the agent or workflow.

Agent timeout (408 Request Timeout)

What it means: An agent step took too long to complete. The default timeout is 5 minutes per step.

How to fix:
1. **Simplify your input.** Large inputs (e.g., entire codebases) take longer to process. Break them into smaller chunks.
2. **Try a different model.** Some models are faster than others. GPT-4o-mini is significantly faster than GPT-4o for most tasks.
3. **Increase the timeout.** In the Workflow Builder, click on the agent step, open **Advanced Settings**, and increase the timeout value.
4. **Check model provider status.** If the provider is experiencing latency issues, the timeout may not be your fault. Check their status page.

Workflow execution failed

What it means: A workflow run stopped before completing all steps. This is a catch-all status that can have several causes.

How to diagnose:
1. Go to **Executions** and click on the failed execution.
2. Expand each step to see which one failed and read the error message.
3. Common causes:
- **Agent not activated.** A step references an agent you haven't activated. Go to the Agent Catalog and activate it.
- **Provider quota exceeded.** Your model provider ran out of credits. See the next section.
- **Invalid variables.** A step expected a variable that wasn't provided by the trigger or a previous step. Check the variable names in your workflow configuration.
- **Step dependency failed.** A downstream step failed because an upstream step produced an error instead of output.

How to fix:
Address the root cause in the failing step, then re-run the workflow from the Executions page by clicking **Retry**.

Provider quota exceeded (402 Payment Required)

What it means: Your model provider (OpenAI, Anthropic, Google) has run out of credits or you've hit a spending limit.

How to fix:
1. Log in to your provider's billing dashboard.
2. Check your current balance and usage.
3. Add credits or increase your spending limit.
4. Return to AgentStack and re-run the failed execution.

This is not an AgentStack billing issue -- it's between you and your model provider. AgentStack passes your key through to the provider and returns whatever response (or error) comes back.

Permission denied (403 Forbidden)

What it means: Your account or API key doesn't have permission to perform the requested action.

How to fix:
1. **If using the API:** Check that your API key has the required scopes. For example, triggering a workflow requires `workflows:write`. See Managing API Keys.
2. **If using the dashboard:** Check your role in the workspace. Members can't delete workflows created by others. Viewers can't run anything. Contact your workspace owner to adjust your role.
3. **If you're the owner and still blocked:** Your plan may not include the feature. For example, API access requires Pro or Team.

Network error

What it means: Your browser or API client couldn't connect to AgentStack.

How to fix:
1. Check your internet connection.
2. Try refreshing the page or retrying the API call.
3. Check status.agentstack.ai for service disruptions.
4. If the problem persists, clear your browser cache and try again.
5. If you're behind a corporate firewall or VPN, ensure `api.agentstack.ai` and `app.agentstack.ai` are not blocked.

Still stuck?

If none of the above resolves your issue:
- Email **support@agentstack.ai** with the execution ID (visible in the Executions page) and a description of what you expected vs. what happened.
- Pro and Team plan users receive priority support.

Article 20: Execution Limits Explained

Category: Troubleshooting
URL slug: /help/execution-limits
Last updated: 2026-03-10

This article explains what counts as an execution, how limits work, and what happens when you reach them.

What counts as 1 execution

One execution = one workflow run, regardless of how many steps the workflow contains.

ACTION	EXECUTIONS USED
Running a 5-step workflow	1
Running a 1-step workflow	1
Test-running a single agent from the Agent Catalog	1

ACTION	EXECUTIONS USED
A webhook triggering a workflow	1
A cron-scheduled workflow running automatically	1
Each individual step within a workflow	0 (steps don't count separately)
Viewing execution results	0
Editing a workflow without running it	0

The key point: **steps are not counted individually.** A workflow with 8 agent steps uses exactly 1 execution.

Monthly limits by plan

PLAN	EXECUTIONS PER MONTH
Free	100
Pro	Unlimited
Team	Unlimited

When does the counter reset?

Your execution counter resets on the **first day of each calendar month** at midnight UTC. If you're on the Free plan and used 95 executions in March, your counter goes back to 0 on April 1.

Checking your current usage

1. Go to **Dashboard > Usage**.
2. You'll see:
   - **Executions used** this month (and the limit, if applicable).
   - **Daily breakdown** as a chart.
   - **Usage by agent** and **usage by workflow** for detailed tracking.

What happens when you hit the limit

On the Free plan, when you reach 100 executions in a month:

1. **New workflow runs are paused.** You'll see a banner: "You've reached your monthly execution limit. Upgrade to Pro for unlimited executions, or wait until {reset_date}."
2. **Existing data is preserved.** Your workflows, agent configurations, and execution history remain exactly as they are. Nothing is deleted.
3. **API calls that trigger executions return `429`.** The error message specifies it's an execution limit, not a rate limit.
4. **Scheduled (cron) workflows skip their next run.** They resume automatically when the counter resets.

Executions in progress when you hit the limit

If a workflow is already running when your counter hits 100, **it is allowed to complete.** AgentStack does not cancel in-progress work. Only new executions are blocked.

This means your actual usage might read "101" or "102" in rare cases -- that's by design. We don't interrupt running workflows.

How to get more executions

You have three options:

1. **Wait for the reset.** Your counter resets on the 1st of next month.
2. **Upgrade to Pro ($49/mo).** Unlimited executions, plus all 82 agents, API access, and custom agents.
3. **Optimize your workflows.** If you're running the same workflow repeatedly with minor variations, consider batching inputs into a single run.

Execution tracking for teams

On Pro and Team plans, execution counts are tracked at the **workspace level**, not per user. If your workspace has 5 members and they each run 100 workflows, the workspace has used 500 executions total -- which is fine on Pro and Team since both are unlimited.

Usage breakdowns by team member are available in **Dashboard > Usage > By Member**.